Создаём свой Lisp

Изучаем C и создаём собственный язык программирования

Дэниел Холден



Русский перевод

Оригинал: Build Your Own Lisp. Перевод текста: CC BY-NC-SA 3.0. Исходный код: BSD 3-Clause. Источники иллюстраций и благодарности.

Содержание

1. Введение
2. Установка
3. Основы
4. Интерактивная командная строка
5. Языки
6. Разбор
7. Вычисление
8. Обработка ошибок
9. S-выражения
10. Q-выражения
11. Переменные
12. Функции
13. Условные выражения
14. Строки
15. Стандартная библиотека
16. Дополнительные проекты
17. Парсер своими руками

Введение • Глава 1

Об этой книге

Из этой книги вы узнаете язык программирования C и одновременно научитесь создавать *свой собственный язык программирования* — минималистичный Lisp, уложившись менее чем в 1000 строк кода! Для части первоначальной работы мы воспользуемся библиотекой, так что с количеством строк я немного хитрю. Но весь остальной код будет написан с нуля, и к концу книги вы действительно создадите небольшой, но мощный Lisp.

Меня вдохновили другие руководства, в которых шаг за шагом создаётся язык программирования с нуля. Я написал эту книгу, чтобы показать: такой увлекательный творческий проект — отличный способ выучить язык, доступный не только опытным программистам и не ограниченный абстрактными высокоуровневыми языками.

Многие хотят изучить C, но не знают, с чего начать. Теперь отговорок нет. Если вы пройдёте эту книгу, обещаю: в худшем случае у вас появится новый классный язык программирования, с которым можно поиграть, а в лучшем — вы ещё и станете опытным программистом на C!

Для кого эта книга

Эта книга для всех, кто хочет выучить C или когда-либо задумывался о создании собственного языка программирования. Она не подходит в качестве самой первой книги по программированию, но любой человек хотя бы с небольшим опытом программирования на любом языке найдёт в ней что-то новое и интересное.

Я постарался сделать книгу как можно дружелюбнее к начинающим. Я особенно рад новичкам, ведь им предстоит столько открытий! Но книга может оказаться для них непростой: мы рассмотрим множество новых понятий и, по сути, будем изучать сразу два новых языка программирования.



Ада Лавлейс • Типичный «бро-граммист».

Обратившись за помощью, вы можете столкнуться с нетерпением. Вместо помощи некоторые предпочтут рассказать, как много *они* знают о предмете. Опытные программисты могут сказать, что вы неправы. В их тоне может читаться намёк: вам лучше остановиться, пока вы не навязали миру свой плохой код.

После нескольких таких встреч вы можете решить, что вы *не программист*, или что вам *на самом деле не нравится программировать*, или что вы просто *ничего не понимаете*.

Возможно, когда-то вам нравилась идея создать свой язык программирования, но теперь вы решили, что она слишком абстрактна и *больше вас не интересует*. Вы обращаетесь к другим увлечениям, а всё, что могло бы принести радость открытия и игры, становится препятствием.

За это я могу только извиниться. Программисты бывают враждебными, самоуверенными, высокомерными, неуверенными в себе и агрессивными. Такому поведению нет оправдания. Знайте: я на вашей стороне. Никто не *схватывает всё* с первого раза. Всем бывает трудно, все сомневаются в своих способностях. Пожалуйста, не сдавайтесь и не позволяйте отнять у вас радость творчества. Гордитесь тем, что создаёте, каким бы оно ни было. Такие люди, как я, не хотят, чтобы вы перестали программировать. Мы хотим услышать ваш голос и то, что вы хотите сказать.

Зачем изучать C

C — один из самых популярных и влиятельных языков программирования в мире. Это основной язык разработки Linux; он широко использовался при создании OS X и в некоторой степени Microsoft Windows. Его применяют и в микрокомпьютерах. Ваш холодильник и автомобиль, вероятно, тоже работают на нём. В современной разработке можно обойтись без самого C, но не без его наследия. Всем, кто хочет сделать карьеру в разработке программного обеспечения, разумно изучить C.



Холодильник • Типичный пользователь C

Но C — не только о разработке и карьере. C — это **свобода**. Он прославился благодаря технологиям сотрудничества и свободы — Unix, Linux и движению свободного программного обеспечения. Он воплощает идею личной свободы в вычислениях и призывает вас взять под контроль технологии, влияющие на вашу жизнь.

Сегодня, когда технологии могущественнее, чем когда-либо, это особенно важно.

Идеология свободы отражена в самой природе C. Он мало что скрывает от вас, включая свои недостатки и изъяны. Он почти ничего не запрещает, в том числе и возможность ужасным образом сломать собственную программу. Программируя на C, вы стоите не на проторенной тропе, а перед целым пространством решений, и C предлагает вам самим выбрать путь.

C — это также язык удовольствия и обучения. До того как за это взялись массовые СМИ, у нас было слово для такого подхода: *хакинг*. Это философия, прославляющая занимательное и остроумное. Она не имеет отношения к незаконному доступу к чужим компьютерам. Хакинг — философия исследования, самовыражения, раздвигания границ и нарушения правил. Она противостоит иерархии и бюрократии и превозносит личность.

Хакинг манит удовольствием, знаниями и славой. Он обещает, что с компьютером и доступом в интернет вы можете изменить мир.

Стремиться освоить C — значит ценить мощное, остроумное и свободное; стремиться стать программистом, у которого под рукой огромные возможности технологий и ответственность использовать их на благо мира.

Как изучать C

Никуда не деться от того, что C — трудный язык. В нём много непривычных понятий, и он не пытается помочь новичку. В этой книге я *не буду* подробно разбирать синтаксис языка, написание циклов и условных операторов.

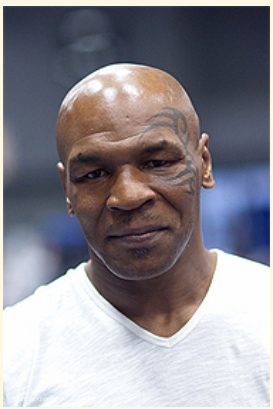
Зато я покажу вам, как создать на C программу, применимую ***в реальном мире***. Такой подход всегда сложнее для читателя, но, надеюсь, научит вас многим неочевидным вещам, недоступным при традиционном обучении. Я не могу гарантировать, что эта книга сделает вас уверенным пользователем C. Но обещаю: эти 1000 строк будут насыщены содержанием, и вы узнаете *что-то* полезное.

Книга состоит из 16 коротких глав. Как их проходить — решать вам. Вполне возможно осилить её за выходные или продвигаться медленнее, читая по главе-две каждый вечер в течение недели. Это не должно занять много времени и, надеюсь, пробудит у вас желание развивать свой язык дальше.

Зачем создавать Lisp

Язык, который мы создадим в этой книге, относится к Lisp. Это семейство языков программирования, характерная особенность которых — представление всех вычислений в виде *списков*. Возможно, звучит страшнее, чем есть на самом деле. В действительности языки Lisp очень просты, самобытны и мощны.

Создание Lisp — замечательный проект по многим причинам. Он позволяет поставить себя на место создателей языков и оценить весь процесс программирования — от языка до машины. Он знакомит с функциональным программированием и новыми взглядами на вычисления. Получившийся продукт становится основой для будущих идей и разработок, отправной точкой для новых экспериментов. Невозможно по-настоящему оценить творческий подход и изобретательность в программировании и информатике, не исследовав сами языки.



Майк Тайсон • Типичный пользователь Lisp

Разновидность Lisp, которую мы создадим, я придумал специально для этой книги. Я стремился к минимализму, простоте и ясности и успел к ней привязаться. Надеюсь, она понравится и вам. По концепции, синтаксису и реализации этот Lisp во многом отличается от других известных представителей семейства. Настолько, что я наверняка получу письма от программистов на Lisp с объяснениями, что это *не Lisp*, поскольку он *не делает то-то, не имеет того-то или выглядит не так*.

Я сделал этот Lisp другим не для того, чтобы запутать новичков, а потому, что быть другим — хорошо.

Если вы хотите изучить семантику и поведение обычных Lisp и научиться на них программировать, эта книга, возможно, не для вас. Вместо этого она предлагает новые и уникальные идеи, самовыражение, творчество и удовольствие. Какими бы ни были ваши цели, сразу примите это предупреждение: не всё, что я говорю, будет объективно правильным или истинным! Вам предстоит решить это самостоятельно.

Ваш собственный Lisp

Лучше всего читать эту книгу, как и сказано в названии, создавая *свой собственный* Lisp. Если вы достаточно уверены в себе, добавляйте свои возможности и вносите изменения. Ваш Lisp должен соответствовать вам и вашей философии. На протяжении книги я буду давать объяснения и делиться мыслями, но вместе с ними приведу *много* кода. Поэтому можно просто копировать каждый фрагмент в программу, не особенно разбираясь в нём. *Пожалуйста, не делайте этого!*

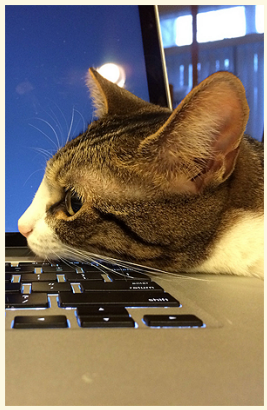
Набирайте каждый пример кода самостоятельно. Это называется *трудным путём*. Не потому, что он технически сложен, а потому, что требует дисциплины. Идя трудным путём, вы поймёте логику того, что набираете. В идеале всё встанет на свои места, пока вы последовательно вводите символ за символом. При чтении вы можете интуитивно чувствовать, почему код *выглядит* правильным или что в нём *может* происходить, но это не всегда превращается в настоящее понимание, пока вы не напишете код сами!

В идеальном мире вы использовали бы мой код как справочник — инструкцию и руководство по созданию языка программирования вашей мечты. На практике это не всегда возможно или удобно. Но основная идея остаётся прежней: если хотите что-то изменить — меняйте.

Установка • Глава 2

Подготовка

Прежде чем начать программировать на C, нам нужно установить несколько вещей и настроить рабочую среду. Поскольку C — очень универсальный язык, это должно быть довольно просто. По сути, нам необходимы две основные вещи: *текстовый редактор* и *компилятор*.



Кот • Устанавливайте на свой страх и риск.

Текстовый редактор

Текстовый редактор — программа для редактирования текстовых файлов, подходящая для написания кода.

В **Linux** я рекомендую gedit. Подойдёт и любой другой простой редактор, установленный в вашем дистрибутиве. Если вы пользуетесь Vim или Emacs, они тоже вполне годятся. Пожалуйста, не используйте интегрированную среду разработки (IDE): для такого небольшого проекта она не нужна и не поможет понять, что происходит.

На **Mac** можно использовать простой редактор TextWrangler. Если предпочитаете другой, это нормально, но, пожалуйста, не используйте XCode для редактирования текста. Проект небольшой, и IDE не поможет вам разобраться в происходящем.

В **Windows** я предпочитаю Notepad++. Можете выбрать другой редактор. Пожалуйста, *не используйте Visual Studio*: полноценной поддержки программирования на C в ней нет, и вы столкнётесь со многими проблемами.

Компилятор

Компилятор преобразует исходный код на C в программу, которую может запустить компьютер. Порядок установки зависит от операционной системы.

Для компиляции и запуска программ на C вам также понадобятся базовые навыки работы с командной строкой. Я не буду их здесь объяснять и предполагаю, что вы хотя бы немного с ней знакомы. Если это не так, найдите руководство по командной строке для своей операционной системы.

В **Linux** компилятор можно установить с помощью пакетов. В Ubuntu или Debian всё необходимое устанавливается командой `sudo apt-get install build-essential`. В Fedora или похожем дистрибутиве используйте `su -c "yum groupinstall development-tools"`.

На **Mac** установите последнюю версию XCode от Apple. Если не знаете как, найдите в интернете инструкции по запросу «installing xcode». Затем установите *Command Line Tools*. В Mac OS X 10.9 это можно сделать командой `xcode-select --install`. В более ранних версиях откройте настройки XCode, раздел Downloads и выберите установку *Command Line Tools*.

В **Windows** скачайте и установите MinGW. Затем добавьте компилятор и другие программы в системную переменную `PATH`. Следуя этой инструкции, добавьте в конец переменной `PATH` значение `;C:\MinGW\bin`. Если переменной нет, её можно создать. Возможно, потребуется перезапустить `cmd.exe`. После этого компилятор можно будет запускать из командной строки `cmd.exe`. Также установятся другие программы, благодаря которым `cmd.exe` будет работать подобно командной строке Unix.

Проверка компилятора

Чтобы проверить правильность установки компилятора C, введите в командной строке:

```
cc --version
```

Если выводится информация о версии компилятора, скорее всего, он установлен правильно. Всё готово! Если появляется сообщение о неизвестной или не найденной команде, установка ещё не завершена. Возможно, нужно перезапустить командную строку или компьютер.

> **Разные команды компилятора.**
>
> В некоторых системах, например Windows, команда компилятора может называться иначе, например `gcc`. Попробуйте её, если система не находит `cc`.

Привет, мир!

Теперь, когда среда настроена, откройте текстовый редактор и введите следующую программу. Создайте каталог для работы по этой книге и сохраните файл под именем `hello_world.c`. Это ваша первая программа на C!

```
#include <stdio.h>

int main(int argc, char** argv) {
  puts("Hello, world!");
  return 0;
}
```

Поначалу здесь может быть мало что понятно. Попробую объяснить всё по шагам.

В первой строке мы *подключаем* так называемый *заголовочный файл*. Это позволяет использовать функции из `stdio.h` — стандартной библиотеки ввода-вывода, входящей в

состав C. Одна из них — функция `puts`, которая встречается далее в программе.

Затем мы *объявляем* функцию `main`. Она возвращает значение типа `int` и принимает на вход `int` с именем `argc` и `char**` с именем `argv`. Эта функция должна присутствовать во всех программах на C: с неё начинается выполнение.

Внутри `main` функция `puts` *вызывается* с аргументом `"Hello, world!"`. Она выводит в командную строку сообщение `Hello, world!`. Название `puts` сокращено от *put string* — «вывести строку». Второй оператор внутри функции — `return 0;`. Он завершает `main` и возвращает `0`. Возврат нуля программой на C означает, что при её выполнении ошибок не возникло.

Компиляция

Прежде чем запустить программу, нужно её скомпилировать. В результате получится *исполняемый файл*, который можно запустить на компьютере. Откройте командную строку и перейдите в каталог с файлом `hello_world.c`. Затем выполните команду:

```
cc -std=c99 -Wall hello_world.c -o hello_world
```

Она компилирует код из `hello_world.c`, сообщает обо всех предупреждениях и записывает программу в новый файл `hello_world`. Флаг `-std=c99` сообщает компилятору, какую *версию*, или *стандарт*, C мы используем. Это позволяет проверить соответствие кода стандарту, чтобы им могли воспользоваться люди с другими операционными системами или компиляторами.

Если всё прошло успешно, в текущем каталоге появится выходной файл. Запустите его командой `./hello_world` (в Windows достаточно `hello_world`). Если всё правильно, вы увидите дружелюбное сообщение `Hello, world!`.

Поздравляю! Вы только что скомпилировали и запустили свою первую программу на C.

Ошибки

Если в программе на C есть проблемы, компиляция может завершиться неудачей. Причиной бывают как простые синтаксические ошибки, так и более сложные, которые труднее понять.

Иногда сообщение компилятора понятно само по себе. Если нет, попробуйте найти его в интернете: поищите краткое объяснение и разберитесь, как исправить ошибку. Помните: многие люди до вас сталкивались с точно такими же трудностями.

Иногда множество ошибок компилятора вызывается одной причиной. Всегда разбирайте их по порядку, от первой к последней.

Иногда программа компилируется, но аварийно завершается при запуске. Отлаживать программы на C в такой ситуации трудно: это может быть целым искусством, выходящим



Ярость • Не лучший способ отладки

далеко за рамки книги.

Начинающим прежде всего стоит выводить много информации во время работы программы. Так можно определить, какая именно часть кода работает неправильно и что именно идёт не так. Это *активный* метод отладки, и в этом его важность. Пока вы что-то делаете, а не просто смотрите на код, процесс менее мучителен и соблазн сдаться меньше.

Более уверенные пользователи могут отлаживать программы на C с помощью `gdb`. Эта программа бывает сложна в использовании, но она очень мощная и может дать ценную информацию о том, где и что пошло не так. Инструкции по работе с `gdb` можно найти в интернете.

На **Mac** последние версии OS X поставляются без `gdb`. Вместо него можно использовать `lldb`, который выполняет примерно ту же работу.

В **Linux** и на **Mac** для поиска утечек памяти и других неприятных ошибок можно применять `valgrind`. Этот инструмент способен сэкономить часы и даже дни отладки. Освоить его несложно, поэтому очень рекомендую с ним познакомиться. Инструкции можно найти в интернете.

Документация

В примерах этой книги вам могут встретиться незнакомые функции. Если захотите узнать, что они делают, обратитесь к онлайн-документации стандартной библиотеки. В ней описаны все её функции, их назначение и применение.

Справочный код

Зачем нужен этот раздел?

Здесь я привожу ссылки на код, написанный для данной главы. После её завершения ваш код, вероятно, будет похож на мой. Если объяснение оказалось неясным, можно обратиться к нему как к справочному материалу.

Если вы обнаружили ошибку, пожалуйста, не копируйте мой код в проект. Попробуйте найти ошибку самостоятельно, используя мой код как подсказку о том, что и где может быть неверно.

Исходный код: hello_world.c.

Дополнительные задания

Зачем нужен этот раздел?

Здесь я перечисляю задания, которые можно выполнить ради удовольствия и обучения.

Попробуйте решить хотя бы некоторые из них. Одни будут трудными, другие — гораздо проще, поэтому не переживайте, если не справитесь со всеми. Некоторые могут оказаться даже невыполнимыми!

Для многих потребуется поиск в интернете. Это неотъемлемая часть изучения нового языка, и избегать её не стоит. Умение самостоятельно учиться — один из ценнейших навыков программиста.

- › Замените приветствие `Hello World!` в программе на другое.
- › Что произойдёт, если не определить функцию `main`?
- › Найдите функцию `puts` в онлайн-документации.
- › Узнайте, как пользоваться `gdb`, и запустите его со своей программой.

Основы • Глава 3

Обзор

В этой главе я подготовил краткий обзор основных возможностей C. Самих *возможностей* в C очень немного, а синтаксис относительно прост. Но это не значит, что язык лёгок: вся глубина скрыта под поверхностью. Поэтому сейчас мы быстро рассмотрим *возможности* и *синтаксис*, а в дальнейшем изучим их подробнее.

Цель главы — дать всем общую основу. Тем, кто совсем не знаком с C, стоит уделить ей время; имеющим некоторый опыт, возможно, будет достаточно бегло её просмотреть и вернуться при необходимости.



Программки • Пригодятся в театре.

Программы

Программа на C состоит только из *определений функций* и *определений структур*.

Таким образом, исходный файл — это список *функций* и *типов*. Функции могут вызывать друг друга и самих себя, а также использовать любые объявленные или встроенные в язык типы данных.

Можно вызывать функции из других библиотек и использовать их типы данных. Так в программировании на C и накапливаются уровни сложности.

Как мы видели в предыдущей главе, выполнение программы на C всегда начинается с функции `main`. Из неё вызываются другие функции, которые выполняют все необходимые действия.

Переменные

Функции в C работают с *переменными* — данными, которым мы даём имена.

У каждой переменной в C есть явно указанный *тип*. Типы объявляются нами или встроены в язык. Чтобы объявить переменную, пишут название типа, затем имя и при желании присваивают значение с помощью `=`. Такое объявление — *оператор*; все операторы в C заканчиваются точкой с запятой `;`.

Создать переменную типа `int` с именем `count` можно так:

```
int count;
```

А чтобы объявить её и присвоить значение:

```
int count = 10;
```

Вот описания и примеры некоторых встроенных типов.

`void`	Пустой тип	
`char`	Один символ или байт	`char last_initial = 'H';`
`int`	Целое число	`int age = 23;`
`long`	Целое число большего диапазона	`long age_of_universe = 13798000000;`
`float`	Дробное число	`float liters_per_pint = 0.568f;`
`double`	Дробное число повышенной точности	`double speed_of_swallow = 0.01072896;`

Объявление функций

Функция — это вычисление, которое обрабатывает переменные и может изменять состояние программы. Она принимает на вход некоторые переменные и возвращает одну переменную.

Чтобы объявить функцию, укажите тип возвращаемой переменной, имя функции и затем в скобках через запятую список входных переменных. Содержимое функции помещается в фигурные скобки `{}` и состоит из выполняемых ею операторов, заканчивающихся точкой с запятой `;`. Оператор `return` завершает функцию и возвращает переменную.

Например, функция, принимающая две переменные `int` с именами `x` и `y` и складывающая их, может выглядеть так:

```
int add_together(int x, int y) {
  int result = x + y;
  return result;
}
```

Для вызова функции пишут её имя и в скобках через запятую передают аргументы. Например, чтобы вызвать функцию выше и сохранить результат в переменной `added`:

```
int added = add_together(10, 18);
```

Объявление структур

Структуры используются для объявления новых *типов*. Структура объединяет несколько переменных в один пакет.

С их помощью можно представлять более сложные типы данных. Например, для точки на плоскости создадим структуру `point`, объединяющую два дробных значения `float` с именами `x` и `y`. Для объявления структуры можно использовать ключевые слова `struct` и `typedef`:

```
typedef struct {
  float x;
  float y;
} point;
```

Это определение нужно поместить перед всеми функциями, которые его используют. Полученный тип ничем не отличается от встроенных и используется так же. Для доступа к отдельному полю пишут точку `.`, а затем имя поля, например `x`.

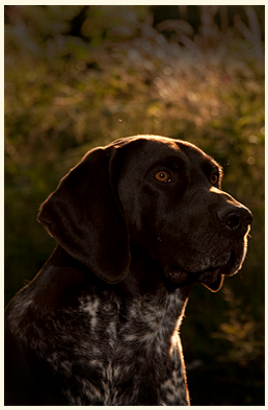
```
point p;
p.x = 0.1;
p.y = 10.0;

float length = sqrt(p.x * p.x + p.y * p.y);
```

Указатели

Указатель — разновидность обычного типа, в записи которого после имени ставится звёздочка. Например, *указатель на целое число* записывается как `int*`. Мы уже видели указатель `char** argv`. Это *указатель на указатели на символы*, используемый как входной аргумент функции `main`.

Указатели применяются для многих целей, например для строк и списков. Это сложная часть C; в следующих главах мы рассмотрим её гораздо подробнее. Пока они нам не понадобятся, поэтому достаточно знать об их существовании и уметь их узнавать. Не позволяйте им вас напугать!



Пойнтер • Короткошёрстный указатель

Строки

В C строки представлены указателем типа `char*`. Внутри они хранятся как последовательность символов, последний из которых — специальный символ, называемый *нулевым терминатором*. Строки — сложная и важная часть C; в следующих главах мы научимся эффективно с ними работать.

Строку можно записать буквально, заключив текст в кавычки. Мы уже делали это в предыдущей главе со строкой `"Hello, World!"`. Пока запомните: увидев `char*`, можно читать это как *строка*.

Условия

Условные операторы позволяют выполнять код только при соблюдении определённых условий.

Для этого используется оператор `if`: после него в круглых скобках записывают условие, а в фигурных — код для выполнения. За `if` может следовать необязательный оператор `else` с другим кодом в фигурных скобках. Он выполняется, если условие ложно.

Проверить несколько условий можно с помощью логических операторов `||` (*или*) и `&&` (*и*).

В скобках условного оператора любое значение, отличное от `0`, считается истинным. Важно это помнить: многие условия используют такую неявную проверку.

Например, чтобы проверить, что целое `x` больше `10` и меньше `100`, напишем:

```
if (x > 10 && x < 100) {
  puts("x is greater than 10 and less than 100!");
} else {
  puts("x is less than 11 or greater than 99!");
}
```

Циклы

Циклы позволяют повторять код, пока некоторое условие не станет ложным или счётчик не достигнет заданного значения.

В C есть два основных вида циклов. Первый — `while`. Он повторно выполняет блок кода, пока условие не станет ложным. После `while` записывается условие в круглых скобках, а затем код в фигурных. Например, обратный отсчёт от `10` до `1` можно записать так:

```
int i = 10;
while (i > 0) {
  puts("Loop Iteration");
  i = i - 1;
}
```

Второй вид — цикл `for`. Вместо одного условия он требует три выражения, разделённых точками с запятой `;`: *инициализацию*, *условие* и *изменение счётчика*. Инициализация выполняется перед началом цикла. Условие проверяется перед каждой итерацией; если оно ложно, цикл завершается. Изменение счётчика выполняется в конце каждой итерации. Эти циклы часто используют для счёта, поскольку они компактнее `while`.

Например, цикл для счёта от `0` до `9` можно записать так. Оператор `++` увеличивает переменную `i`.

```
for (int i = 0; i < 10; i++) {
  puts("Loop Iteration");
}
```

Дополнительные задания

- › Выведите `Hello World!` пять раз с помощью цикла `for`.
- › Сделайте то же с помощью `while`.
- › Объявите функцию, которая выводит `Hello World!` заданное число `n` раз. Вызовите её из `main`.
- › Какие ещё встроенные типы существуют?
- › Какие есть условные операторы помимо *больше* `>` и *меньше* `<`?
- › Какие математические операторы есть помимо *сложения* `+` и *вычитания* `-`?
- › Что такое оператор `+=` и как он работает?
- › Что такое цикл `do` и как он работает?
- › Что такое оператор `switch` и как он работает?
- › Что делает ключевое слово `break`?
- › Что делает ключевое слово `continue`?
- › Что именно делает ключевое слово `typedef`?

Интерактивная командная строка • Глава 4

Чтение, вычисление, вывод

Создавая язык программирования, нам понадобится способ взаимодействовать с ним. В C используется компилятор: можно изменить программу, заново скомпилировать её и запустить. Было бы здорово придумать что-то получше и работать с языком динамически. Тогда мы сможем быстро проверять его поведение в разных условиях. Для этого создадим так называемую *интерактивную командную строку*.



Рептилия • Чем-то похожа на REPL

Эта программа запрашивает ввод пользователя и отвечает на него сообщением. С её помощью проще всего проверять наш язык и наблюдать за его поведением. Такую систему также называют *REPL* — сокращение от *read*, *evaluate*, *print*, *loop* (цикл чтения, вычисления и вывода). Это распространённый способ взаимодействия с языком программирования; возможно, вы уже пользовались им, например в *Python*.

Прежде чем создавать полноценный *REPL*, начнём с более простой системы: она запросит ввод и сразу повторит его. Позже мы сможем расширить её так, чтобы она разбирала и вычисляла введённый текст как настоящую программу на Lisp.

Интерактивная командная строка

Для начала напишем цикл, который постоянно выводит сообщение и ждёт ввода. Получить ввод можно функцией `fgets`, которая читает текст до символа новой строки. Нам также понадобится место для его хранения; для этого объявим буфер ввода постоянного размера.

Сохранив ввод, мы сможем вывести его обратно пользователю с помощью функции `printf`.

```
#include <stdio.h>

/* Declare a buffer for user input of size 2048 */
static char input[2048];

int main(int argc, char** argv) {

  /* Print Version and Exit Information */
  puts("Lispy Version 0.0.0.0.1");
  puts("Press Ctrl+c to Exit\n");

  /* In a never ending loop */
  while (1) {

    /* Output our prompt */
```

```
    fputs("lispy> ", stdout);

    /* Read a line of user input of maximum size 2048 */
    fgets(input, 2048, stdin);

    /* Echo input back to user */
    printf("No you're a %s", input);
  }

  return 0;
}
```

Что это за светло-зелёный текст?

Код выше содержит *комментарии*. Это участки между символами `/*` и `*/`, которые компилятор игнорирует. Они объясняют читателю, что происходит. Обращайте на них внимание!

Рассмотрим программу подробнее.

Строка `static char input[2048];` объявляет глобальный массив из 2048 символов. Это зарезервированный блок данных, доступный из любой части программы. В нём мы будем хранить текст, введённый пользователем в командной строке. Ключевое слово `static` ограничивает область видимости переменной текущим файлом, а `[2048]` задаёт её размер.

Бесконечный цикл записывается как `while (1)`. В условии `1` всегда считается истиной, поэтому команды внутри будут выполняться бесконечно.

Для вывода приглашения используется `fputs` — разновидность `puts`, которая не добавляет символ новой строки. Для получения ввода из командной строки используется `fgets`. Обеим функциям нужен файл для записи или чтения. Мы передаём специальные переменные `stdin` и `stdout`, объявленные в `<stdio.h>`: они представляют ввод и вывод командной строки. Получив такую переменную, `fgets` ждёт, пока пользователь введёт строку, и сохраняет её в буфере `input` вместе с символом новой строки. Чтобы функция не прочитала слишком много данных, ей также передаётся размер буфера `2048`.

Чтобы повторить сообщение пользователю, используем `printf`. Эта функция выводит сообщения из нескольких элементов, сопоставляя аргументы с шаблонами в строке. В нашем случае шаблон `%s` заменяется следующим переданным аргументом, интерпретируемым как строка.

Подробнее о различных шаблонах см. в документации `printf`.

Откуда мне знать о функциях вроде `fgets` и `printf`?

Не всегда очевидно, какие стандартные функции существуют и когда их применять. Чтобы узнать, решена ли ваша задача библиотечной функцией, нужен опыт.

К счастью, стандартная библиотека C очень мала, и почти всю её можно изучить на практике. Если задача кажется довольно простой или фундаментальной, стоит заглянуть в справочную документацию и проверить, нет ли в библиотеке подходящей функции.

Компиляция

Скомпилировать программу можно той же командой, что и во второй главе.

```
cc -std=c99 -Wall prompt.c -o prompt
```

После компиляции попробуйте запустить её. Закончив работу, нажмите `Ctrl+c`. Если всё правильно, программа будет работать примерно так:

```
Lispy Version 0.0.0.0.1
Press Ctrl+c to Exit

lispy> hello
No you're a hello
lispy> my name is Dan
No you're a my name is Dan
lispy> Stop being so rude!
No you're a Stop being so rude!
lispy>
```

Редактирование ввода

В Linux или Mac при попытке исправить введённый текст клавишами со стрелками вы заметите странное поведение.

```
Lispy Version 0.0.0.0.3
Press Ctrl+c to Exit

lispy> hel^[[D^[[C
```

Вместо перемещения курсора стрелки создают странные символы `^[[D` или `^[[C`. А нам нужно перемещаться по строке, удалять и изменять текст, если мы ошиблись.

В Windows это поведение доступно по умолчанию. В Linux и Mac его обеспечивает библиотека `editline`. Нам нужно заменить вызовы `fputs` и `fgets` функциями этой библиотеки.

Если вы работаете в Windows и хотите скорее продолжить, можете пропустить несколько следующих разделов и перейти к концу главы.

Использование Editline

Библиотека `editline` предоставляет две нужные нам функции: `readline` и `add_history`. Первая считывает ввод, позволяя его редактировать. Вторая сохраняет историю ввода, чтобы к ней можно было возвращаться стрелками вверх и вниз.

Заменим вызовы `fputs` и `fgets` этими функциями:

```
#include <stdio.h>
#include <stdlib.h>

#include <editline/readline.h>
#include <editline/history.h>

int main(int argc, char** argv) {

  /* Print Version and Exit Information */
  puts("Lispy Version 0.0.0.0.1");
  puts("Press Ctrl+c to Exit\n");

  /* In a never ending loop */
  while (1) {

    /* Output our prompt and get input */
    char* input = readline("lispy> ");

    /* Add input to history */
    add_history(input);

    /* Echo input back to user */
    printf("No you're a %s\n", input);

    /* Free retrieved input */
    free(input);

  }

  return 0;
}
```

Мы *подключили* несколько новых *заголовков*. Директива `#include <stdlib.h>` даёт доступ к функции `free`, используемой ниже. Также добавлены `#include <editline/readline.h>` и `#include <editline/history.h>`, предоставляющие функции `editline`: `readline` и `add_history`.

Вместо вывода приглашения и чтения через `fgets` мы выполняем оба действия одним вызовом `readline`. Результат передаём в `add_history`, чтобы сохранить его, а затем, как и прежде, выводим через `printf`.

В отличие от `fgets`, функция `readline` удаляет завершающий символ новой строки, поэтому его нужно добавить в `printf`. Кроме того, полученный от `readline` ввод нужно освободить с помощью `free`. Дело в том, что `fgets` записывает данные в существующий буфер, а `readline` при вызове выделяет новую память. Когда нужно освобождать память, мы подробно рассмотрим в следующих главах.

Компиляция с Editline

Если сразу попытаться скомпилировать программу прежней командой, возникнет ошибка: сначала нужно установить библиотеку `editline`.

```
fatal error: editline/readline.h: No such file or directory #include <editline/readline.h>
```

На **Mac** библиотека `editline` входит в *Command Line Tools*. Инструкции по установке приведены в главе 2. При этом всё ещё может возникнуть ошибка из-за отсутствия заголовка истории. В таком случае удалите строку `#include <editline/history.h>`: возможно, этот заголовок не нужен.

В **Linux** установите *editline* командой `sudo apt-get install libedit-dev`. В Fedora можно использовать `su -c "yum install libedit-dev*"`.

После установки снова попробуйте скомпилировать программу. На этот раз появится другая ошибка:

```
undefined reference to `readline'
undefined reference to `add_history'
```

Это означает, что программа не *связана* с `editline`. Процесс *компоновки* позволяет компилятору встроить вызовы `editline` в программу. Чтобы выполнить компоновку, добавьте флаг `-ledit` в команду компиляции перед флагом выходного файла:

```
cc -std=c99 -Wall prompt.c -ledit -o prompt
```

Запустите программу и убедитесь, что теперь можно редактировать ввод во время набора.

> **Всё ещё не работает!**
>
> В разных системах установка, подключение заголовков и компоновка с `editline` могут немного отличаться. Например, в Arch Linux заголовок истории называется `histedit.h`. Если возникли трудности, найдите инструкции по установке и использованию `editline` для вашего дистрибутива. Если это не помогло, поищите инструкции для библиотеки `readline`, которая может заменить editline. На Mac её можно установить через HomeBrew или MacPorts.

Препроцессор C

Для такого небольшого проекта различия в программировании под разные операционные системы могут быть приемлемы. Но если я захочу отправить исходный код другу с другой системой, чтобы он помог мне, возникнут проблемы. В идеале исходный код должен компилироваться где угодно и на любом компьютере. Это общая проблема C, называемая *переносимостью*. Простое и правильное решение есть не всегда.

Но в C есть механизм, который помогает решить эту проблему: *препроцессор*.



Осьминог • Почти как октоторп

Препроцессор запускается перед компилятором. Он служит нескольким целям, и мы уже пользовались им, сами того не зная. Любая строка, начинающаяся с символа октоторпа `#` (проще говоря, решётки), является командой препроцессора. Мы использовали его для *подключения* заголовочных файлов, получая доступ к функциям стандартной и других библиотек.

Кроме того, препроцессор может определить, в какой операционной системе компилируется код, и в зависимости от этого выдавать разный код.

Именно так мы его и используем. В Windows препроцессор выдаст подготовленные мной заменители функций `readline` и `add_history`; в противном случае он подключит заголовки `editline`.

Чтобы указать нужный код, обрамим его директивами препроцессора `#ifdef`, `#else` и `#endif`. Они похожи на `if`, который выполняется до компиляции. Если условие истинно, используется содержимое между первым `#ifdef` и следующим `#else`; иначе — содержимое между `#else` и завершающим `#endif`. Окружив ими заменители функций и заголовки editline, мы получим код, который должен компилироваться в Windows, Linux и Mac.

```
#include <stdio.h>
#include <stdlib.h>

/* If we are compiling on Windows compile these functions */
#ifdef _WIN32
#include <string.h>

static char buffer[2048];

/* Fake readline function */
char* readline(char* prompt) {
  fputs(prompt, stdout);
  fgets(buffer, 2048, stdin);
  char* cpy = malloc(strlen(buffer)+1);
  strcpy(cpy, buffer);
  cpy[strlen(cpy)-1] = '\0';
  return cpy;
}

/* Fake add_history function */
void add_history(char* unused) {}

/* Otherwise include the editline headers */
#else
#include <editline/readline.h>
#include <editline/history.h>
#endif

int main(int argc, char** argv) {

  puts("Lispy Version 0.0.0.0.1");
  puts("Press Ctrl+c to Exit\n");
```

```
  while (1) {

    /* Now in either case readline will be correctly defined */
    char* input = readline("lispy> ");
    add_history(input);

    printf("No you're a %s\n", input);
    free(input);

  }

  return 0;
}
```

Справочный код

Исходный код: prompt_unix.c, prompt_windows.c, prompt.c.

Дополнительные задания

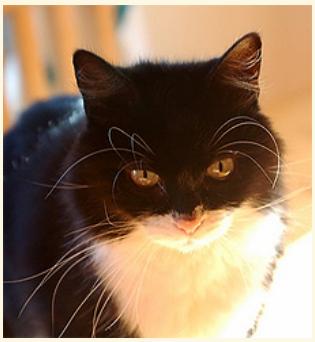
- › Замените приглашение `lispy>` на любое другое.
- › Измените ответ, который выводится пользователю.
- › Добавьте сообщение к информации о версии и выходе.
- › Что означает `\n` в этих строках?
- › Какие ещё шаблоны можно использовать с `printf`?
- › Что произойдёт, если передать `printf` переменную, не соответствующую шаблону?
- › Что делает директива препроцессора `#ifndef`?
- › Что делает директива `#define`?
- › Если в Windows определён `_WIN32`, что определено в Linux или Mac?

Языки • Глава 5

Что такое язык программирования?

Язык программирования очень похож на естественный язык. У него есть структура и правила, определяющие, какие высказывания допустимы, а какие нет. Читая и записывая текст на естественном языке, мы неосознанно изучаем эти правила; то же верно и для языков программирования. Мы используем правила, чтобы понимать других и создавать собственную речь или код.

В 1950-х годах лингвист Ноам Хомский формализовал ряд важных наблюдений о языках. Многие из них лежат в основе современного понимания языка. Одно из наблюдений заключается в том, что естественные языки состоят из рекурсивных и повторяющихся подструктур.



Кот • Не умеет говорить или программировать

Например, рассмотрим фразу:

› `The cat walked on the carpet.` — «Кот прошёл по ковру».

По правилам английского языка существительное `cat` можно заменить двумя существительными, соединёнными союзом `and` («и»):

› `The cat and dog walked on the carpet.` — «Кот и собака прошли по ковру».

Каждое из новых существительных, в свою очередь, можно снова заменить. Можно применить прежнее правило и заменить `cat` двумя существительными, соединёнными `and`. А можно применить другое правило и заменить каждое существительное сочетанием прилагательного и существительного, добавив описание.

› `The cat and mouse and dog walked on the carpet.`

› `The white cat and black dog walked on the carpet.`

Это лишь два примера, а в английском языке есть множество правил изменения, преобразования и замены слов разных типов.

Точно такое же поведение мы наблюдаем и в языках программирования. В C тело оператора `if` содержит список других операторов. Каждый из них может сам оказаться оператором `if`. Подобные повторяющиеся структуры и замены встречаются во всех частях языка. Их иногда называют *правилами переписывания*, поскольку они говорят, как одно можно *переписать* в виде другого.

› `if (x > 5) { return x; }`

› `if (x > 5) { if (x > 10) { return x; } }`

Из этого наблюдения Хомского следует важный вывод: хотя на определённом языке можно сказать или записать бесконечное множество разных вещей, их всё же можно обработать и понять с помощью конечного числа правил переписывания. Набор таких правил называется *грамматикой*.

Правила переписывания можно описывать по-разному. Например, словами: «*предложение* должно быть *глагольной группой*» или «*глагольная группа* может состоять из *глагола* либо из *наречия* и *глагола*». Для людей этот способ хорош, но для компьютера слишком неточен. В программировании грамматику необходимо описывать более формально.

Чтобы написать язык программирования, такой как наш Lisp, нужно понимать грамматики. Для чтения ввода пользователя мы создадим *грамматику*, которая его описывает. Затем с её помощью можно определить, допустим ли ввод, и построить структурированное внутреннее представление. Это значительно упростит понимание и последующее вычисление — выполнение закодированных в нём операций.

Здесь на помощь приходит библиотека `mpc`.

Комбинаторы парсеров

`mpc` — написанная мной библиотека *комбинаторов парсеров*. Она позволяет создавать программы, понимающие и обрабатывающие определённые языки. Такие программы называются *парсерами*, или синтаксическими анализаторами. Существует много способов их создания, но прелесть библиотеки комбинаторов в том, что она позволяет легко строить парсеры, просто задавая *грамматику*... ну, почти.

Многие такие библиотеки позволяют писать обычный код, который *немного похож* на грамматику, вместо непосредственного задания грамматики. Во многих случаях это удобно, но иногда получается громоздко и сложно. К счастью, `mpc` позволяет писать обычный код, похожий на грамматику, *или* задавать грамматику напрямую с помощью специальной записи!

Грамматики в коде

Итак, как же *выглядит* код, похожий на грамматику? Рассмотрим `mpc` на примере кода для грамматики, распознающей язык сиба-ину, более известный как *Doge*. Определим его так:

› *Прилагательное* — это «wow», «many», «so» или «such».

› *Существительное* — это «lisp», «language», «c», «book» или «build».

› *Фраза* состоит из *прилагательного*, за которым следует *существительное*.

› *Doge* состоит из нуля или более *фраз*.

Начнём с определения *прилагательного* и *существительного*. Создадим два парсера типа `mpc_parser_t*` и сохраним их в переменных `Adjective` и `Noun`. Функция `mpc_or` создаёт парсер, выбирающий один из нескольких вариантов, а `mpc_sym` оборачивает исходные строки.

Если немного прищуриться, код можно прочитать как правила, заданные выше.

```
/* Build a parser 'Adjective' to recognize descriptions */
mpc_parser_t* Adjective = mpc_or(4,
  mpc_sym("wow"), mpc_sym("many"),
  mpc_sym("so"),  mpc_sym("such")
);

/* Build a parser 'Noun' to recognize things */
mpc_parser_t* Noun = mpc_or(5,
  mpc_sym("lisp"), mpc_sym("language"),
  mpc_sym("book"),mpc_sym("build"),
  mpc_sym("c")
);
```

Как получить доступ к функциям `mpc`?

Пока не беспокойтесь о компиляции или запуске примеров этой главы. Сосредоточьтесь на теории грамматик. В следующей главе мы настроим `mpc` и используем её для языка, более близкого к нашему Lisp.

Для определения `Phrase` можно обратиться к существующим парсерам. Нам нужна функция `mpc_and`, которая задаёт последовательность: сначала требуется одно, потом другое. Мы передаём ей ранее определённые парсеры `Adjective` и `Noun`. Функция также принимает аргументы `mpcf_strfold` и `free`, определяющие, как объединять или удалять результаты парсеров. Пока не обращайте на них внимания.

```
mpc_parser_t* Phrase = mpc_and(2, mpcf_strfold,
  Adjective, Noun, free);
```

Для определения *Doge* необходимо указать, что требуется *ноль или более* повторений некоторого парсера. Для этого используем `mpc_many`. Как и прежде, функция принимает специальную переменную `mpcf_strfold`, задающую объединение результатов; пока её можно игнорировать.

```
mpc_parser_t* Doge = mpc_many(mpcf_strfold, Phrase);
```

Создав парсер, который ищет *ноль или более* повторений другого парсера, мы получили интересный результат. Наш `Doge` принимает ввод любой длины, а значит, его язык *бесконечен*. Вот лишь некоторые примеры строк, которые он может принять. Как мы выяснили в начале главы, конечного числа правил переписывания достаточно для создания бесконечного языка.

```
"wow book such language many lisp"
"so c such build such language"
```

```
"many build wow c"
""
"wow lisp wow c many language"
"so c"
```

Применяя другие функции `mpc`, мы можем постепенно создавать парсеры всё более сложных языков. Код *отчасти* читается как грамматика, но по мере усложнения становится беспорядочнее, поэтому такой подход не всегда прост. В репозитории mpc описан целый набор вспомогательных функций, построенных на простых конструкциях и облегчающих распространённые задачи. Для сложных языков этот подход хорош тем, что позволяет тонко управлять процессом, но нам он не понадобится.

Естественная запись грамматик

`mpc` позволяет записывать грамматики и в более естественной форме. Вместо функций C, не слишком похожих на грамматику, можно задать всё одной длинной строкой. При этом не нужно беспокоиться об объединении или удалении результатов функциями вроде `mpcf_strfold` и `free`: всё делается автоматически.

Вот как можно воспроизвести предыдущие примеры этим способом:

```
mpc_parser_t* Adjective = mpc_new("adjective");
mpc_parser_t* Noun      = mpc_new("noun");
mpc_parser_t* Phrase    = mpc_new("phrase");
mpc_parser_t* Doge      = mpc_new("doge");

mpca_lang(MPCA_LANG_DEFAULT,
  "                                        \
    adjective : \"wow\" | \"many\"         \
              |  \"so\" | \"such\";        \
    noun      : \"lisp\" | \"language\"    \
              | \"book\" | \"build\" | \"c\"; \
    phrase    : <adjective> <noun>;        \
    doge      : <phrase>*;                 \
  ",
  Adjective, Noun, Phrase, Doge);

/* Do some parsing here... */

mpc_cleanup(4, Adjective, Noun, Phrase, Doge);
```

Даже не разобравшись до конца в синтаксисе длинной строки, легко заметить, насколько *понятнее* выглядит грамматика. Если узнать значение специальных символов, почти не придётся щуриться.

Обратите также внимание, что процесс теперь состоит из двух шагов. Сначала мы создаём и называем несколько правил с помощью `mpc_new`, а затем определяем их с помощью `mpca_lang`.

Первый аргумент `mpca_lang` — флаги параметров; мы используем значения по умолчанию. Второй — длинная многострочная строка C. Это описание *грамматики*, состоящее из нескольких *правил переписывания*. В каждом слева указано имя правила, затем двоеточие `:`, а справа — определение, заканчивающееся точкой с запятой `;`.

Специальные символы в правой части правил работают так:

`"ab"`	Требуется строка `ab`.
`'a'`	Требуется символ `a`.
`'a' 'b'`	Сначала требуется `'a'`, затем `'b'`.
`'a' | 'b'`	Требуется либо `'a'`, либо `'b'`.
`'a'*`	Требуется ноль или более `'a'`.
`'a'+`	Требуется один или более `'a'`.
`<abba>`	Требуется правило с именем `abba`.

Звучит знакомо...

Заметили, что описание входной строки для `mpca_lang` похоже на задание грамматики? Так оно и есть. Внутри `mpc` использует саму себя для разбора входа, переданного в `mpca_lang`. Для этого она задаёт *грамматику* в коде предыдущим способом. Здорово, правда?

С помощью таблицы выше проверьте, что эта запись эквивалентна грамматике, которую мы задали в коде.

Именно этот способ описания грамматик мы будем использовать в следующих главах. Поначалу он может показаться сложным: грамматики бывает непросто понять. Но по мере работы вы гораздо лучше освоите их создание и изменение.

Эта глава посвящена теории, поэтому, выполняя дополнительные задания, не слишком переживайте о правильности. Важнее мыслить в нужном направлении. Смело придумывайте обозначения для понятий, если это упрощает запись. Для некоторых заданий могут понадобиться циклические или рекурсивные структуры грамматики, так что не бойтесь их использовать!

Справочный код

Исходный код: doge_code.c, doge_grammar.c.

Дополнительные задания

- › Запишите ещё несколько примеров строк языка `Doge`.
- › Зачем в грамматике стоят обратные косые черты `\` перед кавычками `"`?
- › Зачем обратные косые черты `\` стоят в конце строк грамматики?
- › Опишите словами грамматику дробных чисел, например `0.01` или `52.221`.
- › Опишите словами грамматику веб-адресов, например `http://www.buildyourownlisp.com`.
- › Опишите словами грамматику простых английских предложений, например `the cat sat on the mat`.
- › Опишите эти грамматики более формально, используя `|`, `*` или собственные обозначения.
- › Если вы знакомы с JSON, опишите словами его грамматику.

Разбор • Глава 6

Польская запись

Чтобы опробовать `mpc`, реализуем простую грамматику, похожую на математическую часть нашего Lisp. Она называется польской записью: в ней арифметический оператор стоит перед операндами.

Например:



Польский дворянин • Типичный пользователь польской записи

`1 + 2 + 6`	записывается как	`+ 1 2 6`
`6 + (2 * 9)`	как	`+ 6 (* 2 9)`
`(10 * 2) / (4 + 2)`	как	`/ (* 10 2) (+ 4 2)`

Нам нужна грамматика, описывающая эту запись. Сначала опишем её словами, а затем уточним формально.

В польской записи оператор всегда стоит в начале выражения; за ним идут числа или другие выражения в скобках. Значит, *программа* — это *оператор*, за которым следует одно или несколько *выражений*; *выражение* — это либо *число*, либо заключённые в скобки оператор и одно или несколько выражений.

Более формально:

`Program`	начало ввода, `Operator`, одно или несколько `Expression` и конец ввода.
`Expression`	либо `Number`, либо `'('`, `Operator`, одно или несколько `Expression` и `')'`.
`Operator`	`'+'`, `'-'`, `'*'` или `'/'`.
`Number`	необязательный знак `-` и один или несколько символов от `0` до `9`.

Регулярные выражения

Большинство правил можно записать уже знакомыми средствами, но с *Number* и *Program* могут возникнуть трудности. Мы ещё не знаем, как обозначать начало и конец ввода, необязательные символы и диапазоны символов.

Для этого существуют *регулярные выражения*. С их помощью записывают грамматики небольших фрагментов текста, например слов и чисел. Такая грамматика не может состоять из нескольких правил, зато позволяет точно и кратко указать, какие строки ей соответствуют. Вот основные правила:

`.`	Любой один символ.
`a`	Символ `a`.
`[abcdef]`	Любой символ из набора `abcdef`.
`[a-f]`	Любой символ от `a` до `f`.
`a?`	Необязательный символ `a`.
`a*`	Ноль или более символов `a`.
`a+`	Один или более символов `a`.
`^`	Начало ввода.
`$`	Конец ввода.

Пока этого достаточно. О регулярных выражениях написаны целые книги; любознательный читатель найдёт дополнительную информацию в сети. В следующих главах мы будем их использовать, поэтому основы пригодятся, но осваивать всё прямо сейчас не нужно.

В грамматике `mpc` регулярные выражения заключают между косыми чертами `/`. По приведённым правилам наше число можно записать так: `/-?[0-9]+/`.

Установка mpc

Прежде чем писать грамматику, нужно подключить заголовочные файлы `mpc` и скомпоновать программу с библиотекой, как мы делали для `editline` в Linux и Mac. Возьмите код из главы 4, переименуйте файл в `parsing.c` и скачайте `mpc.h` и `mpc.c` из репозитория mpc. Поместите их в каталог с исходным файлом.

Для подключения добавьте `#include "mpc.h"` в начало файла, а для компоновки укажите `mpc.c` в команде компиляции. В Linux необходимо также подключить математическую библиотеку флагом `-lm`.

В **Linux** и **Mac**:

```
cc -std=c99 -Wall parsing.c mpc.c -ledit -lm -o parsing
```

В **Windows**:

```
cc -std=c99 -Wall parsing.c mpc.c -o parsing
```

Подождите, разве не нужно писать `#include <mpc.h>`?

В C файлы можно подключать двумя способами: с угловыми скобками `<>`, как мы делали раньше, и с кавычками `""`.

Разница в порядке поиска: угловые скобки сначала ищут заголовок в системных каталогах, а кавычки — в текущем каталоге. Поэтому системные заголовки, например `<stdio.h>`, обычно заключают в угловые скобки, а локальные, например `"mpc.h"`, — в кавычки.

Грамматика польской записи

Уточнив правила и воспользовавшись регулярными выражениями, получим окончательную грамматику. Прочитайте код и убедитесь, что он соответствует нашему словесному описанию польской записи.

```
/* Create Some Parsers */
mpc_parser_t* Number   = mpc_new("number");
mpc_parser_t* Operator = mpc_new("operator");
mpc_parser_t* Expr     = mpc_new("expr");
mpc_parser_t* Lispy    = mpc_new("lispy");

/* Define them with the following Language */
mpca_lang(MPCA_LANG_DEFAULT,
  "                                                     \
    number   : /-?[0-9]+/ ;                             \
    operator : '+' | '-' | '*' | '/' ;                  \
    expr     : <number> | '(' <operator> <expr>+ ')' ;  \
    lispy    : /^/ <operator> <expr>+ /$/ ;             \
  ",
  Number, Operator, Expr, Lispy);
```

Добавим этот код в интерактивную программу из главы 4: поместите его в начале функции `main`, до вывода версии и подсказки о выходе. В конце программы парсеры необходимо удалить. Непосредственно перед возвратом из `main` добавьте:

```
/* Undefine and Delete our Parsers */
mpc_cleanup(4, Number, Operator, Expr, Lispy);
```

Я получаю ошибку `undefined reference to `mpc_lang'`.

Нужно написать `mpca_lang`, с буквой `a` в конце!

Разбор пользовательского ввода

Новый код создаёт парсер `mpc` для нашего языка польской записи, но нам ещё нужно применить его к строке, вводимой пользователем. Изменим цикл `while`: вместо простого вывода введённой строки он будет пытаться разобрать её. Замените вызов `printf` следующим кодом, использующим парсер программы `Lispy`:

```
/* Attempt to Parse the user Input */
mpc_result_t r;
if (mpc_parse("<stdin>", input, Lispy, &r)) {
  /* On Success Print the AST */
  mpc_ast_print(r.output);
  mpc_ast_delete(r.output);
} else {
  /* Otherwise Print the Error */
  mpc_err_print(r.error);
  mpc_err_delete(r.error);
}
```

Здесь вызывается `mpc_parse` с парсером `Lispy` и строкой `input`. Результат разбора записывается в `r`; функция возвращает `1` при успехе и `0` при ошибке. При передаче `r` мы используем оператор получения адреса `&`; подробнее о нём поговорим позже.

При успехе внутренняя структура записывается в поле `output` переменной `r`. Её можно вывести с помощью `mpc_ast_print` и удалить с помощью `mpc_ast_delete`.

В противном случае ошибка записывается в поле `error`. Её можно вывести через `mpc_err_print` и удалить через `mpc_err_delete`.

Скомпилируйте обновлённую программу и поэкспериментируйте. Вводите разные строки и наблюдайте за реакцией. Правильное поведение выглядит так:

```
Lispy Version 0.0.0.0.2
Press Ctrl+c to Exit

lispy> + 5 (* 2 2)
>
  regex
  operator|char:1:1 '+'
  expr|number|regex:1:3 '5'
  expr|>
    char:1:5 '('
    operator|char:1:6 '*'
    expr|number|regex:1:8 '2'
    expr|number|regex:1:10 '2'
    char:1:11 ')'
  regex
lispy> hello
<stdin>:1:1: error: expected whitespace, '+', '-', '*' or '/' at 'h'
lispy> / 1dog
<stdin>:1:4: error: expected one of '0123456789', whitespace, '-', one or more of one of '0123456789',
'(' or end of input at 'd'
lispy>
```

Я получаю ошибку `<stdin>:1:1: error: Parser Undefined!`.

Это означает, что грамматика, переданная в `mpca_lang`, записана с ошибкой. Попробуйте найти неверную часть. Сверьтесь с примером кода к этой главе, чтобы проверить правильную запись.

Справочные материалы

Исходный код: parsing.c.

Дополнительные задания

- › Напишите регулярное выражение для строк, состоящих из любых символов `a` и `b`, например `aababa` или `bbaa`.
- › Напишите выражение для строк с чередующимися `a` и `b`, например `abababab` или `aba`.
- › Напишите выражение, которому соответствуют `pit`, `pot` и `respite`, но не `peat`, `spit` и `part`.
- › Добавьте в грамматику новый оператор, например `%`.
- › Измените грамматику так, чтобы она распознавала операторы `add`, `sub`, `mul`, `div`.
- › Добавьте распознавание десятичных дробей, например `0.01`, `5.21` и `10.2`.
- › Измените грамматику так, чтобы операторы записывались привычным образом между двумя выражениями.
- › Используйте грамматику предыдущей главы для разбора `Doge`. Добавьте обозначения начала и конца ввода.

Вычисление • Глава 7

Деревья

Теперь мы можем читать ввод и представлять его в виде внутренней структуры, но пока не умеем его вычислять. В этой главе добавим код, который обходит структуру и выполняет записанные в ней вычисления.

Именно эту структуру программа выводила в предыдущей главе. Она называется *абстрактным синтаксическим деревом* и представляет структуру программы на основе введённого текста. В листьях дерева находятся числа и операторы — собственно данные для обработки. В ветвях находятся правила, по которым получены соответствующие части дерева, то есть сведения о том, как его обходить и вычислять.



Абстрактная рождественская ёлка • Сезонная разновидность

Прежде чем разбираться с обходом, посмотрим, как устроена эта структура. Загляните в `mpc.h`: там определена `mpc_ast_t` — структура данных, полученная в результате разбора.

```
typedef struct mpc_ast_t {
  char* tag;
  char* contents;
  mpc_state_t state;
  int children_num;
  struct mpc_ast_t** children;
} mpc_ast_t;
```

У структуры несколько полей. Рассмотрим каждое по очереди.

Первое — `tag`. При выводе дерева эта информация предшествовала содержимому узла. Это строка со списком правил, использованных при разборе данного элемента, например `expr|number|regex`. Поле важно, потому что показывает, какие правила создали узел.

Второе поле, `contents`, содержит данные узла, например `'*'`, `'('` или `'5'`. У ветвей оно пусто, а у листьев позволяет узнать оператор или число.

Поле `state` содержит информацию о состоянии парсера при нахождении узла, например номер строки и столбца. Мы не будем его использовать.

Наконец, поля `children_num` и `children` помогут обходить дерево. Первое сообщает количество дочерних узлов, второе представляет собой их массив.

Тип поля `children` — `mpc_ast_t**`, то есть двойной указатель. Это не так страшно, как выглядит; в следующих главах мы разберём его подробнее. Пока можно считать его списком

дочерних узлов дерева.

Доступ к дочернему узлу осуществляется через запись для массива: после имени `children` указывается индекс в квадратных скобках. Например, первый узел — `children[0]`. Обратите внимание: в C индексы начинаются с `0`.

Поскольку `mpc_ast_t*` — *указатель* на структуру, доступ к её полям записывается немного иначе: вместо точки `.` нужна стрелка `->`. Пока просто запомните, что для доступа к полю через указатель используют стрелку.

```
/* Load AST from output */
mpc_ast_t* a = r.output;
printf("Tag: %s\n", a->tag);
printf("Contents: %s\n", a->contents);
printf("Number of children: %i\n", a->children_num);

/* Get First Child */
mpc_ast_t* c0 = a->children[0];
printf("First Child Tag: %s\n", c0->tag);
printf("First Child Contents: %s\n", c0->contents);
printf("First Child Number of children: %i\n",
  c0->children_num);
```

Рекурсия

У этой структуры есть любопытная особенность: она ссылается сама на себя. Каждый дочерний узел тоже является деревом, как и его дочерние узлы. Подобно нашим языкам и правилам переписывания, структура состоит из повторяющихся частей, похожих на родительскую.

Такие вложенные структуры могут продолжаться сколько угодно. Поэтому функция для любых деревьев не должна ограничиваться несколькими уровнями: она должна работать с деревом произвольной глубины.

К счастью, это возможно благодаря повторяемости структуры и приёму под названием *рекурсия*. Проще говоря, рекурсивная функция вызывает саму себя в ходе вычисления.



Рекурсия • Опасна при пожаре

Звучит странно — определять функцию через неё саму. Но функция может выдавать разные результаты при разных входных данных. Если при рекурсивном вызове передавать изменённые данные и предусмотреть условия, при которых новых вызовов не будет, такая функция может выполнять полезную работу.

Например, напишем рекурсивную функцию подсчёта узлов дерева. Начнём с простейшего случая: у входного дерева нет дочерних узлов. Ответ равен единице. Если же дочерние узлы есть, ответ — единица за сам узел плюс количество узлов во всех его потомках.

Как узнать количество узлов в потомках? Воспользоваться функцией, которую мы как раз определяем! *Да, это рекурсия.* На C её можно записать так:

```
int number_of_nodes(mpc_ast_t* t) {
  if (t->children_num == 0) { return 1; }
  if (t->children_num >= 1) {
    int total = 1;
    for (int i = 0; i < t->children_num; i++) {
      total = total + number_of_nodes(t->children[i]);
    }
    return total;
  }
  return 0;
}
```

Рекурсивные функции требуют своеобразного прыжка веры: сначала мы предполагаем, что функция уже умеет правильно решать задачу, а затем используем её, чтобы написать ту самую функцию!

Обычно рекурсия следует одной схеме. Сначала определяется *базовый случай*, завершающий рекурсию, например `t->children_num == 0`. Затем определяется *рекурсивный случай*, например `t->children_num >= 1`: он разбивает вычисление на меньшие части, рекурсивно вычисляет их и объединяет результаты.

Рекурсия требует размышлений. Прежде чем продолжать, убедитесь, что поняли её: в оставшейся части книги мы будем активно её использовать. Если остались сомнения, попробуйте выполнить дополнительные задания этой главы.

Вычисление

Для вычисления дерева разбора напишем рекурсивную функцию. Но сначала посмотрим, что можно заметить в структуре входного дерева. Выведите несколько выражений программой из предыдущей главы. Что вы видите?

```
lispy> * 10 (+ 1 51)
>
  regex
  operator|char:1:1 '*'
  expr|number|regex:1:3 '10'
  expr|>
    char:1:6 '('
    operator|char:1:7 '+'
    expr|number|regex:1:9 '1'
    expr|number|regex:1:11 '51'
    char:1:13 ')'
  regex
```

Узел с тегом `number` всегда является числом и не имеет потомков; его содержимое достаточно преобразовать в целое число. Это будет базовый случай рекурсии.

Если узел помечен `expr` и не является `number`, нужно посмотреть на его второй дочерний узел (первый — всегда `'('`) и определить оператор. Затем применить оператор к результатам

вычисления остальных дочерних узлов, кроме последнего, который всегда равен `')'`. Это рекурсивный случай. То же нужно сделать для корневого узла.

Как и при подсчёте узлов, при вычислении дерева нам придётся накапливать результат. Для него используем тип C `long` — длинное целое число.

Чтобы узнать тег узла или получить число, понадобятся поля `tag` и `contents`. Они содержат строки, поэтому сначала познакомимся с несколькими функциями для работы со строками.

`atoi`	Преобразует `char*` в `int`.
`strcmp`	Принимает две строки `char*` и возвращает `0`, если они равны.
`strstr`	Принимает две строки `char*` и возвращает указатель на вхождение второй в первую либо `0`, если вторая не является подстрокой первой.

С помощью `strcmp` определим оператор, а с помощью `strstr` проверим, содержит ли тег нужную подстроку. Рекурсивная функция вычисления выглядит так:

```
long eval(mpc_ast_t* t) {

  /* If tagged as number return it directly. */
  if (strstr(t->tag, "number")) {
    return atoi(t->contents);
  }

  /* The operator is always second child. */
  char* op = t->children[1]->contents;

  /* We store the third child in `x` */
  long x = eval(t->children[2]);

  /* Iterate the remaining children and combining. */
  int i = 3;
  while (strstr(t->children[i]->tag, "expr")) {
    x = eval_op(x, op, eval(t->children[i]));
    i++;
  }

  return x;
}
```

Функцию `eval_op` определим следующим образом. Она принимает число, строку с оператором и ещё одно число, определяет оператор и выполняет соответствующую операцию C.

```
/* Use operator string to see which operation to perform */
long eval_op(long x, char* op, long y) {
  if (strcmp(op, "+") == 0) { return x + y; }
  if (strcmp(op, "-") == 0) { return x - y; }
  if (strcmp(op, "*") == 0) { return x * y; }
  if (strcmp(op, "/") == 0) { return x / y; }
  return 0;
}
```

Вывод

Теперь вместо дерева нужно выводить результат вычисления. Передадим дерево в функцию `eval` и выведем результат через `printf` со спецификатором `%li`, предназначенным для типа `long`. Не забудем удалить дерево после вычисления.

```
long result = eval(r.output);
printf("%li\n", result);
mpc_ast_delete(r.output);
```

Если всё получилось, на нашем новом языке уже можно выполнять простые вычисления!

```
Lispy Version 0.0.0.0.3
Press Ctrl+c to Exit

lispy> + 5 6
11
lispy> - (* 10 10) (+ 1 1 1)
97
```

Справочные материалы

Исходный код: evaluation.c.

Дополнительные задания

- › Напишите рекурсивную функцию подсчёта листьев дерева.
- › Напишите рекурсивную функцию подсчёта ветвей дерева.
- › Напишите рекурсивную функцию поиска наибольшего числа потомков у одной ветви дерева.
- › Как с помощью `strstr` проверить, что узел имеет тег `expr`?
- › Как с помощью `strcmp` проверить, что содержимое узла — `'('` или `')'`?
- › Добавьте оператор `%`, возвращающий остаток от деления. Например, `% 10 6` равно `4`.
- › Добавьте оператор `^` для возведения в степень. Например, `^ 4 2` равно `16`.
- › Добавьте функцию `min`, возвращающую наименьшее число. Например, `min 1 5 3` равно `1`.
- › Добавьте функцию `max`, возвращающую наибольшее число. Например, `max 1 5 3` равно `5`.
- › Измените оператор вычитания `-`, чтобы при одном аргументе он менял его знак.

Обработка ошибок • Глава 8

Аварийное завершение

Некоторые из вас, возможно, заметили проблему в программе из предыдущей главы. Попробуйте ввести следующую строку и посмотрите, что произойдёт:

```
Lispy Version 0.0.0.0.3
Press Ctrl+c to Exit

lispy> / 10 0
```

Ой! Программа аварийно завершилась при попытке деления на ноль. Во время разработки это допустимо, но готовая программа в идеале не должна падать и должна всегда объяснять пользователю, что пошло не так.



Уолтер Уайт • Гейзенберг

Сейчас программа сообщает о синтаксических ошибках, но ещё не умеет сообщать об ошибках вычисления выражений. Нужно добавить обработку таких ошибок. В C это бывает неудобно, но правильное начало окупится, когда система станет сложнее.

Аварийное завершение программ на C — часть жизни. Если что-то идёт не так, операционная система их останавливает. Причин и способов падения множество. Вам обязательно встретится хотя бы один гейзенбаг.

Но в работе программ на C нет никакой магии. Столкнувшись с трудной ошибкой, не сдавайтесь и не смотрите в экран до боли в глазах. Лучше воспользуйтесь случаем и научитесь работать с `gdb` и `valgrind`. Это новые инструменты в вашем арсенале; потратив время на их освоение, вы сэкономите много времени и нервов.

Значение Lisp

В C есть несколько способов обработки ошибок. Здесь я предпочитаю сделать ошибку одним из возможных результатов вычисления. Тогда выражение Lispy даёт либо *число*, либо *ошибку*. Например, `+ 1 2` даёт число, а `/ 10 0` — ошибку.

Нам нужна структура, которая может представлять и то и другое. Для простоты используем `struct` с отдельными полями для каждого варианта и специальным полем `type`, указывающим, какие поля имеют смысл.

Назовём её `lval`, сокращение от *Lisp Value* — значение Lisp.

```
/* Declare New lval Struct */
typedef struct {
  int type;
  long num;
  int err;
} lval;
```

Перечисления

Поля `type` и `err` имеют тип `int`, то есть представлены целым числом. Каждому значению мы присвоим определённый смысл: например, если `type` равно `0`, структура представляет число, а если `1` — ошибку. Это простой и эффективный способ.

Однако множество отдельных `0` и `1` сделало бы код непонятным. Вместо них используем именованные константы с этими целочисленными значениями. По имени читатель поймёт, почему происходит сравнение и что оно означает.

В C для этого существует `enum`:

```
/* Create Enumeration of Possible lval Types */
enum { LVAL_NUM, LVAL_ERR };
```

`enum` объявляет именованные значения, которым автоматически присваиваются целочисленные константы. Выше мы объявили возможные значения поля `type`.

Перечисление понадобится и для поля ошибки. В нашей программе возможны три ошибки: деление на ноль, неизвестный оператор и число, слишком большое для представления типом `long`. Перечислим их:

```
/* Create Enumeration of Possible Error Types */
enum { LERR_DIV_ZERO, LERR_BAD_OP, LERR_BAD_NUM };
```

Функции для значений Lisp

Тип `lval` почти готов. В отличие от `long`, у нас пока нет способа создавать его экземпляры. Объявим две функции, которые создают `lval` типа «ошибка» или «число».

```
/* Create a new number type lval */
lval lval_num(long x) {
  lval v;
  v.type = LVAL_NUM;
  v.num = x;
  return v;
}

/* Create a new error type lval */
lval lval_err(int x) {
  lval v;
  v.type = LVAL_ERR;
  v.err = x;
```

```
  return v;
}
```

Обе функции создают `lval` с именем `v`, заполняют поля и возвращают его.

Теперь значение `lval` может быть одним из двух вариантов, поэтому просто вывести его через `printf` уже нельзя: поведение зависит от типа. Для этого в C удобно использовать оператор `switch`. Он сравнивает входное значение с известными вариантами (*case*) и при совпадении выполняет следующий за вариантом код до оператора `break`.

С его помощью напишем функцию вывода `lval` любого типа:

```
/* Print an "lval" */
void lval_print(lval v) {
  switch (v.type) {
    /* In the case the type is a number print it */
    /* Then 'break' out of the switch. */
    case LVAL_NUM: printf("%li", v.num); break;

    /* In the case the type is an error */
    case LVAL_ERR:
      /* Check what type of error it is and print it */
      if (v.err == LERR_DIV_ZERO) {
        printf("Error: Division By Zero!");
      }
      if (v.err == LERR_BAD_OP)   {
        printf("Error: Invalid Operator!");
      }
      if (v.err == LERR_BAD_NUM)  {
        printf("Error: Invalid Number!");
      }
    break;
  }
}

/* Print an "lval" followed by a newline */
void lval_println(lval v) { lval_print(v); putchar('\n'); }
```

Вычисление с ошибками

Теперь изменим функции вычисления так, чтобы они использовали `lval` вместо `long`. Кроме типов в сигнатурах, нужно изменить и поведение при получении числа или ошибки.

В `eval_op` при обнаружении ошибки следует сразу её вернуть; вычисление выполняется только если оба аргумента — числа. Вместо попытки деления на ноль будем возвращать ошибку. Это устранит падение, показанное в начале главы.

```
lval eval_op(lval x, char* op, lval y) {

  /* If either value is an error return it */
  if (x.type == LVAL_ERR) { return x; }
  if (y.type == LVAL_ERR) { return y; }

  /* Otherwise do maths on the number values */
  if (strcmp(op, "+") == 0) { return lval_num(x.num + y.num); }
  if (strcmp(op, "-") == 0) { return lval_num(x.num - y.num); }
  if (strcmp(op, "*") == 0) { return lval_num(x.num * y.num); }
  if (strcmp(op, "/") == 0) {
```

```
    /* If second operand is zero return error */
    return y.num == 0
      ? lval_err(LERR_DIV_ZERO)
      : lval_num(x.num / y.num);
  }

  return lval_err(LERR_BAD_OP);
}
```

Что здесь делает знак `?`?

Для проверки второго аргумента деления используется знак вопроса `?`, за которым следует двоеточие `:`. Это *тернарный оператор*, позволяющий записать условное выражение в одну строку.

Его форма: `<условие> ? <тогда> : <иначе>`. Если условие истинно, результатом становится выражение после `?`, иначе — после `:`.

Некоторые не любят этот оператор, считая его менее понятным. Сначала он может показаться неудобным, но после знакомства с ним проблемы возникают редко.

Похожим образом изменим функцию `eval`. Поскольку `eval_op` уже надёжно обрабатывает ошибки, здесь достаточно добавить проверку при преобразовании числа.

Используем `strtol` для преобразования строки в `long`. Это позволит проверить специальную переменную `errno` и убедиться в успехе преобразования. Такой способ надёжнее прежнего с `atoi`.

```
lval eval(mpc_ast_t* t) {

  if (strstr(t->tag, "number")) {
    /* Check if there is some error in conversion */
    errno = 0;
    long x = strtol(t->contents, NULL, 10);
    return errno != ERANGE ? lval_num(x) : lval_err(LERR_BAD_NUM);
  }

  char* op = t->children[1]->contents;
  lval x = eval(t->children[2]);

  int i = 3;
  while (strstr(t->children[i]->tag, "expr")) {
    x = eval_op(x, op, eval(t->children[i]));
    i++;
  }

  return x;
}
```

Осталось изменить вывод результата вычисления: воспользуемся новой функцией, которая умеет выводить `lval` любого типа.

```
lval result = eval(r.output);
lval_println(result);
mpc_ast_delete(r.output);
```

Готово! Запустите новую программу и убедитесь, что при делении на ноль она больше не падает.

```
lispy> / 10 0
Error: Division By Zero!
lispy> / 10 2
5
```

Соединяем детали

Некоторые читатели к этому моменту могут чувствовать себя неуютно: инструкции вроде бы удаётся выполнять, но ясного понимания всех механизмов за кулисами пока нет.



Водопровод • Сложнее, чем кажется

Если это про вас, хочу вас успокоить: вы отлично справляетесь. Если внутреннее устройство непонятно, возможно, я просто недостаточно подробно всё объяснил. Это нормально.

Умение продвигаться вперёд и заставлять код работать в таких условиях — ценный навык программиста. Если вы дошли сюда, значит, он у вас есть.

В программировании такую работу можно назвать *соединением деталей*: мы следуем инструкциям, связывая библиотеки или компоненты, не до конца понимая их внутреннее устройство.

Для этого нужны *вера* и *интуиция*. Вера — что, если звёзды сойдутся и все заклинания для волшебной машины будут произнесены правильно, всё действительно заработает. Интуиция — чтобы понять, что пошло не так и как это исправить.

К сожалению, непосредственно научить этому нельзя. Но если вы дошли сюда, то уже преодолели сложный рубеж. Обещаю: в следующих главах мы закончим соединять детали и займёмся настоящим, увлекательным программированием.

Справочные материалы

Исходный код: error_handling.c.

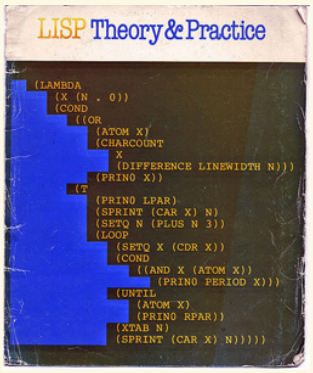
Дополнительные задания

- › Запустите код предыдущей главы через `gdb` и вызовите аварийное завершение. Посмотрите, что произойдёт.
- › Как дать имя перечислению `enum`?
- › Что такое типы `union` и как они работают?
- › Каковы преимущества `union` по сравнению с `struct`?
- › Можно ли использовать `union` в определении `lval`?
- › Добавьте в разбор и вычисление поддержку оператора остатка `%`.
- › Добавьте поддержку дробных чисел через поле типа `double`.

S-выражения • Глава 9

Списки и Lisp

Диалекты Lisp известны тем, что почти не различают данные и код: для их представления используются одни и те же структуры. Это позволяет делать многое из того, что недоступно другим языкам. Чтобы дать такую возможность нашему языку, придётся разделить процессы *чтения* ввода и *вычисления* сохранённого результата.



ВСЁ ЗАГЛАВНЫМИ • ТАК ПРАВИЛЬНО И ТАК НЕПРАВИЛЬНО

Поведение программы к концу главы изменится лишь немного: мы займёмся её внутренним устройством. Такая работа называется *рефакторингом* и впоследствии значительно облегчит нам жизнь. Как и при приготовлении обеда, мы не теряем время только потому, что пока не раскладываем еду по тарелкам. Порой ожидание даже приятнее самой еды!

Для хранения программы нам понадобится внутренняя структура списка, рекурсивно составленного из чисел, символов и других списков. В Lisp она обычно называется S-выражением, от *symbolic expression* — символьное выражение. Расширим `lval`, чтобы представлять такие выражения. Вычисление S-выражения происходит привычным для Lisp образом: первый элемент списка принимается за оператор, а все остальные — за операнды, к которым он применяется.

Добавив S-выражения, мы наконец вступим в мир Lisp.

Указатели

В C нельзя серьёзно исследовать списки, не разобравшись с указателями. Указатели известны тем, что их часто понимают неправильно. Их трудно преподавать: сама идея проста, но появляется множество новых терминов, а понятный пример применения часто отсутствует. Поэтому указатели кажутся гораздо страшнее, чем есть на самом деле. К счастью, у нас есть два идеальных и весьма типичных для C примера — вероятно, именно так вы будете использовать указатели в 90% случаев.

Указатели нужны из-за того, как в C вызываются функции. Аргументы всегда передаются *по значению*, то есть вызываемая функция получает их *копии*. Это относится к `int`, `long`, `char` и

определённым пользователем структурам, таким как `lval`. Обычно это удобно, но иногда возникают проблемы.

Например, большая структура может содержать множество вложенных структур. При каждом вызове функции её придётся копировать, и объём копируемых данных может оказаться огромным.

Другая проблема: структура `struct` всегда имеет фиксированный размер. У неё ограниченное количество полей, размер каждого тоже фиксирован. Если нужно передать функции список элементов, число которых меняется от вызова к вызову, одной структурой здесь не обойтись.

Чтобы решить эти проблемы, разработчики C (или кто-то ещё) придумали изящную идею. Представим память компьютера как огромный список байтов, каждому из которых присвоен общий индекс, или позиция, вроде номера дома. Первый байт имеет номер `0`, второй — `1` и так далее.

Все данные компьютера, включая структуры и переменные работающей программы, начинаются с некоторого индекса в этом списке. Если вместо копирования самих данных передать функции номер, обозначающий начало данных, она сможет обратиться к любому необходимому объёму.

Используя *адреса*, а не сами данные, мы позволяем функции читать и изменять память без копирования этих данных. Указатели можно применять и для других задач, например для записи результата по адресу, переданному на вход.

Размер адреса фиксирован, но объём данных по нему может увеличиваться или уменьшаться, если мы ведём его учёт. Поэтому можно создать структуру переменного размера и передать её функции для просмотра и изменения.

Итак, указатель — это число, представляющее начальный адрес данных в памяти. Тип указателя подсказывает нам и компилятору, данные какого типа там находятся. Чтобы объявить указатель, добавляют `*` к существующему типу. Мы уже видели `mpc_parser_t*`, `mpc_ast_t*` и `char*`.

Для получения адреса данных используется оператор `&`. Мы уже передавали указатель в `mpc_parse`, чтобы функция записала результат в `mpc_result_t`. Для получения данных по адресу — *разыменования* — используется оператор `*` перед переменной. Для доступа к полю структуры через указатель используют стрелку `->`, как мы видели в главе 7.

Стек и куча

Я предложил представлять память длинным списком байтов. На самом деле полезнее разделить её на две области: *стек* и *кучу*.

Возможно, вы слышали загадочные утверждения вроде «стек растёт вниз, а куча — вверх» или «стеков может быть много, а куча одна». Здесь это не так важно. Работа со стеком и

кучей в C может быть сложной, но не обязана оставаться загадкой: по сути, это две области памяти для разных задач.

Стек

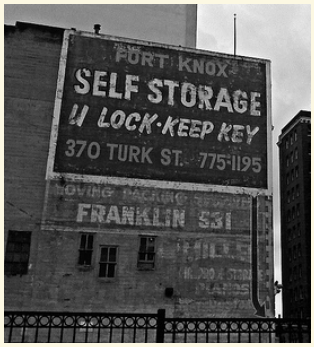


Стек • Как стопка кирпичей

В стеке находятся временные переменные и структуры программы. При каждом вызове функции для неё выделяется новая область стека. Туда помещаются локальные переменные, копии аргументов и служебная информация, например кто вызвал функцию и что делать после её завершения. Когда функция заканчивает работу, эта область освобождается для повторного использования.

Я представляю стек как строительную площадку. Для каждой новой задачи мы отгораживаем место для инструментов и материалов и приступаем к работе. При необходимости можно посетить другие части площадки или выйти за её пределы, но работа идёт на выделенном участке. Закончив, мы переносим результат в другое место и убираем за собой.

Куча



Куча • Запри и сохрани ключ

Куча — область памяти для объектов с более долгим сроком жизни. Память здесь нужно выделять и освобождать вручную. Функция `malloc` принимает нужное количество байтов и возвращает указатель на новый блок памяти такого размера.

Закончив работу с этой памятью, её необходимо освободить, передав полученный от `malloc` указатель функции `free`.

Работать с кучей сложнее, чем со стеком: программисту нужно помнить о правильном вызове `free`. Иначе программа может непрерывно выделять всё больше памяти. Это называется *утечкой памяти*. Простое правило: для каждого вызова `malloc` должен быть один соответствующий вызов `free`. Соблюдая его, можно правильно управлять кучей.

Я представляю кучу как большой склад. Мы звоним на стойку через `malloc` и просим несколько ящиков. Ими можно пользоваться как угодно: они сохранятся независимо от беспорядка на стройке. Между складом и площадкой можно переносить вещи; там удобно хранить материалы и крупные предметы, нужные лишь изредка. Но, закончив, нужно снова позвонить через `free`. Иначе все ящики закончатся, места не останется, а счёт станет огромным.

Разбор выражений

Теперь мы мыслим S-выражениями, а не польской записью, поэтому изменим парсер. S-выражение — это несколько выражений в круглых скобках; каждое из них может быть числом, оператором или другим S-выражением. Изменим правила разбора и переименуем `operator` в `symbol` в расчёте на будущие операторы, переменные и функции.

```
mpc_parser_t* Number = mpc_new("number");
mpc_parser_t* Symbol = mpc_new("symbol");
mpc_parser_t* Sexpr  = mpc_new("sexpr");
mpc_parser_t* Expr   = mpc_new("expr");
mpc_parser_t* Lispy  = mpc_new("lispy");

mpca_lang(MPCA_LANG_DEFAULT,
  "                                          \
    number : /-?[0-9]+/ ;                    \
    symbol : '+' | '-' | '*' | '/' ;         \
    sexpr  : '(' <expr>* ')' ;               \
    expr   : <number> | <symbol> | <sexpr> ; \
    lispy  : /^/ <expr>* /$/ ;               \
  ",
  Number, Symbol, Sexpr, Expr, Lispy);
```

При выходе не забудем очистить правила:

```
mpc_cleanup(5, Number, Symbol, Sexpr, Expr, Lispy);
```

Структура выражения

Нам нужен способ хранить S-выражения как `lval`, а значит, также хранить символы и числа. Добавим в перечисление два типа: `LVAL_SYM` для операторов вроде `+` и `LVAL_SEXPR` для S-выражений.

```
enum { LVAL_ERR, LVAL_NUM, LVAL_SYM, LVAL_SEXPR };
```

S-выражения — списки значений переменной длины. Структуры переменной длины создать нельзя, поэтому воспользуемся указателем. Поле `cell` будет указывать на список указателей `lval*`, каждый из которых указывает на отдельное значение. Значит, тип поля — двойной указатель `lval**`. Добавим также поле `count` для учёта количества элементов.

Символы будем представлять строками. Ошибки тоже изменим на строки: вместо кода можно будет хранить отдельное сообщение, улучшив и расширив отчёты об ошибках. Старое перечисление ошибок больше не нужно. Обновлённая структура:

```
typedef struct lval {
  int type;
  long num;
  /* Error and Symbol types have some string data */
  char* err;
  char* sym;
  /* Count and Pointer to a list of "lval*" */
  int count;
  struct lval** cell;
} lval;
```

Бывают ли указатели на указатели на указатели?

Есть старая программистская шутка: мастерство программиста C можно оценить по числу звёздочек в его указателях.

Новички используют `char*` и иногда `int*` — это однозвёздочные программисты. В программах среднего уровня встречаются двойные указатели вроде `lval**`, поэтому их авторы — двухзвёздочные. Тройной указатель — особое зрелище: перед вами работа великого и ужасного мастера, код которого не предназначен для глаз смертных. Поэтому звание трёхзвёздочного программиста редко бывает комплиментом.

Насколько мне известно, указатель с четырьмя звёздочками в дикой природе ещё не наблюдался.

Зачем здесь слово `struct`?

Новому определению `lval` нужна ссылка на самого себя. Поэтому перед фигурной скобкой мы указываем имя структуры и внутри определения обращаемся к ней как `struct lval`. Структура может ссылаться на собственный тип, но содержать должна только указатели на него, а не сам тип: иначе её размер зависел бы от собственного размера и при вычислении стал бы бесконечным!

Конструкторы и деструкторы

Изменим функции создания `lval` так, чтобы они возвращали указатели на значения. Это облегчит работу с переменными. Используем `malloc` и `sizeof` для выделения достаточного места под структуру `lval`, затем заполним поля через стрелку `->`.

Поля созданного `lval` могут содержать указатели на другие объекты в куче, поэтому нужна осторожность. Закончив работу с `lval`, необходимо удалить и объекты, на которые он указывает. Примем правило: освобождая память `lval`, освобождаем и всё, на что он указывает.

```
/* Construct a pointer to a new Number lval */
lval* lval_num(long x) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_NUM;
  v->num = x;
  return v;
}
```

```
/* Construct a pointer to a new Error lval */
lval* lval_err(char* m) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_ERR;
  v->err = malloc(strlen(m) + 1);
  strcpy(v->err, m);
```

```
  return v;
}
```

```
/* Construct a pointer to a new Symbol lval */
lval* lval_sym(char* s) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_SYM;
  v->sym = malloc(strlen(s) + 1);
  strcpy(v->sym, s);
  return v;
}
```

```
/* A pointer to a new empty Sexpr lval */
lval* lval_sexpr(void) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_SEXPR;
  v->count = 0;
  v->cell = NULL;
  return v;
}
```

Что такое `NULL`?

`NULL` — специальная константа нулевого указателя. Её часто используют для обозначения отсутствующего значения или пустых данных. Выше она показывает, что указатель пока не указывает на элементы данных. В дальнейшем `NULL` встретится ещё не раз, так что обращайте на него внимание.

Почему используется `strlen(s) + 1`?

Строки в C завершаются нулём: последний символ — `\0`, обозначающий конец строки. Это соглашение необходимо соблюдать, иначе программы будут неприятно ломаться.

Функция `strlen` возвращает число байтов без завершающего нуля. Поэтому нужно добавить единицу, чтобы выделить достаточно места и для него!

Теперь нужна функция удаления `lval*`. Она вызовет `free` для самого указателя, освобождая память от `malloc`, но прежде проверит тип значения и освободит память, на которую указывают его поля. Если каждому вызову конструктора соответствует `lval_del`, утечек памяти не будет.

```
void lval_del(lval* v) {

  switch (v->type) {
    /* Do nothing special for number type */
    case LVAL_NUM: break;

    /* For Err or Sym free the string data */
    case LVAL_ERR: free(v->err); break;
    case LVAL_SYM: free(v->sym); break;

    /* If Sexpr then delete all elements inside */
    case LVAL_SEXPR:
      for (int i = 0; i < v->count; i++) {
```

```
        lval_del(v->cell[i]);
      }
      /* Also free the memory allocated to contain the pointers */
      free(v->cell);
    break;
  }

  /* Free the memory allocated for the "lval" struct itself */
  free(v);
}
```

Чтение выражений

Сначала мы *прочитаем* программу и создадим представляющий её `lval*`, а затем *вычислим* его. Первый этап преобразует абстрактное синтаксическое дерево в S-выражение, второй вычислит его по обычным правилам Lisp.

Для первого этапа рекурсивно рассмотрим каждый узел дерева и создадим соответствующий `lval*` в зависимости от полей `tag` и `contents`. Для узла `number` или `symbol` сразу вызываем соответствующий конструктор. Для корня или `sexpr` создаём пустое S-выражение и постепенно добавляем в него допустимые подвыражения дерева.

Для добавления элемента напишем `lval_add`. Она увеличивает счётчик элементов и с помощью `realloc` изменяет размер области `v->cell`, чтобы поместился дополнительный `lval*`. Затем последнему элементу `v->cell[v->count-1]` присваивается переданное значение `lval* x`.

Разве в Lisp не используются cons-ячейки?

В других диалектах Lisp S-выражения определяются немного иначе: обычно это либо атом, например символ или число, либо пара объединённых операцией *cons* S-выражений.

Это естественным образом приводит к реализации через связные списки — структуру, отличную от нашей. В этой книге ради простоты я представляю S-выражения массивом переменного размера, но важно знать, что формальное определение и типичная реализация несколько отличаются.

```
lval* lval_read_num(mpc_ast_t* t) {
  errno = 0;
  long x = strtol(t->contents, NULL, 10);
  return errno != ERANGE ?
    lval_num(x) : lval_err("invalid number");
}
```

```
lval* lval_read(mpc_ast_t* t) {

  /* If Symbol or Number return conversion to that type */
  if (strstr(t->tag, "number")) { return lval_read_num(t); }
  if (strstr(t->tag, "symbol")) { return lval_sym(t->contents); }
```

```
  /* If root (>) or sexpr then create empty list */
  lval* x = NULL;
  if (strcmp(t->tag, ">") == 0) { x = lval_sexpr(); }
  if (strstr(t->tag, "sexpr"))  { x = lval_sexpr(); }

  /* Fill this list with any valid expression contained within */
  for (int i = 0; i < t->children_num; i++) {
    if (strcmp(t->children[i]->contents, "(") == 0) { continue; }
    if (strcmp(t->children[i]->contents, ")") == 0) { continue; }
    if (strcmp(t->children[i]->tag,  "regex") == 0) { continue; }
    x = lval_add(x, lval_read(t->children[i]));
  }

  return x;
}
```

```
lval* lval_add(lval* v, lval* x) {
  v->count++;
  v->cell = realloc(v->cell, sizeof(lval*) * v->count);
  v->cell[v->count-1] = x;
  return v;
}
```

Вывод выражений

Мы почти готовы испытать изменения. Нужно доработать функцию вывода для S-выражений. Выведя прочитанные выражения и сравнив их с введёнными, мы проверим этап чтения.

Создадим функцию, которая перебирает подвыражения и выводит их через пробелы — так же, как они вводятся.

```
void lval_expr_print(lval* v, char open, char close) {
  putchar(open);
  for (int i = 0; i < v->count; i++) {

    /* Print Value contained within */
    lval_print(v->cell[i]);

    /* Don't print trailing space if last element */
    if (i != (v->count-1)) {
      putchar(' ');
    }
  }
  putchar(close);
}
```

```
void lval_print(lval* v) {
  switch (v->type) {
    case LVAL_NUM:   printf("%li", v->num); break;
    case LVAL_ERR:   printf("Error: %s", v->err); break;
    case LVAL_SYM:   printf("%s", v->sym); break;
    case LVAL_SEXPR: lval_expr_print(v, '(', ')'); break;
  }
}

void lval_println(lval* v) { lval_print(v); putchar('\n'); }
```

Не получается объявить функции: они вызывают друг друга.

`lval_expr_print` вызывает `lval_print`, и наоборот. Перестановкой в файле эту зависимость не устранить. Вместо этого нужно заранее объявить одну функцию без тела. Тогда другие функции смогут её вызывать, а определить её можно позднее. Запишите определение, заменив тело точкой с запятой `;`. В этом примере поместите `void lval_print(lval* v);` перед `lval_expr_print`.

Вы наверняка встретите это ещё раз, а я не всегда буду предупреждать, поэтому запомните решение!

В основном цикле пока уберём вычисление и вместо него прочитаем результат и выведем прочитанное:

```
lval* x = lval_read(r.output);
lval_println(x);
lval_del(x);
```

Если всё работает, ввод программы даст примерно такой результат:

```
lispy> + 2 2
(+ 2 2)
lispy> + 2 (* 7 6) (* 2 5)
(+ 2 (* 7 6) (* 2 5))
lispy> *     55     101  (+ 0 0 0)
(* 55 101 (+ 0 0 0))
lispy>
```

Вычисление выражений

Поведение функции вычисления в целом остаётся прежним. Приспособим её к `lval*` и более свободному определению выражения. Можно представить функцию как преобразователь: она принимает `lval*` и возвращает другой. Иногда возвращается тот же объект; иногда вход изменяется и возвращается; во многих случаях вход удаляется и возвращается совсем другое значение. Возвращая новый объект, всегда не забывайте удалять входной `lval*`.

Для S-выражения сначала вычисляем всех его потомков. Если среди результатов есть ошибки, возвращаем первую с помощью функции `lval_take`, которую определим позже.

Если потомков нет, возвращаем само выражение: это пустое выражение `()`. Если потомок один, как в `(5)`, возвращаем его.

Иначе в выражении больше одного элемента. Отделяем первый функцией `lval_pop`, которую тоже определим позже, и проверяем, что он является символом. Если это символ, передаём его вместе с аргументами функции `builtin_op` для вычисления. Если нет — удаляем первый элемент и остальные значения, после чего возвращаем ошибку.

Все остальные типы возвращаются без изменений.

```
lval* lval_eval_sexpr(lval* v) {

  /* Evaluate Children */
  for (int i = 0; i < v->count; i++) {
    v->cell[i] = lval_eval(v->cell[i]);
  }

  /* Error Checking */
  for (int i = 0; i < v->count; i++) {
    if (v->cell[i]->type == LVAL_ERR) { return lval_take(v, i); }
  }

  /* Empty Expression */
  if (v->count == 0) { return v; }

  /* Single Expression */
  if (v->count == 1) { return lval_take(v, 0); }

  /* Ensure First Element is Symbol */
  lval* f = lval_pop(v, 0);
  if (f->type != LVAL_SYM) {
    lval_del(f); lval_del(v);
    return lval_err("S-expression Does not start with symbol!");
  }

  /* Call builtin with operator */
  lval* result = builtin_op(v, f->sym);
  lval_del(f);
  return result;
}
```

```
lval* lval_eval(lval* v) {
  /* Evaluate Sexpressions */
  if (v->type == LVAL_SEXPR) { return lval_eval_sexpr(v); }
  /* All other lval types remain the same */
  return v;
}
```

В коде использованы ещё не определённые функции `lval_pop` и `lval_take`. Это функции общего назначения для работы с S-выражениями, которые пригодятся и сейчас, и позже.

`lval_pop` извлекает элемент с индексом `i` и сдвигает остальные элементы списка, чтобы удалённого `lval*` в нём больше не было. Затем она возвращает извлечённое значение, не удаляя исходный список. Следовательно, и извлечённый элемент, и оставшийся список в какой-то момент необходимо удалить через `lval_del`.

`lval_take` похожа на `lval_pop`, но удаляет список, из которого извлекла элемент. Это небольшое отличие делает код местами понятнее. После неё через `lval_del` нужно удалить только взятый элемент.

```
lval* lval_pop(lval* v, int i) {
  /* Find the item at "i" */
  lval* x = v->cell[i];

  /* Shift memory after the item at "i" over the top */
  memmove(&v->cell[i], &v->cell[i+1],
    sizeof(lval*) * (v->count-i-1));

  /* Decrease the count of items in the list */
  v->count--;

  /* Reallocate the memory used */
```

```
  v->cell = realloc(v->cell, sizeof(lval*) * v->count);
  return x;
}
```

```
lval* lval_take(lval* v, int i) {
  lval* x = lval_pop(v, i);
  lval_del(v);
  return x;
}
```

Также определим `builtin_op`. Она похожа на `eval_op` из предыдущей главы, но принимает один `lval*` со списком всех аргументов. Проверка ошибок должна быть строже: если какой-либо аргумент не является числом, нужно вернуть ошибку.

Сначала функция проверяет, что все аргументы — числа, и извлекает первый. Если подвыражений больше нет, а оператор — вычитание, она меняет знак первого числа. Благодаря этому выражения вроде `(- 5)` вычисляются правильно.

Если аргументы остались, функция последовательно извлекает их и выполняет соответствующую арифметическую операцию. При делении на ноль она удаляет временные значения `x`, `y` и список аргументов `a`, затем возвращает ошибку. Если ошибок не было, входные аргументы удаляются и возвращается новое выражение.

```
lval* builtin_op(lval* a, char* op) {

  /* Ensure all arguments are numbers */
  for (int i = 0; i < a->count; i++) {
    if (a->cell[i]->type != LVAL_NUM) {
      lval_del(a);
      return lval_err("Cannot operate on non-number!");
    }
  }

  /* Pop the first element */
  lval* x = lval_pop(a, 0);

  /* If no arguments and sub then perform unary negation */
  if ((strcmp(op, "-") == 0) && a->count == 0) {
    x->num = -x->num;
  }

  /* While there are still elements remaining */
  while (a->count > 0) {

    /* Pop the next element */
    lval* y = lval_pop(a, 0);

    if (strcmp(op, "+") == 0) { x->num += y->num; }
    if (strcmp(op, "-") == 0) { x->num -= y->num; }
    if (strcmp(op, "*") == 0) { x->num *= y->num; }
    if (strcmp(op, "/") == 0) {
      if (y->num == 0) {
        lval_del(x); lval_del(y);
        x = lval_err("Division By Zero!"); break;
      }
      x->num /= y->num;
    }

    lval_del(y);
  }
```

```
  lval_del(a); return x;
}
```

Функции вычисления готовы. Осталось снова изменить `main`, чтобы входные данные вычислялись перед выводом.

```
lval* x = lval_eval(lval_read(r.output));
lval_println(x);
lval_del(x);
```

Теперь выражения должны вычисляться правильно, как и в предыдущей главе. Передохните и поэкспериментируйте с новой реализацией. Убедитесь, что всё работает как ожидалось. В следующей главе эти изменения помогут реализовать новые интересные возможности.

```
lispy> + 1 (* 7 5) 3
39
lispy> (- 100)
-100
lispy>
()
lispy> /
/
lispy> (/ ())
Error: Cannot operate on non-number!
lispy>
```

Справочные материалы

Исходный код: s_expressions.c.

Дополнительные задания

- › Приведите пример переменной нашей программы, находящейся в стеке.
- › Приведите пример переменной нашей программы, указывающей на кучу.
- › Что делает функция `strcpy`?
- › Что делает функция `realloc`?
- › Что делает функция `memmove`?
- › Чем `memmove` отличается от `memcpy`?
- › Добавьте в разбор и вычисление оператор остатка `%`.
- › Добавьте поддержку дробных чисел с помощью поля типа `double`.

Q-выражения • Глава 10

Добавление возможностей

Вы заметите, что следующие главы построены по одной схеме. Она отражает типичный подход к добавлению возможностей языка: несколько шагов от начала до конца. Они перечислены ниже, и именно их мы выполним в этой главе, добавляя Q-выражения.

Синтаксис	Добавить правило в грамматику языка.
Представление	Добавить разновидность типа данных для новой возможности.
Разбор	Добавить функции чтения из абстрактного синтаксического дерева.
Семантика	Добавить функции вычисления и обработки.

Цитируемые выражения

В этой главе мы реализуем новый тип значения Lisp — Q-выражение. Буква Q означает *quoted expression*, то есть цитируемое выражение. Оно не вычисляется обычным механизмом Lisp: функция вычисления оставляет его как есть. Поэтому Q-выражения отлично подходят для хранения и обработки других значений Lisp — чисел, символов и даже S-выражений.

Добавив Q-выражения, мы реализуем небольшой набор операторов для работы с ними. Подобно арифметическим операторам, они станут основой наших представлений об выражениях и экспериментов с ними.

Синтаксис Q-выражений похож на синтаксис S-выражений, только вместо круглых скобок `()` используются фигурные `{}`. Добавим это в грамматику:

> **Я никогда не слышал о Q-выражениях.**
>
> В других диалектах Lisp их нет. Там для предотвращения вычисления используются *макросы*: они похожи на обычные функции, но не вычисляют аргументы. Специальный макрос quote `'` позволяет предотвратить вычисление почти чего угодно. Он послужил источником идеи Q-выражений, уникальных для нашего Lisp. Вместо макросов мы будем использовать их для тех же задач и не только.
>
> Моё использование терминов S-выражение и Q-выражение несколько вольно, но надеюсь, эта маленькая неточность делает поведение нашего Lisp понятнее.

```
mpc_parser_t* Number = mpc_new("number");
mpc_parser_t* Symbol = mpc_new("symbol");
```

```
mpc_parser_t* Sexpr  = mpc_new("sexpr");
mpc_parser_t* Qexpr  = mpc_new("qexpr");
mpc_parser_t* Expr   = mpc_new("expr");
mpc_parser_t* Lispy  = mpc_new("lispy");

mpca_lang(MPCA_LANG_DEFAULT,
  "                                                     \
    number : /-?[0-9]+/ ;                               \
    symbol : '+' | '-' | '*' | '/' ;                    \
    sexpr  : '(' <expr>* ')' ;                          \
    qexpr  : '{' <expr>* '}' ;                          \
    expr   : <number> | <symbol> | <sexpr> | <qexpr> ; \
    lispy  : /^/ <expr>* /$/ ;                          \
  ",
  Number, Symbol, Sexpr, Qexpr, Expr, Lispy);
```

Не забудем обновить функцию очистки с учётом нового правила:

```
mpc_cleanup(6, Number, Symbol, Sexpr, Qexpr, Expr, Lispy);
```

Чтение Q-выражений

Q-выражения очень похожи на S-выражения, поэтому многое в их внутреннем поведении будет одинаковым. Мы используем те же поля данных, но добавим в перечисление отдельный тип.

```
enum { LVAL_ERR, LVAL_NUM, LVAL_SYM, LVAL_SEXPR, LVAL_QEXPR };
```

Добавим и конструктор:

```
/* A pointer to a new empty Qexpr lval */
lval* lval_qexpr(void) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_QEXPR;
  v->count = 0;
  v->cell = NULL;
  return v;
}
```

Вывод и удаление Q-выражений выполняются почти так же, как для S-выражений. Добавим соответствующие строки в наши функции:

```
void lval_print(lval* v) {
  switch (v->type) {
    case LVAL_NUM:   printf("%li", v->num); break;
    case LVAL_ERR:   printf("Error: %s", v->err); break;
    case LVAL_SYM:   printf("%s", v->sym); break;
    case LVAL_SEXPR: lval_expr_print(v, '(', ')'); break;
    case LVAL_QEXPR: lval_expr_print(v, '{', '}'); break;
  }
}
```

```
void lval_del(lval* v) {

  switch (v->type) {
    case LVAL_NUM: break;
```

```
    case LVAL_ERR: free(v->err); break;
    case LVAL_SYM: free(v->sym); break;

    /* If Qexpr or Sexpr then delete all elements inside */
    case LVAL_QEXPR:
    case LVAL_SEXPR:
      for (int i = 0; i < v->count; i++) {
        lval_del(v->cell[i]);
      }
      /* Also free the memory allocated to contain the pointers */
      free(v->cell);
    break;
  }

  free(v);
}
```

Теперь обновим функцию чтения `lval_read`. Поскольку поля данных общие, можно повторно использовать функции для S-выражений, например `lval_add`. Нужно лишь добавить в `lval_read` создание пустого Q-выражения непосредственно после кода, который обнаруживает и создаёт пустые S-выражения из абстрактного синтаксического дерева.

```
if (strstr(t->tag, "qexpr"))  { x = lval_qexpr(); }
```

Также научим `lval_read` распознавать фигурные скобки:

```
if (strcmp(t->children[i]->contents, "(") == 0) { continue; }
if (strcmp(t->children[i]->contents, ")") == 0) { continue; }
if (strcmp(t->children[i]->contents, "}") == 0) { continue; }
if (strcmp(t->children[i]->contents, "{") == 0) { continue; }
```

Особого способа вычисления Q-выражений нет, поэтому функции вычисления менять не нужно. Всё готово к проверке: скомпилируйте и запустите программу, попробуйте новый тип данных и убедитесь, что выражения не вычисляются.

```
lispy> {1 2 3 4}
{1 2 3 4}
lispy> {1 2 (+ 5 6) 4}
{1 2 (+ 5 6) 4}
lispy> {{2 3 4} {1}}
{{2 3 4} {1}}
lispy>
```

Встроенные функции

Мы умеем читать Q-выражения, но пока они бесполезны: нужен способ работать с ними.

Определим встроенные операторы для нашего типа списка. Важно выбрать компактный набор: несколько основных операций позволят определять новые без дополнительного кода на C. Возможны разные варианты, но я выбрал набор, которого нам хватит для всего необходимого:

`list`	Принимает один или несколько аргументов и возвращает новое Q-выражение, содержащее их.
`head`	Принимает Q-выражение и возвращает Q-выражение только с первым элементом.
`tail`	Принимает Q-выражение и возвращает его без первого элемента.
`join`	Принимает одно или несколько Q-выражений и объединяет их.
`eval`	Принимает Q-выражение и вычисляет его как S-выражение.

Как и математические операторы, добавим эти функции в список допустимых символов, а затем определим их поведение аналогично `builtin_op`.

```
mpca_lang(MPCA_LANG_DEFAULT,
  "                                                     \
    number : /-?[0-9]+/ ;                               \
    symbol : \"list\" | \"head\" | \"tail\"             \
           | \"join\" | \"eval\" | '+' | '-' | '*' | '/' ; \
    sexpr  : '(' <expr>* ')' ;                          \
    qexpr  : '{' <expr>* '}' ;                          \
    expr   : <number> | <symbol> | <sexpr> | <qexpr> ;  \
    lispy  : /^/ <expr>* /$/ ;                          \
  ",
  Number, Symbol, Sexpr, Qexpr, Expr, Lispy)
```

Первая попытка

Встроенные функции должны иметь тот же интерфейс, что и `builtin_op`: аргументы объединены в S-выражение, которое функция использует и затем удаляет. Результат вычисления возвращается как новый `lval*`.

Реализовать получение головы и хвоста Q-выражения несложно: воспользуемся функциями для S-выражений, например `lval_take` и `lval_pop`. Но, как и в `builtin_op`, нужно проверять корректность входных данных.

Рассмотрим сначала `head` и `tail`. Обе функции требуют ровно одного аргумента, причём он должен быть непустым Q-выражением.

Функция `head` может извлекать и удалять элемент с индексом `1`, пока в списке не останется только первый элемент. Функция `tail` ещё проще: она извлекает и удаляет элемент с индексом `0`, оставляя хвост. Первая версия может выглядеть так:

```
lval* builtin_head(lval* a) {
  /* Check Error Conditions */
  if (a->count != 1) {
    lval_del(a);
    return lval_err("Function 'head' passed too many arguments!");
  }

  if (a->cell[0]->type != LVAL_QEXPR) {
    lval_del(a);
    return lval_err("Function 'head' passed incorrect types!");
  }
```

```
  if (a->cell[0]->count == 0) {
    lval_del(a);
    return lval_err("Function 'head' passed {}!");
  }

  /* Otherwise take first argument */
  lval* v = lval_take(a, 0);

  /* Delete all elements that are not head and return */
  while (v->count > 1) { lval_del(lval_pop(v, 1)); }
  return v;
}
```

```
lval* builtin_tail(lval* a) {
  /* Check Error Conditions */
  if (a->count != 1) {
    lval_del(a);
    return lval_err("Function 'tail' passed too many arguments!");
  }

  if (a->cell[0]->type != LVAL_QEXPR) {
    lval_del(a);
    return lval_err("Function 'tail' passed incorrect types!");
  }

  if (a->cell[0]->count == 0) {
    lval_del(a);
    return lval_err("Function 'tail' passed {}!");
  }

  /* Take first argument */
  lval* v = lval_take(a, 0);

  /* Delete first element and return */
  lval_del(lval_pop(v, 0));
  return v;
}
```

Макросы

Функции `head` и `tail` работают правильно, но код длинный и не очень понятный: за проверками ошибок трудно увидеть основное действие. Один способ привести его в порядок — использовать *макрос*.



Клубника • Вкусное макро

Макрос — конструкция препроцессора, создающая нечто похожее на функцию и обрабатываемая до компиляции. Макросы применяются по-разному; здесь они помогут сделать код чище.

Макрос принимает аргументы, которыми может быть почти что угодно, и подставляет их в заданный шаблон. Изменяя шаблон или аргументы, мы меняем создаваемый код. Для определения используется директива препроцессора `#define`, затем имя макроса и имена аргументов в скобках, а после — шаблон создаваемого кода.

Создадим макрос `LASSERT` для проверки ошибок. Имена макросов обычно пишут заглавными буквами, чтобы отличать их от обычных функций C. Макрос принимает три аргумента: `args`, `cond` и `err`, и подставляет их в показанный шаблон. Такой шаблон подходит для всех наших проверок.

```
#define LASSERT(args, cond, err) \
  if (!(cond)) { lval_del(args); return lval_err(err); }
```

С его помощью можно переписать функции, не меняя фактически создаваемый код. Программисту станет легче его читать, а печатать придётся меньше. Остальные проверки ошибок тоже будет легко написать!

Head и Tail

Теперь определим `head` и `tail` так. Обратите внимание, насколько яснее видно их основное действие.

```
lval* builtin_head(lval* a) {
  LASSERT(a, a->count == 1,
    "Function 'head' passed too many arguments!");
  LASSERT(a, a->cell[0]->type == LVAL_QEXPR,
    "Function 'head' passed incorrect type!");
  LASSERT(a, a->cell[0]->count != 0,
    "Function 'head' passed {}!");

  lval* v = lval_take(a, 0);
  while (v->count > 1) { lval_del(lval_pop(v, 1)); }
  return v;
}
```

```
lval* builtin_tail(lval* a) {
  LASSERT(a, a->count == 1,
    "Function 'tail' passed too many arguments!");
  LASSERT(a, a->cell[0]->type == LVAL_QEXPR,
    "Function 'tail' passed incorrect type!");
  LASSERT(a, a->cell[0]->count != 0,
    "Function 'tail' passed {}!");

  lval* v = lval_take(a, 0);
  lval_del(lval_pop(v, 0));
  return v;
}
```

List и Eval

Функция `list` проста: она превращает входное S-выражение в Q-выражение и возвращает его.

`eval` выполняет похожее обратное действие: принимает одно Q-выражение, превращает его в S-выражение и вычисляет с помощью `lval_eval`.

```
lval* builtin_list(lval* a) {
  a->type = LVAL_QEXPR;
```

```
  return a;
}
```

```
lval* builtin_eval(lval* a) {
  LASSERT(a, a->count == 1,
    "Function 'eval' passed too many arguments!");
  LASSERT(a, a->cell[0]->type == LVAL_QEXPR,
    "Function 'eval' passed incorrect type!");

  lval* x = lval_take(a, 0);
  x->type = LVAL_SEXPR;
  return lval_eval(x);
}
```

Join

Осталось определить `join`. В отличие от остальных, она может получать несколько аргументов, поэтому её структура больше похожа на `builtin_op`. Сначала проверяем, что все аргументы — Q-выражения, а затем соединяем их по одному функцией `lval_join`. Она последовательно извлекает элементы из `y` и добавляет к `x`, пока `y` не опустеет; затем удаляет `y` и возвращает `x`.

```
lval* builtin_join(lval* a) {

  for (int i = 0; i < a->count; i++) {
    LASSERT(a, a->cell[i]->type == LVAL_QEXPR,
      "Function 'join' passed incorrect type.");
  }

  lval* x = lval_pop(a, 0);

  while (a->count) {
    x = lval_join(x, lval_pop(a, 0));
  }

  lval_del(a);
  return x;
}
```

```
lval* lval_join(lval* x, lval* y) {

  /* For each cell in 'y' add it to 'x' */
  while (y->count) {
    x = lval_add(x, lval_pop(y, 0));
  }

  /* Delete the empty 'y' and return 'x' */
  lval_del(y);
  return x;
}
```

Поиск встроенных функций

Все встроенные функции определены. Теперь нужна функция, которая при вычислении вызовет нужную по встреченному символу. Используем `strcmp` и `strstr`:

```
lval* builtin(lval* a, char* func) {
  if (strcmp("list", func) == 0) { return builtin_list(a); }
  if (strcmp("head", func) == 0) { return builtin_head(a); }
  if (strcmp("tail", func) == 0) { return builtin_tail(a); }
  if (strcmp("join", func) == 0) { return builtin_join(a); }
  if (strcmp("eval", func) == 0) { return builtin_eval(a); }
  if (strstr("+-/*", func)) { return builtin_op(a, func); }
  lval_del(a);
  return lval_err("Unknown Function!");
}
```

Затем изменим строку вычисления в `lval_eval_sexpr`, чтобы она вызывала `builtin` вместо `builtin_op`:

```
/* Call builtin with operator */
lval* result = builtin(v, f->sym);
lval_del(f);
return result;
```

Теперь наш язык должен полностью поддерживать Q-выражения. Скомпилируйте и запустите последнюю версию, попробуйте новые операторы списков. Помещайте код и символы в списки и вычисляйте их разными способами. Возможность помещать S-выражения в список с помощью Q-выражений замечательна: она позволяет обращаться с кодом как с данными. Это одна из главных особенностей Lisp, которую фактически нельзя воспроизвести в таких языках, как C!

```
lispy> list 1 2 3 4
{1 2 3 4}
lispy> {head (list 1 2 3 4)}
{head (list 1 2 3 4)}
lispy> eval {head (list 1 2 3 4)}
{1}
lispy> tail {tail tail tail}
{tail tail}
lispy> eval (tail {tail tail {5 6 7}})
{6 7}
lispy> eval (head {(+ 1 2) (+ 10 20)})
3
```

Справочные материалы

Исходный код: q_expressions.c.

Дополнительные задания

- › Назовите четыре типичных шага добавления возможностей языка.
- › Создайте макрос специально для проверки неверного количества аргументов.
- › Создайте макрос для проверки вызова с пустым списком.
- › Добавьте встроенную функцию `cons`, которая принимает значение и Q-выражение и добавляет значение в начало.
- › Добавьте встроенную функцию `len`, возвращающую количество элементов Q-выражения.
- › Добавьте встроенную функцию `init`, возвращающую все элементы Q-выражения, кроме последнего.

Переменные • Глава 11

Неизменяемость

В предыдущих главах мы существенно продвинулись в построении основы нашего языка.

Мы уже можем делать ряд интересных вещей, недоступных другим языкам, например помещать код в списки. Пришло время добавить *возможности*, которые сделают язык практически полезным. Первой из них будут *переменные*.



Черепашка-ниндзя • Не неизменяемая.

Они называются переменными, но название это обманчиво: наши переменные не будут меняться. Они *неизменяемы*, то есть изменить их нельзя. До сих пор всё в нашем языке вело себя как *неизменяемое*. Вычисляя выражение, мы представляли, что прежний объект удаляется и возвращается новый. В реализации нам часто проще повторно использовать данные прежнего объекта при создании нового, но с точки зрения концепции такой способ представлять работу языка очень полезен.

Таким образом, наши переменные — это просто способ *называть значения*. Они позволяют присвоить *имя* некоторому *значению*, а позднее получить копию этого *значения*, когда оно понадобится.

Чтобы *называть значения*, нужно создать структуру, хранящую имена и значения всех именованных объектов программы. Мы называем её *окружением*. При запуске интерактивной оболочки мы создадим для неё новое окружение, в котором будет вычисляться каждый новый ввод. Так во время программирования мы сможем сохранять и извлекать переменные.

Что произойдёт, если заново связать имя с другим значением? Разве это не изменяемость?

В нашем Лиспе при повторном присваивании имени мы удалим старую связь и создадим новую. Возникает иллюзия, будто объект, связанный с именем, изменился и является изменяемым, но на самом деле мы удалили старый объект и присвоили новый. Это отличается от C, где данные, на которые указывает указатель или которые хранятся в структуре, действительно можно изменить без удаления и создания заново.

Синтаксис символов

Теперь, когда пользователи смогут определять переменные, нужно сделать грамматику символов более гибкой. Она должна распознавать не только встроенные функции, но и любые допустимые символы. В отличие от C, где имена переменных довольно сильно ограничены, мы разрешим использовать в них самые разные знаки.

Доступный диапазон символов можно выразить следующим регулярным выражением.

```
/[a-zA-Z0-9_+\\-*\\/\\\\=<>!&]+/
```

На первый взгляд кажется, будто мы просто ударили руками по клавиатуре. На самом деле это регулярное выражение с большим указателем диапазона `[]`. Внутри него специальные символы теряют своё значение, но некоторые всё же нужно экранировать обратной косой чертой. Поскольку выражение находится внутри строки C, для одной обратной косой черты во входных данных нужно написать две.

Это правило допускает обычные символы идентификаторов C `a-zA-Z0-9_`, символы арифметических операторов `+\\-*\\/`, обратную косую черту `\\\\`, операторы сравнения `=<>!` и амперсанд `&`. Этого достаточно для гибкого определения новых и существующих символов.

```
mpca_lang(MPCA_LANG_DEFAULT,
  "                                          \
    number : /-?[0-9]+/ ;                    \
    symbol : /[a-zA-Z0-9_+\\-*\\/\\\\=<>!&]+/ ;  \
    sexpr  : '(' <expr>* ')' ;               \
    qexpr  : '{' <expr>* '}' ;               \
    expr   : <number> | <symbol> | <sexpr> | <qexpr> ;  \
    lispy  : /^/ <expr>* /$/ ;               \
  ",
  Number, Symbol, Sexpr, Qexpr, Expr, Lispy);
```

Указатели на функции

После добавления переменных символы перестанут обозначать только функции: они станут именами, по которым мы ищем в окружении и получаем значение.

Поэтому нам нужен новый вид значения для представления функций, который можно вернуть при встрече со встроенным символом. Чтобы создать этот тип `lval`, мы воспользуемся *указателем на функцию*.

Указатели на функции — замечательная возможность C, позволяющая хранить и передавать указатели на функции. Изменять данные по таким указателям бессмысленно. Вместо этого мы вызываем функцию, на которую указывает указатель, как обычную функцию.

Как и обычные указатели, указатели на функции имеют тип. Он определяет тип функции, на которую они указывают, а не тип данных. Благодаря этому компилятор может проверить правильность вызова.

В предыдущей главе встроенные функции принимали `lval*` и возвращали `lval*`. В этой главе они будут дополнительно принимать указатель на окружение `lenv*`. Для таких функций можно объявить новый тип указателя `lbuiltin` следующим образом.

```
typedef lval*(*lbuiltin)(lenv*, lval*);
```

Почему синтаксис такой необычный?

Местами синтаксис C может выглядеть особенно странно. Полезно понять, почему он именно такой. Разберём объявление выше по частям.

Сначала `typedef`. Его можно поставить перед любым обычным объявлением переменной. В результате имя переменной становится именем нового типа, соответствующего типу, который был бы выведен для этой переменной. Поэтому в объявлении выше то, что похоже на имя функции, становится именем нового типа.

Теперь все эти звёздочки `*`. В C звёздочку указателя правильнее воспринимать как относящуюся к имени переменной слева, а не к типу справа: `int *x;`. Синтаксис типов C работает как своего рода вывод. Вместо *«Создать указатель* `x` *на* `int`*»* его следует читать так: *«Создать переменную* `x`*, разыменование которой даёт* `int`*»*. Следовательно, `x` выводится как указатель на `int`.

Эта идея распространяется и на указатели на функции. Объявление выше можно прочесть так: *«Чтобы получить* `lval*`*, разыменуем* `lbuiltin` *и вызовем его с* `lenv*` *и* `lval*`*»*. Следовательно, `lbuiltin` — указатель на функцию, принимающую `lenv*` и `lval*` и возвращающую `lval*`.

Циклические зависимости типов

Тип `lbuiltin` ссылается на типы `lval` и `lenv`. Значит, они должны быть объявлены ранее в исходном файле.

Но мы хотим добавить поле `lbuiltin` в структуру `lval`, чтобы создавать значения-функции. Значит, объявление `lbuiltin` должно предшествовать объявлению `lval`. Возникает циклическая зависимость типов: два типа зависят друг от друга.

Ранее мы встречали такую проблему со взаимозависимыми функциями. Её решило *предварительное объявление* функции без её тела.

В C то же самое можно сделать с типами. Сначала объявим два типа `struct` без тела. Затем с помощью typedef дадим им имена `lval` и `lenv`. После этого определим тип указателя на функцию `lbuiltin` и, наконец, тело структуры `lval`. Теперь проблемы с типами решены, и компилятор перестанет жаловаться.

```
/* Forward Declarations */
```

```
struct lval;
struct lenv;
typedef struct lval lval;
typedef struct lenv lenv;

/* Lisp Value */

enum { LVAL_ERR, LVAL_NUM,   LVAL_SYM,
       LVAL_FUN, LVAL_SEXPR, LVAL_QEXPR };

typedef lval*(*lbuiltin)(lenv*, lval*);

struct lval {
  int type;

  long num;
  char* err;
  char* sym;
  lbuiltin fun;

  int count;
  lval** cell;
};
```

Тип функции

Мы добавили новый тип `lval` с элементом перечисления `LVAL_FUN`. Нужно обновить все соответствующие функции, работающие с `lval`, чтобы они правильно обрабатывали новый тип. В большинстве случаев достаточно добавить ветви в операторы switch.

Начнём с новой функции-конструктора этого типа.

```
lval* lval_fun(lbuiltin func) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_FUN;
  v->fun = func;
  return v;
}
```

При **удалении** указателя на функцию делать ничего особенного не нужно.

```
case LVAL_FUN: break;
```

При **печати** можно просто вывести условную строку.

```
case LVAL_FUN:   printf("<function>"); break;
```

Мы также добавим функцию **копирования** `lval`. Она пригодится при помещении значений в окружение и извлечении из него. Для чисел и функций можно напрямую скопировать соответствующие поля. Для строк потребуются `malloc` и `strcpy`. Для списков нужно выделить достаточно памяти и скопировать каждый элемент отдельно.

```
lval* lval_copy(lval* v) {

  lval* x = malloc(sizeof(lval));
  x->type = v->type;
```

```
  switch (v->type) {

    /* Copy Functions and Numbers Directly */
    case LVAL_FUN: x->fun = v->fun; break;
    case LVAL_NUM: x->num = v->num; break;

    /* Copy Strings using malloc and strcpy */
    case LVAL_ERR:
      x->err = malloc(strlen(v->err) + 1);
      strcpy(x->err, v->err); break;

    case LVAL_SYM:
      x->sym = malloc(strlen(v->sym) + 1);
      strcpy(x->sym, v->sym); break;

    /* Copy Lists by copying each sub-expression */
    case LVAL_SEXPR:
    case LVAL_QEXPR:
      x->count = v->count;
      x->cell = malloc(sizeof(lval*) * x->count);
      for (int i = 0; i < x->count; i++) {
        x->cell[i] = lval_copy(v->cell[i]);
      }
    break;
  }

  return x;
}
```

Окружение

Структура окружения должна хранить список связей между *именами* и *значениями*. Построить такую структуру можно многими способами. Мы выберем простейший хорошо работающий вариант: два списка одинаковой длины. Один содержит `lval*`, другой — `char*`. Каждому элементу одного списка соответствует элемент другого на той же позиции.

Мы уже предварительно объявили структуру `lenv`, поэтому теперь можем определить её.

```
struct lenv {
  int count;
  char** syms;
  lval** vals;
};
```

Нам нужны функции создания и удаления этой структуры. Они довольно просты: создание инициализирует поля, а удаление перебирает элементы обоих списков и удаляет их или освобождает память.

```
lenv* lenv_new(void) {
  lenv* e = malloc(sizeof(lenv));
  e->count = 0;
  e->syms = NULL;
  e->vals = NULL;
  return e;
}
```

```
void lenv_del(lenv* e) {
  for (int i = 0; i < e->count; i++) {
    free(e->syms[i]);
    lval_del(e->vals[i]);
  }
  free(e->syms);
  free(e->vals);
  free(e);
}
```

Далее создадим две функции для получения значений из окружения и помещения их в него.

Чтобы получить значение, перебираем элементы окружения и проверяем, совпадает ли заданный символ с одной из сохранённых строк. Если совпадение найдено, возвращаем копию сохранённого значения. Иначе возвращаем ошибку.

Функция помещения новой переменной немного сложнее. Сначала проверяем, существует ли уже переменная с таким именем. Если да, заменяем её значение новым. Для этого перебираем существующие переменные и сравниваем имена. Найдя совпадение, удаляем значение в этой ячейке и сохраняем там копию входного значения.

Если значение с таким именем не найдено, нужно выделить дополнительную память. Воспользуемся `realloc` и сохраним копии `lval` и имени в новых ячейках.

```
lval* lenv_get(lenv* e, lval* k) {

  /* Iterate over all items in environment */
  for (int i = 0; i < e->count; i++) {
    /* Check if the stored string matches the symbol string */
    /* If it does, return a copy of the value */
    if (strcmp(e->syms[i], k->sym) == 0) {
      return lval_copy(e->vals[i]);
    }
  }
  /* If no symbol found return error */
  return lval_err("unbound symbol!");
}
```

```
void lenv_put(lenv* e, lval* k, lval* v) {

  /* Iterate over all items in environment */
  /* This is to see if variable already exists */
  for (int i = 0; i < e->count; i++) {

    /* If variable is found delete item at that position */
    /* And replace with variable supplied by user */
    if (strcmp(e->syms[i], k->sym) == 0) {
      lval_del(e->vals[i]);
      e->vals[i] = lval_copy(v);
      return;
    }
  }

  /* If no existing entry found allocate space for new entry */
  e->count++;
  e->vals = realloc(e->vals, sizeof(lval*) * e->count);
  e->syms = realloc(e->syms, sizeof(char*) * e->count);

  /* Copy contents of lval and symbol string into new location */
  e->vals[e->count-1] = lval_copy(v);
  e->syms[e->count-1] = malloc(strlen(k->sym)+1);
```

```
  strcpy(e->syms[e->count-1], k->sym);
}
```

Вычисление переменных

Теперь вычисление зависит от окружения. Нужно передавать его как аргумент и использовать для получения значения при встрече с символом. Поскольку окружение возвращает копию значения, не забудьте удалить входной символ `lval`.

```
lval* lval_eval(lenv* e, lval* v) {
  if (v->type == LVAL_SYM) {
    lval* x = lenv_get(e, v);
    lval_del(v);
    return x;
  }
  if (v->type == LVAL_SEXPR) { return lval_eval_sexpr(e, v); }
  return v;
}
```

Поскольку мы добавили тип функции, вычисление S-выражений тоже меняется. Вместо проверки на символ нужно проверить, является ли значение функцией. Если да, можно вызвать поле `fun` значения `lval`, используя обычный синтаксис вызова функции.

```
lval* lval_eval_sexpr(lenv* e, lval* v) {

  for (int i = 0; i < v->count; i++) {
    v->cell[i] = lval_eval(e, v->cell[i]);
  }

  for (int i = 0; i < v->count; i++) {
    if (v->cell[i]->type == LVAL_ERR) { return lval_take(v, i); }
  }

  if (v->count == 0) { return v; }
  if (v->count == 1) { return lval_take(v, 0); }

  /* Ensure first element is a function after evaluation */
  lval* f = lval_pop(v, 0);
  if (f->type != LVAL_FUN) {
    lval_del(v); lval_del(f);
    return lval_err("first element is not a function");
  }

  /* If so call function to get result */
  lval* result = f->fun(e, v);
  lval_del(f);
  return result;
}
```

Встроенные функции

Теперь вычисление опирается на новый тип функции, поэтому до запуска интерактивной оболочки нужно зарегистрировать все встроенные функции в окружении. Сейчас их типы не подходят. Измените сигнатуры так, чтобы они принимали окружение, а там, где необходимо, передавали его другим вызываемым функциям. Я не привожу здесь этот код: измените

сигнатуры встроенных функций, добавив `lenv*` первым аргументом. Если возникнут трудности, обратитесь к примеру кода этой главы.

Например, с помощью `builtin_op` можно определить отдельные встроенные функции для каждой поддерживаемой арифметической операции.

```
lval* builtin_add(lenv* e, lval* a) {
  return builtin_op(e, a, "+");
}

lval* builtin_sub(lenv* e, lval* a) {
  return builtin_op(e, a, "-");
}

lval* builtin_mul(lenv* e, lval* a) {
  return builtin_op(e, a, "*");
}

lval* builtin_div(lenv* e, lval* a) {
  return builtin_op(e, a, "/");
}
```

Приведя встроенные функции к нужному типу, создадим функцию для их регистрации в окружении.

Для каждой встроенной функции создаём значение-функцию `lval` и значение-символ `lval` с заданным именем. Затем регистрируем их в окружении с помощью `lenv_put`. Окружение всегда принимает и возвращает копии, поэтому после регистрации оба временных значения `lval` нужно удалить.

Разделив задачу на две функции, мы сможем аккуратно зарегистрировать все встроенные функции в окружении.

```
void lenv_add_builtin(lenv* e, char* name, lbuiltin func) {
  lval* k = lval_sym(name);
  lval* v = lval_fun(func);
  lenv_put(e, k, v);
  lval_del(k); lval_del(v);
}

void lenv_add_builtins(lenv* e) {
  /* List Functions */
  lenv_add_builtin(e, "list", builtin_list);
  lenv_add_builtin(e, "head", builtin_head);
  lenv_add_builtin(e, "tail", builtin_tail);
  lenv_add_builtin(e, "eval", builtin_eval);
  lenv_add_builtin(e, "join", builtin_join);

  /* Mathematical Functions */
  lenv_add_builtin(e, "+", builtin_add);
  lenv_add_builtin(e, "-", builtin_sub);
  lenv_add_builtin(e, "*", builtin_mul);
  lenv_add_builtin(e, "/", builtin_div);
}
```

Последний шаг — вызвать эту функцию до запуска интерактивной оболочки. Не забудьте удалить окружение после окончания работы.

```
lenv* e = lenv_new();
lenv_add_builtins(e);

while (1) {

  char* input = readline("lispy> ");
  add_history(input);

  mpc_result_t r;
  if (mpc_parse("<stdin>", input, Lispy, &r)) {

    lval* x = lval_eval(e, lval_read(r.output));
    lval_println(x);
    lval_del(x);

    mpc_ast_delete(r.output);
  } else {
    mpc_err_print(r.error);
    mpc_err_delete(r.error);
  }

  free(input);

}

lenv_del(e);
```

Если всё работает правильно, поэкспериментируйте в оболочке и убедитесь, что функции теперь действительно являются новым типом значений, а не символами.

```
lispy> +
<function>
lispy> eval (head {5 10 11 15})
5
lispy> eval (head {+ - + - * /})
<function>
lispy> (eval (head {+ - + - * /})) 10 20
30
lispy> hello
Error: unbound symbol!
lispy>
```

Функция определения

Мы зарегистрировали встроенные функции как переменные, но пользователь всё ещё не может определять собственные переменные.

Здесь есть небольшая трудность. Пользователь должен передать символ, которому нужно дать значение, и само значение. Но символ не может появиться отдельно: функция вычисления попытается найти соответствующее значение в окружении.

Единственный способ передать символ без вычисления — заключить его между `{}` в цитируемое выражение. Именно так поступит наша функция определения. Она примет список символов и несколько других значений, а затем присвоит каждому символу соответствующее значение.

Эта функция должна работать как другие встроенные функции: сначала проверять условия ошибок, затем выполнять действие и возвращать значение. В нашем случае она сначала проверит типы аргументов, затем переберёт символы и значения и поместит их в окружение. При ошибке мы вернём её, а при успехе — пустое выражение `()`.

```
lval* builtin_def(lenv* e, lval* a) {
  LASSERT(a, a->cell[0]->type == LVAL_QEXPR,
    "Function 'def' passed incorrect type!");

  /* First argument is symbol list */
  lval* syms = a->cell[0];

  /* Ensure all elements of first list are symbols */
  for (int i = 0; i < syms->count; i++) {
    LASSERT(a, syms->cell[i]->type == LVAL_SYM,
      "Function 'def' cannot define non-symbol");
  }

  /* Check correct number of symbols and values */
  LASSERT(a, syms->count == a->count-1,
    "Function 'def' cannot define incorrect "
    "number of values to symbols");

  /* Assign copies of values to symbols */
  for (int i = 0; i < syms->count; i++) {
    lenv_put(e, syms->cell[i], a->cell[i+1]);
  }

  lval_del(a);
  return lval_sexpr();
}
```

Нужно зарегистрировать новую встроенную функцию с помощью `lenv_add_builtins`.

```
/* Variable Functions */
lenv_add_builtin(e, "def",  builtin_def);
```

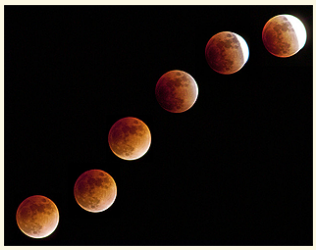
Теперь пользовательские переменные должны работать. Поскольку функция `def` принимает список символов, можно хранить и преобразовывать символы в списках до их определения. Поэкспериментируйте в оболочке и проверьте работу. Результат должен выглядеть следующим образом. Исследуйте другие сложные способы определения и вычисления переменных. Когда мы перейдём к определению функций, полезные стороны этого подхода станут особенно заметны.

```
lispy> def {x} 100
()
lispy> def {y} 200
()
lispy> x
100
lispy> y
200
lispy> + x y
300
lispy> def {a b} 5 6
()
lispy> + a b
11
lispy> def {arglist} {a b x y}
()
lispy> arglist
```

```
{a b x y}
lispy> def arglist 1 2 3 4
()
lispy> list a b x y
{1 2 3 4}
lispy>
```

Сообщения об ошибках

Пока наши сообщения об ошибках не очень хороши. Мы сообщаем о возникновении ошибки и приблизительно описываем проблему, но не даём точных сведений о том, что произошло. Например, при отсутствии привязки символа полезно указать, какого именно. Это поможет находить ошибки, опечатки и другие простые проблемы.



Затмения • Похожи на многоточия.

Было бы здорово создать функцию сообщения об ошибках, работающую наподобие `printf`. Идеально, если бы ей можно было передавать строки, целые числа и другие данные для более подробных сообщений.

Функция `printf` в C особенная: она принимает переменное число аргументов. Мы можем создавать собственные функции с *переменным числом аргументов* и воспользуемся этим для улучшения сообщений об ошибках.

Изменим `lval_err`, чтобы она, как `printf`, принимала строку формата, а затем переменное число аргументов для подстановки.

Чтобы указать в сигнатуре переменное число аргументов, используется специальный синтаксис — многоточие `...`, обозначающее остальные аргументы.

```
lval* lval_err(char* fmt, ...);
```

Внутри функции можно использовать стандартные библиотечные средства, чтобы узнать, что передал вызывающий код.

Сначала создаём структуру `va_list` и инициализируем её с помощью `va_start`, передав последний именованный аргумент. В других случаях отдельные аргументы можно получать через `va_arg`, но мы передадим весь список переменных аргументов непосредственно в `vsnprintf`. Она похожа на `printf`, но записывает результат в строку и принимает `va_list`. Закончив работу с аргументами, вызовем `va_end` для освобождения ресурсов.

Функция `vsnprintf` записывает результат в строку, память для которой нужно выделить заранее. Поскольку до выполнения функции мы не знаем её размер, сначала выделим буфер на `512` символов, а после записи уменьшим его. Сообщение длиннее 512 символов будет обрезано, но надеемся, что такого не случится.

Объединив всё это, получаем новую функцию ошибки.

```
lval* lval_err(char* fmt, ...) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_ERR;

  /* Create a va list and initialize it */
  va_list va;
  va_start(va, fmt);

  /* Allocate 512 bytes of space */
  v->err = malloc(512);

  /* printf the error string with a maximum of 511 characters */
  vsnprintf(v->err, 511, fmt, va);

  /* Reallocate to number of bytes actually used */
  v->err = realloc(v->err, strlen(v->err)+1);

  /* Cleanup our va list */
  va_end(va);

  return v;
}
```

Теперь можно улучшать сообщения в других функциях. Например, в `lenv_get` при отсутствии символа мы можем сообщить его имя вместо общего сообщения об ошибке.

```
return lval_err("Unbound Symbol '%s'", k->sym);
```

Также можно изменить макрос `LASSERT`, чтобы он принимал переменное число аргументов. Поскольку это макрос, а не обычная функция, синтаксис немного отличается. В левой части определения снова используем многоточие, а в правой — специальную переменную `__VA_ARGS__`, которая подставляет остальные аргументы.

Перед этой переменной нужно поставить два знака решётки `##`. Они обеспечивают правильную подстановку, если дополнительных аргументов нет: по сути, удаляют предшествующую запятую `,`, как будто дополнительные аргументы не передавались.

Поскольку `args` может использоваться при построении сообщения, нельзя удалять его до создания значения ошибки.

```
#define LASSERT(args, cond, fmt, ...) \
  if (!(cond)) { \
    lval* err = lval_err(fmt, ##__VA_ARGS__); \
    lval_del(args); \
    return err; \
  }
```

Теперь можно сделать некоторые сообщения информативнее. Например, при неверном числе аргументов указать, сколько требовалось и сколько получено.

```
LASSERT(a, a->count == 1,
  "Function 'head' passed too many arguments. "
  "Got %i, Expected %i.",
  a->count, 1);
```

Также улучшим сообщения об ошибках типов: укажем, какой тип ожидала функция и какой она получила. Для этого полезно сначала создать функцию, принимающую элемент перечисления типов и возвращающую его строковое представление.

```
char* ltype_name(int t) {
  switch(t) {
    case LVAL_FUN: return "Function";
    case LVAL_NUM: return "Number";
    case LVAL_ERR: return "Error";
    case LVAL_SYM: return "Symbol";
    case LVAL_SEXPR: return "S-Expression";
    case LVAL_QEXPR: return "Q-Expression";
    default: return "Unknown";
  }
}
```

```
LASSERT(a, a->cell[0]->type == LVAL_QEXPR,
  "Function 'head' passed incorrect type for argument 0. "
  "Got %s, Expected %s.",
  ltype_name(a->cell[0]->type), ltype_name(LVAL_QEXPR));
```

Используйте `LASSERT`, чтобы подробнее сообщать об ошибках во всём коде. Это значительно упростит отладку следующих этапов, когда мы начнём писать сложный код на нашем языке. Попробуйте использовать макросы, чтобы сократить объём ручного ввода и автоматически создавать типовые сообщения об ошибках.

```
lispy> + 1 {5 6 7}
Error: Function '+' passed incorrect type for argument 1. Got Q-Expression, Expected Number.
lispy> head {1 2 3} {4 5 6}
Error: Function 'head' passed incorrect number of arguments. Got 2, Expected 1.
lispy>
```

Справочник

Исходный код: variables.c.

Дополнительные задания

- › Создайте макрос специально для сообщений об ошибках типов.
- › Создайте макрос специально для сообщений о неверном числе аргументов.
- › Создайте макрос специально для сообщений об ошибках пустого списка.
- › Измените печать встроенной функции так, чтобы выводилось её имя.
- › Напишите функцию печати всех именованных значений окружения.
- › Переопределите одну из встроенных переменных, присвоив ей другое значение.
- › Сделайте переопределение встроенной переменной ошибкой.
- › Создайте функцию `exit`, завершающую работу оболочки.

Функции • Глава 12

Что такое функция?

Функции — сама суть программирования. На заре информатики они воплощали наивную мечту: разбивать вычисления на всё более мелкие фрагменты кода, пригодные для повторного использования. Предполагалось, что при достаточном времени и правильной организации библиотек мы в конце концов напишем код для любых вычислительных нужд. Людям больше не придётся писать собственные функции, а программирование превратится в простое соединение готовых компонентов.

Эта мечта пока не сбылась, но продолжает жить, несмотря на все её недостатки. Каждый новый приём или парадигма программирования немного меняет эту идею. Они обещают более эффективное повторное использование кода, лучшие абстракции и более простую жизнь для всех.

На деле каждая парадигма предоставляет лишь *другие* абстракции. Всегда приходится идти на компромисс: с каждым новым уровнем мышления о программировании какая-то часть отбрасывается. И как бы тщательно мы ни выбирали, что сохранить, а от чего отказаться, иногда кому-то понадобится именно утраченное. Но несмотря на всё это, функции так или иначе сохранялись и неизменно доказывали свою эффективность.



Бананофон • Ещё одна наивная мечта.

Мы использовали функции в C и знаем, *как они выглядят*, но не вполне понимаем, *что они собой представляют*. Вот несколько способов об этом подумать.

Можно рассматривать функцию как описание вычисления, которое нужно выполнить позднее. Определяя функцию, мы словно говорим: «Когда я использую *это* имя, я хочу, чтобы произошло *вот это*». Такой практический взгляд интуитивен и напоминает обычный язык: так мы отдаём команду человеку или животному. Мне также нравится, что он отражает отложенную природу функций: они определяются один раз, но могут вызываться многократно.

Другой способ — представить функцию чёрным ящиком, который принимает входные данные и выдаёт результат. Этот взгляд немного отличается от предыдущего. Он более алгебраический и не говорит о *вычислениях* или *командах*. Это математическое понятие, не привязанное к конкретной машине или языку. Иногда такой взгляд исключительно полезен: мы можем рассуждать о функциях, не заботясь об их устройстве и точном способе вычисления. Затем функции можно объединять и составлять из них композиции, не опасаясь незаметных ошибок. Это основная идея абстракции: именно она позволяет слоям сложности взаимодействовать, а не конфликтовать. Однако её сила может стать и слабостью. Поскольку модель ничего не говорит о вычислениях, она не затрагивает вопросы реального

мира: *«Сколько времени займёт работа функции?»*, *«Эффективна ли она?»*, *«Изменит ли она состояние программы? Если да, то как?»*



Чёрный ящик • Типичная функция.

Третий способ — воспринимать функции как *частичные вычисления*. Как и в математической модели, они могут принимать входные данные. Эти значения нужны, прежде чем функция сможет завершить вычисление, поэтому оно и называется *частичным*. Но, как и в вычислительной модели, тело функции состоит из вычисления, записанного на некотором языке команд. Входные данные называются *свободными переменными*; чтобы завершить вычисление, достаточно их предоставить. Это похоже на шестерёнку, вставленную в механизм, который прежде вращался вхолостую: теперь у него есть всё необходимое, и он начинает работать. Результат таких *частичных вычислений* сам является переменной с неизвестным значением. Его можно передать на вход новой функции, так что одна функция зависит от другой.

Преимущество этой идеи перед математической моделью в том, что мы признаём: функции *содержат вычисление*. При его выполнении в машине происходит физический процесс. Значит, мы учитываем, что некоторые действия занимают время, а функция может изменить состояние программы или совершить что-то ещё, чего мы заранее не знаем.

Все эти идеи исследуются в *лямбда-исчислении* — области, объединяющей логику, математику и информатику. Название происходит от греческой буквы лямбда, используемой при обозначении *связывания переменных*. Лямбда-исчисление позволяет определять, комбинировать и строить *функции* с помощью простой математической записи.

Мы используем все эти идеи, чтобы добавить в язык пользовательские функции. Лисп уже хорошо приспособлен к экспериментам с подобными понятиями, поэтому их реализация потребует не так уж много работы.

Первым шагом станет встроенная функция для создания пользовательских функций. Вот один из вариантов её устройства. Первым аргументом может быть список символов, как в нашей функции `def`. Эти символы называются *формальными аргументами*, или *свободными переменными*. Они служат входами нашего *частичного вычисления*. Вторым аргументом может стать другой список; при вызове функции он будет вычисляться встроенной функцией `eval`.

Назовём эту функцию просто `\` в честь лямбда-исчисления: символ `\` немного похож на лямбду. Чтобы создать функцию, складывающую два аргумента, можно написать следующее.

```
\ {x y} {+ x y}
```

Вызвать функцию можно, поставив её на первое место в обычном S-выражении.

```
(\ {x y} {+ x y}) 10 20
```

Если мы хотим дать функции имя, можно передать её существующей встроенной функции `def`, как любое другое значение, и сохранить в окружении.

```
def {add-together} (\ {x y} {+ x y})
```

После этого её можно вызвать по имени.

```
add-together 10 20
```

Тип функции

Чтобы хранить функцию как `lval`, нужно точно определить, из чего она состоит.

Согласно приведённому определению функция состоит из трёх частей. Первая — список *формальных аргументов*, которые нужно связать перед вычислением. Вторая — Q-выражение, представляющее тело функции. Наконец, необходимо место для хранения значений, присвоенных *формальным аргументам*. К счастью, структура для хранения переменных у нас уже есть — это *окружение*.

Встроенные и пользовательские функции будут храниться под одним типом `LVAL_FUN`. Значит, внутри программы нужен способ различать их. Для этого проверим указатель `lbuiltin`: если он не равен `NULL`, перед нами встроенная функция, иначе — пользовательская.

```
struct lval {
  int type;

  /* Basic */
  long num;
  char* err;
  char* sym;

  /* Function */
  lbuiltin builtin;
  lenv* env;
  lval* formals;
  lval* body;

  /* Expression */
  int count;
  lval** cell;
};
```

Мы переименовали поле типа `lbuiltin` из `fun` в `builtin`. Не забудьте изменить имя во всех местах, где оно используется.

Также нужен конструктор пользовательских функций `lval`. Он создаёт новое окружение функции и присваивает полям `formals` и `body` переданные значения.

```
lval* lval_lambda(lval* formals, lval* body) {
  lval* v = malloc(sizeof(lval));
```

```
  v->type = LVAL_FUN;

  /* Set Builtin to Null */
  v->builtin = NULL;

  /* Build new environment */
  v->env = lenv_new();

  /* Set Formals and Body */
  v->formals = formals;
  v->body = body;
  return v;
}
```

Как всегда при изменении типа `lval`, нужно обновить функции *удаления*, *копирования* и *печати*. Вычисление мы рассмотрим подробнее.

Для **удаления**…

```
case LVAL_FUN:
  if (!v->builtin) {
    lenv_del(v->env);
    lval_del(v->formals);
    lval_del(v->body);
  }
break;
```

Для **копирования**…

```
case LVAL_FUN:
  if (v->builtin) {
    x->builtin = v->builtin;
  } else {
    x->builtin = NULL;
    x->env = lenv_copy(v->env);
    x->formals = lval_copy(v->formals);
    x->body = lval_copy(v->body);
  }
break;
```

Для **печати**…

```
case LVAL_FUN:
  if (v->builtin) {
    printf("<builtin>");
  } else {
    printf("(\\ "); lval_print(v->formals);
    putchar(' '); lval_print(v->body); putchar(')');
  }
break;
```

Лямбда-функция

Теперь добавим встроенную функцию создания лямбды. Она должна принимать список символов и список, представляющий код, а возвращать значение-функцию `lval`. Мы уже определили несколько встроенных функций, и эта будет устроена так же. Как в `def`, сначала

проверим число и типы аргументов с помощью новых макросов. Затем извлечём два первых аргумента из списка и передадим ранее созданной функции `lval_lambda`.

```
lval* builtin_lambda(lenv* e, lval* a) {
  /* Check Two arguments, each of which are Q-Expressions */
  LASSERT_NUM("\\", a, 2);
  LASSERT_TYPE("\\", a, 0, LVAL_QEXPR);
  LASSERT_TYPE("\\", a, 1, LVAL_QEXPR);

  /* Check first Q-Expression contains only Symbols */
  for (int i = 0; i < a->cell[0]->count; i++) {
    LASSERT(a, (a->cell[0]->cell[i]->type == LVAL_SYM),
      "Cannot define non-symbol. Got %s, Expected %s.",
      ltype_name(a->cell[0]->cell[i]->type),ltype_name(LVAL_SYM));
  }

  /* Pop first two arguments and pass them to lval_lambda */
  lval* formals = lval_pop(a, 0);
  lval* body = lval_pop(a, 0);
  lval_del(a);

  return lval_lambda(formals, body);
}
```

Откуда взялись `LASSERT_NUM` и `LASSERT_TYPE`?

Я позволил себе улучшить макросы сообщений об ошибках для этой главы. Это задание предлагалось в дополнительных упражнениях предыдущей главы. Оно делает код настолько чище, что устоять было трудно!

Если вы собирались выполнить это задание самостоятельно, сейчас подходящий момент. Иначе можете посмотреть пример кода этой главы, узнать, какой подход выбрал я, и включить его в свою программу.

Зарегистрируем функцию вместе с другими встроенными функциями.

```
lenv_add_builtin(e, "\\", builtin_lambda);
```

Родительское окружение

Мы дали функциям собственное окружение. В нём будут храниться значения, присвоенные их формальным аргументам. Вычисляя тело функции в этом окружении, мы будем знать, что переменные имеют правильные значения.



Игровая группа • Типичное родительское окружение.

Но в идеале функциям также нужен доступ к переменным глобального окружения, например к встроенным функциям.

Эту проблему можно решить, добавив в определение окружения ссылку на некоторое *родительское* окружение. Тогда при вычислении функции мы назначим её *родителем* глобальное окружение, где определены все встроенные функции.

При добавлении в структуру `lenv` это концептуально будет именно *ссылка* на родительское окружение, а не вложенное окружение. Поэтому её не следует *удалять* при удалении `lenv` или копировать само окружение при копировании `lenv`.

Родительское окружение работает просто. Если `lenv_get` не находит символ в текущем окружении, оно ищет значение по имени в родительском, повторяя процесс, пока переменная не найдена или родители не закончились. Отсутствие родителя обозначается ссылкой `NULL`.

В конструкторе для этого достаточно небольших изменений.

```
struct lenv {
  lenv* par;
  int count;
  char** syms;
  lval** vals;
};

lenv* lenv_new(void) {
  lenv* e = malloc(sizeof(lenv));
  e->par = NULL;
  e->count = 0;
  e->syms = NULL;
  e->vals = NULL;
  return e;
}
```

При получении значения из окружения нужно добавить поиск в родительском окружении, если символ не найден.

```
lval* lenv_get(lenv* e, lval* k) {

  for (int i = 0; i < e->count; i++) {
    if (strcmp(e->syms[i], k->sym) == 0) {
      return lval_copy(e->vals[i]);
    }
  }

  /* If no symbol check in parent otherwise error */
  if (e->par) {
    return lenv_get(e->par, k);
  } else {
    return lval_err("Unbound Symbol '%s'", k->sym);
  }
}
```

Поскольку новый тип `lval` содержит собственное окружение, нам нужна функция копирования окружений при копировании структур `lval`.

```
lenv* lenv_copy(lenv* e) {
  lenv* n = malloc(sizeof(lenv));
  n->par = e->par;
  n->count = e->count;
  n->syms = malloc(sizeof(char*) * n->count);
  n->vals = malloc(sizeof(lval*) * n->count);
  for (int i = 0; i < e->count; i++) {
    n->syms[i] = malloc(strlen(e->syms[i]) + 1);
    strcpy(n->syms[i], e->syms[i]);
    n->vals[i] = lval_copy(e->vals[i]);
  }
  return n;
}
```

Наличие родительских окружений также меняет наше представление об *определении* переменной.

Теперь переменную можно определить двумя способами: в локальном, самом внутреннем окружении или в глобальном, самом внешнем. Добавим функции для обоих случаев. Метод `lenv_put` оставим без изменений: он определяет переменную локально. А для глобального определения добавим `lenv_def`: она просто проходит по цепочке родителей до конца и вызывает `lenv_put` в найденном окружении.

```
void lenv_def(lenv* e, lval* k, lval* v) {
  /* Iterate till e has no parent */
  while (e->par) { e = e->par; }
  /* Put value in e */
  lenv_put(e, k, v);
}
```

Сейчас это различие может показаться бесполезным, но позже мы будем сохранять промежуточные результаты вычислений в локальных переменных функции. Добавим встроенную функцию *локального* присваивания. В C назовём её `put`, а в Лиспе дадим ей символ `=`. Можно изменить `builtin_def` и повторно использовать общий код, как мы делали с математическими операторами.

Затем зарегистрируем эти встроенные функции.

```
lenv_add_builtin(e, "def", builtin_def);
lenv_add_builtin(e, "=",   builtin_put);
```

```
lval* builtin_def(lenv* e, lval* a) {
  return builtin_var(e, a, "def");
}
```

```
lval* builtin_put(lenv* e, lval* a) {
  return builtin_var(e, a, "=");
}
```

```
lval* builtin_var(lenv* e, lval* a, char* func) {
  LASSERT_TYPE(func, a, 0, LVAL_QEXPR);

  lval* syms = a->cell[0];
  for (int i = 0; i < syms->count; i++) {
    LASSERT(a, (syms->cell[i]->type == LVAL_SYM),
```

```
      "Function '%s' cannot define non-symbol. "
      "Got %s, Expected %s.", func,
      ltype_name(syms->cell[i]->type),
      ltype_name(LVAL_SYM));
  }

  LASSERT(a, (syms->count == a->count-1),
    "Function '%s' passed too many arguments for symbols. "
    "Got %i, Expected %i.", func, syms->count, a->count-1);

  for (int i = 0; i < syms->count; i++) {
    /* If 'def' define in globally. If 'put' define in locally */
    if (strcmp(func, "def") == 0) {
      lenv_def(e, syms->cell[i], a->cell[i+1]);
    }

    if (strcmp(func, "=")   == 0) {
      lenv_put(e, syms->cell[i], a->cell[i+1]);
    }
  }

  lval_del(a);
  return lval_sexpr();
}
```

Вызов функции

Теперь нужно написать код, выполняемый при вычислении выражения и вызове функции `lval`.

Если функция встроенная, можно вызвать её по указателю, как раньше. Для пользовательской функции нужно другое: связать каждый переданный аргумент с соответствующим символом из поля `formals`. Затем вычислить поле `body`, используя `env` как окружение, а окружение вызова — как родительское.

Первая попытка без проверки ошибок могла бы выглядеть так:

```
lval* lval_call(lenv* e, lval* f, lval* a) {

  /* If Builtin then simply call that */
  if (f->builtin) { return f->builtin(e, a); }

  /* Assign each argument to each formal in order */
  for (int i = 0; i < a->count; i++) {
      lenv_put(f->env, f->formals->cell[i], a->cell[i]);
  }

  lval_del(a);

  /* Set the parent environment */
  f->env->par = e;

  /* Evaluate the body */
  return builtin_eval(f->env,
    lval_add(lval_sexpr(), lval_copy(f->body)));
}
```

Однако она работает неправильно, если число переданных аргументов отличается от числа формальных: в таком случае программа завершится с ошибкой.

На самом деле это интересная ситуация, в которой есть несколько вариантов. Мы *могли бы* просто сообщать об ошибке при неверном числе аргументов, но можно сделать кое-что интереснее. Если аргументов слишком мало, свяжем первые формальные аргументы и вернём функцию, оставив остальные несвязанными.

Так получится *частично вычисленная* функция, что соответствует нашему прежнему представлению о функции как о *частичном вычислении*. Если функция принимает два аргумента, а мы передаём только один, можно связать первый аргумент и вернуть новую функцию с заполненным первым формальным аргументом и свободным вторым.

Эта метафора рисует забавную картину работы функций. Представим функцию в начале выражения: она по очереди поглощает входные данные непосредственно справа. Проглотив очередной аргумент, она либо насыщается, если больше аргументов не нужно, вычисляется и заменяет себя новым значением, либо заменяет себя другой, более полной функцией с одной связанной переменной. Процесс повторяется, пока не получится конечное значение программы.

Так можно представить функцию маленьким Пакманом: он не проглатывает все входы сразу, а по очереди ест их справа, растёт и растёт, пока не насытится и не взорвётся, создав нечто новое. На самом деле мы реализуем это иначе, но представить такую картину всё равно забавно.

```
lval* lval_call(lenv* e, lval* f, lval* a) {

  /* If Builtin then simply apply that */
  if (f->builtin) { return f->builtin(e, a); }

  /* Record Argument Counts */
  int given = a->count;
  int total = f->formals->count;

  /* While arguments still remain to be processed */
  while (a->count) {

    /* If we've ran out of formal arguments to bind */
    if (f->formals->count == 0) {
      lval_del(a); return lval_err(
        "Function passed too many arguments. "
        "Got %i, Expected %i.", given, total);
    }

    /* Pop the first symbol from the formals */
    lval* sym = lval_pop(f->formals, 0);

    /* Pop the next argument from the list */
    lval* val = lval_pop(a, 0);

    /* Bind a copy into the function's environment */
    lenv_put(f->env, sym, val);

    /* Delete symbol and value */
    lval_del(sym); lval_del(val);
  }

  /* Argument list is now bound so can be cleaned up */
  lval_del(a);

  /* If all formals have been bound evaluate */
  if (f->formals->count == 0) {
```

```
    /* Set environment parent to evaluation environment */
    f->env->par = e;

    /* Evaluate and return */
    return builtin_eval(
      f->env, lval_add(lval_sexpr(), lval_copy(f->body)));
  } else {
    /* Otherwise return partially evaluated function */
    return lval_copy(f);
  }

}
```

Приведённая выше функция делает именно то, что мы описали, с правильной обработкой ошибок. Сначала она перебирает переданные аргументы и пытается поместить каждый в окружение. Затем проверяет, заполнено ли оно: если да, выполняет вычисление; иначе возвращает копию функции с частью заполненных аргументов.

Изменив функцию вычисления `lval_eval_sexpr`, чтобы она вызывала `lval_call`, мы сможем опробовать новую систему.

```
lval* f = lval_pop(v, 0);
if (f->type != LVAL_FUN) {
  lval* err = lval_err(
    "S-Expression starts with incorrect type. "
    "Got %s, Expected %s.",
    ltype_name(f->type), ltype_name(LVAL_FUN));
  lval_del(f); lval_del(v);
  return err;
}

lval* result = lval_call(e, f, v);
```

Попробуйте определить несколько функций и проверить, как работает частичное вычисление.

```
lispy> def {add-mul} (\ {x y} {+ x (* x y)})
()
lispy> add-mul 10 20
210
lispy> add-mul 10
(\ {y} {+ x (* x y)})
lispy> def {add-mul-ten} (add-mul 10)
()
lispy> add-mul-ten 50
510
lispy>
```

Переменное число аргументов

Некоторые встроенные функции мы определили так, чтобы они принимали переменное число аргументов. Например, `+` и `join` работают с любым их количеством. Нужно найти способ предоставить такую возможность и пользовательским функциям.

К сожалению, изящно сделать это без специального синтаксиса не получится. Поэтому мы встроим в язык особый механизм с символом `&`.

Пользователь сможет записать формальные аргументы как `{x & xs}`. Это значит, что функция принимает один аргумент `x`, а затем ноль или более дополнительных аргументов, объединённых в список `xs`. Это немного похоже на многоточие, которым мы обозначали переменное число аргументов в C.

При связывании формальных аргументов будем искать символ `&`. Найдя его, возьмём следующий формальный аргумент и присвоим ему все оставшиеся переданные значения. Важно преобразовать этот список в Q-выражение. Также нужно проверить, что после `&` действительно следует символ; иначе следует сообщить об ошибке.

Эту особую обработку можно добавить в цикл `while` функции `lval_call` сразу после извлечения первого символа из формальных аргументов.

```
/* Special Case to deal with '&' */
if (strcmp(sym->sym, "&") == 0) {

  /* Ensure '&' is followed by another symbol */
  if (f->formals->count != 1) {
    lval_del(a);
    return lval_err("Function format invalid. "
      "Symbol '&' not followed by single symbol.");
  }

  /* Next formal should be bound to remaining arguments */
  lval* nsym = lval_pop(f->formals, 0);
  lenv_put(f->env, nsym, builtin_list(e, a));
  lval_del(sym); lval_del(nsym);
  break;
}
```

Предположим, пользователь передал только первые именованные аргументы, без дополнительных. Тогда символу после `&` нужно присвоить пустой список. Добавим обработку этого случая сразу после удаления списка аргументов и до проверки того, вычислены ли все формальные аргументы.

```
/* If '&' remains in formal list bind to empty list */
if (f->formals->count > 0 &&
  strcmp(f->formals->cell[0]->sym, "&") == 0) {

  /* Check to ensure that & is not passed invalidly. */
  if (f->formals->count != 2) {
    return lval_err("Function format invalid. "
      "Symbol '&' not followed by single symbol.");
  }

  /* Pop and delete '&' symbol */
  lval_del(lval_pop(f->formals, 0));

  /* Pop next symbol and create empty list */
  lval* sym = lval_pop(f->formals, 0);
  lval* val = lval_qexpr();

  /* Bind to environment and delete */
  lenv_put(f->env, sym, val);
  lval_del(sym); lval_del(val);
}
```

Интересные функции

Function Definition

Лямбды — очевидно простой и мощный способ определять функции. Но синтаксис немного громоздок: слишком много скобок и символов. Вот интересная идея: попробуем написать функцию, которая сама определяет функцию с помощью более простого синтаксиса.

По сути нужна функция, выполняющая два шага сразу: создающая новую функцию и присваивающая ей имя. Хитрость в том, чтобы пользователь передавал имя и формальные аргументы вместе в одном списке, а мы сами разделяли их и использовали при определении. Следующая функция принимает аргументы и тело. Первый элемент списка аргументов она использует как имя функции, остальные — как формальные аргументы. Тело передаётся лямбде напрямую.

```
\ {args body} {def (head args) (\ (tail args) body)}
```

Назвать эту функцию, например, `fun` можно, как обычно передав её в `def`.

```
def {fun} (\ {args body} {def (head args) (\ (tail args) body)})
```

Теперь функции можно определять намного проще и красивее. Например, упомянутую ранее `add-together` можно определить так. Функции, определяющие функции! Такого мы никогда не смогли бы сделать в C. Здорово, правда?

```
fun {add-together x y} {+ x y}
```

Currying



Каррирование • Звучит вкуснее, чем есть на самом деле.

Сейчас такие функции, как `+`, принимают переменное число аргументов. Иногда это прекрасно, но что, если аргументы находятся в списке? В таком случае функция становится почти бесполезной.

Попробуем снова решить проблему собственной функцией. Если создать список в нужном для выражения формате, можно воспользоваться `eval`, чтобы вычислить его как выражение. В случае `+` можно поместить функцию в начало списка и затем выполнить вычисление.

Определим функцию `unpack`, которая делает именно это: принимает функцию и список, ставит функцию в начало списка и вычисляет результат.

```
fun {unpack f xs} {eval (join (list f) xs)}
```

Иногда возникает противоположная задача: функция принимает список, а мы хотим вызвать её с переменным числом аргументов. Решение ещё проще: воспользуемся тем, что синтаксис & сам собирает переменные аргументы в список.

```
fun {pack f & xs} {f xs}
```

В некоторых языках это называют соответственно *каррированием* и *обратным каррированием*. Название происходит от имени *Хаскелла Карри* и, к сожалению, никак не связано с любимым острым блюдом.

```
lispy> def {uncurry} pack
()
lispy> def {curry} unpack
()
lispy> curry + {5 6 7}
18
lispy> uncurry head 5 6 7
{5}
```

Благодаря тому, как работает наше *частичное вычисление*, нам не нужно думать о *каррировании* для конкретного набора аргументов. Можно считать, что сами функции представлены в *каррированной* или *некаррированной* форме.

```
lispy> def {add-uncurried} +
()
lispy> def {add-curried} (curry +)
()
lispy> add-curried {5 6 7}
18
lispy> add-uncurried 5 6 7
18
```

Поэкспериментируйте и попробуйте придумать другие интересные и мощные функции. В следующей главе мы добавим условные выражения, которые сделают наш язык ещё полноценнее. Но это не мешает вам уже сейчас придумать что-нибудь интересное. Наш Лисп становится богаче.

Справочник

Исходный код: functions.c.

Дополнительные задания

- › Определите функцию Лиспа, возвращающую первый элемент списка.
- › Определите функцию Лиспа, возвращающую второй элемент списка.
- › Определите функцию Лиспа, вызывающую другую функцию с двумя аргументами в обратном порядке.
- › Определите функцию Лиспа, вызывающую функцию с аргументами и передающую результат другой функции.
- › Определите функцию C `builtin_fun`, эквивалентную функции Лиспа `fun`.
- › Измените работу с переменным числом аргументов так, чтобы до вычисления требовался хотя бы один дополнительный аргумент.

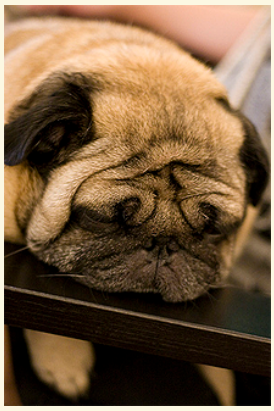
Условные выражения • Глава 13

Попробуйте самостоятельно

Мы уже прошли долгий путь. Ваших знаний C должно хватать, чтобы действовать немного самостоятельнее. Если вы уверены в себе, эта глава — отличная возможность расправить крылья и попробовать сделать что-то самому. Она довольно короткая и по сути посвящена добавлению нескольких встроенных функций сравнения и определения порядка.

Если вы готовы, попробуйте прямо сейчас реализовать в языке сравнение и определение порядка. Создайте встроенные функции для операторов *больше*, *меньше*, *равно* и остальных операторов сравнения, используемых в C. Попробуйте написать функцию `if`, которая проверяет условие и в зависимости от результата вычисляет один или другой фрагмент кода. Закончив, вернитесь и сравните свою работу с моей. Обратите внимание на различия и решите, какой вариант вам больше нравится.

Если пока не чувствуете уверенности, не беспокойтесь. Следуйте за мной, и я объясню свой подход.



Мопс • **Если** мопс спит, **то** он милый.

Сравнение порядка

Для простоты я снова использую наш числовой тип для результатов сравнения. Установим правило, похожее на правило C: любое число, кроме `0`, считается истинным в выражении `if`, а `0` всегда считается ложным.

Поэтому функции сравнения порядка будут похожи на упрощённую версию арифметических функций. Они работают только с числами, причём ровно с двумя аргументами.

Если проверки ошибок пройдены, вычисления просты. Нужно вернуть числовое значение `lval` — `0` или `1` — в зависимости от результата сравнения двух входных значений `lval`. Для этого можно использовать операторы сравнения C. Как и в арифметических функциях, воспользуемся одной общей функцией для всех сравнений.

Сначала проверим условия ошибок, затем сравним числа в аргументах и получим результат. Наконец вернём его как числовое значение.

```
lval* builtin_gt(lenv* e, lval* a) {
  return builtin_ord(e, a, ">");
}
```

```
lval* builtin_lt(lenv* e, lval* a) {
  return builtin_ord(e, a, "<");
}
```

```
lval* builtin_ge(lenv* e, lval* a) {
  return builtin_ord(e, a, ">=");
}
```

```
lval* builtin_le(lenv* e, lval* a) {
  return builtin_ord(e, a, "<=");
}
```

```
lval* builtin_ord(lenv* e, lval* a, char* op) {
  LASSERT_NUM(op, a, 2);
  LASSERT_TYPE(op, a, 0, LVAL_NUM);
  LASSERT_TYPE(op, a, 1, LVAL_NUM);

  int r;
  if (strcmp(op, ">")  == 0) {
    r = (a->cell[0]->num >  a->cell[1]->num);
  }
  if (strcmp(op, "<")  == 0) {
    r = (a->cell[0]->num <  a->cell[1]->num);
  }
  if (strcmp(op, ">=") == 0) {
    r = (a->cell[0]->num >= a->cell[1]->num);
  }
  if (strcmp(op, "<=") == 0) {
    r = (a->cell[0]->num <= a->cell[1]->num);
  }
  lval_del(a);
  return lval_num(r);
}
```

Равенство

Проверка равенства отличается от сравнения порядка: она должна работать не только с числами. Полезно проверять, равен ли аргумент пустому списку или совпадают ли две переданные функции. Поэтому нам нужна функция, проверяющая равенство двух значений `lval` различных типов.

По сути, эта функция проверяет равенство всех полей данных соответствующего типа `lval`. Если равны все поля, значения считаются равными; если есть хотя бы одно различие — неравными.

```
int lval_eq(lval* x, lval* y) {

  /* Different Types are always unequal */
  if (x->type != y->type) { return 0; }

  /* Compare Based upon type */
  switch (x->type) {
    /* Compare Number Value */
    case LVAL_NUM: return (x->num == y->num);

    /* Compare String Values */
    case LVAL_ERR: return (strcmp(x->err, y->err) == 0);
```

```
    case LVAL_SYM: return (strcmp(x->sym, y->sym) == 0);

    /* If builtin compare, otherwise compare formals and body */
    case LVAL_FUN:
      if (x->builtin || y->builtin) {
        return x->builtin == y->builtin;
      } else {
        return lval_eq(x->formals, y->formals)
          && lval_eq(x->body, y->body);
      }

    /* If list compare every individual element */
    case LVAL_QEXPR:
    case LVAL_SEXPR:
      if (x->count != y->count) { return 0; }
      for (int i = 0; i < x->count; i++) {
        /* If any element not equal then whole list not equal */
        if (!lval_eq(x->cell[i], y->cell[i])) { return 0; }
      }
      /* Otherwise lists must be equal */
      return 1;
    break;
  }
  return 0;
}
```

С этой функцией встроенную операцию сравнения на равенство добавить очень просто. Мы проверяем, что переданы два аргумента, и сравниваем их. Результат сохраняем в новом `lval` и возвращаем.

```
lval* builtin_cmp(lenv* e, lval* a, char* op) {
  LASSERT_NUM(op, a, 2);
  int r;
  if (strcmp(op, "==") == 0) {
    r =  lval_eq(a->cell[0], a->cell[1]);
  }
  if (strcmp(op, "!=") == 0) {
    r = !lval_eq(a->cell[0], a->cell[1]);
  }
  lval_del(a);
  return lval_num(r);
}

lval* builtin_eq(lenv* e, lval* a) {
  return builtin_cmp(e, a, "==");
}

lval* builtin_ne(lenv* e, lval* a) {
  return builtin_cmp(e, a, "!=");
}
```

Функция if

Чтобы операторы сравнения приносили пользу, нам понадобится функция `if`. Она немного похожа на тернарный оператор C: если условие истинно, вычисляется одно выражение, если ложно — другое.

Мы снова можем воспользоваться Q-выражениями для записи вычислений. Сначала пользователь передаёт результат сравнения, затем два Q-выражения с кодом, который нужно вычислить при истинном или ложном условии.

```
lval* builtin_if(lenv* e, lval* a) {
  LASSERT_NUM("if", a, 3);
  LASSERT_TYPE("if", a, 0, LVAL_NUM);
  LASSERT_TYPE("if", a, 1, LVAL_QEXPR);
  LASSERT_TYPE("if", a, 2, LVAL_QEXPR);

  /* Mark Both Expressions as evaluable */
  lval* x;
  a->cell[1]->type = LVAL_SEXPR;
  a->cell[2]->type = LVAL_SEXPR;

  if (a->cell[0]->num) {
    /* If condition is true evaluate first expression */
    x = lval_eval(e, lval_pop(a, 1));
  } else {
    /* Otherwise evaluate second expression */
    x = lval_eval(e, lval_pop(a, 2));
  }

  /* Delete argument list and return */
  lval_del(a);
  return x;
}
```

Осталось зарегистрировать все новые встроенные функции, и мы снова готовы к работе.

```
/* Comparison Functions */
lenv_add_builtin(e, "if", builtin_if);
lenv_add_builtin(e, "==", builtin_eq);
lenv_add_builtin(e, "!=", builtin_ne);
lenv_add_builtin(e, ">",  builtin_gt);
lenv_add_builtin(e, "<",  builtin_lt);
lenv_add_builtin(e, ">=", builtin_ge);
lenv_add_builtin(e, "<=", builtin_le);
```

Немного поэкспериментируйте и проверьте, что всё работает правильно.

```
lispy> > 10 5
1
lispy> <= 88 5
0
lispy> == 5 6
0
lispy> == 5 {}
0
lispy> == 1 1
1
lispy> != {} 56
1
lispy> == {1 2 3 {5 6}} {1   2  3   {5 6}}
1
lispy> def {x y} 100 200
()
lispy> if (== x y) {+ x y} {- x y}
-100
```

Рекурсивные функции

Добавив условные выражения, мы сделали язык гораздо мощнее: теперь в нём можно создавать рекурсивные функции.

Рекурсивные функции вызывают сами себя. Мы уже применяли их в C для чтения и вычисления выражений. Условные выражения нужны им, чтобы определить момент, когда рекурсию следует остановить.

Например, с помощью условия можно реализовать функцию `len`, возвращающую число элементов списка. Если список пуст, возвращаем `0`. Иначе возвращаем длину хвоста `tail` входного списка плюс `1`. Подумайте, почему это работает: функция `len` вызывается снова и снова, пока не достигнет пустого списка. Тогда она возвращает `0`, и все остальные промежуточные результаты складываются.

```
(fun {len l} {
  if (== l {})
    {0}
    {+ 1 (len (tail l))}
})
```

Как и в C, в такой рекурсивной функции есть приятная симметрия. Сначала мы выполняем действие для пустого списка (*базовый случай*). Если список больше, отделяем часть — например, его голову — и что-то с ней делаем, а затем объединяем результат с остальной частью, к которой уже применена функция.

А вот ещё одна функция, переворачивающая список. Как и прежде, она проверяет, пуст ли список, но на этот раз возвращает пустой список. Это логично: пустой список, записанный в обратном порядке, остаётся пустым. Если же список больше, функция переворачивает хвост и помещает его перед головой.

```
(fun {reverse l} {
  if (== l {})
    {{}}
    {join (reverse (tail l)) (head l)}
})
```

Мы будем использовать этот приём для создания многих функций, потому что именно он станет основным способом организации циклов в нашем языке.

Справочник

Исходный код: conditionals.c.

Дополнительные задания

- › Создайте встроенные логические операторы *или* `||`, *и* `&&`, *не* `!` и добавьте их в язык.
- › Определите рекурсивную функцию Лиспа, возвращающую `nth`-й элемент списка.
- › Определите рекурсивную функцию Лиспа, возвращающую `1`, если элемент принадлежит списку, и `0` в противном случае.
- › Определите функцию Лиспа, возвращающую последний элемент списка.
- › Определите на Лиспе функции логических операторов `or`, `and` и `not`.
- › Добавьте в язык отдельный логический тип и встроенные переменные `true` и `false`.

Строки • Глава 14

Библиотеки

Наконец наш Лисп работает. Мы можем написать почти любую желаемую функцию, построить довольно сложные конструкции и даже сделать кое-что интересное, недоступное многим популярным тяжеловесным языкам.

Каждый раз, когда мы изменяем программу и запускаем её снова, заново вводить все функции утомительно. В этой главе мы добавим загрузку и выполнение кода из файла. Так мы сможем начать создавать стандартную библиотеку. Заодно добавим поддержку комментариев, строк и вывода.



Верёвка • Какова её длина?

Строковый тип

Чтобы загрузить файл, пользователь должен иметь возможность указать его имя в виде строки. Наш язык поддерживает символы, но пока не поддерживает строки, которые могут содержать пробелы и другие знаки. Нам нужно добавить такой тип `lval` для указания имён файлов.

Как и в предыдущих главах, начнём с добавления элемента в перечисление и поля в `lval` для хранения данных нового типа.

```
enum { LVAL_ERR, LVAL_NUM,   LVAL_SYM, LVAL_STR,
       LVAL_FUN, LVAL_SEXPR, LVAL_QEXPR };
```

```
/* Basic */
long num;
char* err;
char* sym;
char* str;
```

Теперь добавим функцию создания строкового `lval`, очень похожую на функцию создания символов.

```
lval* lval_str(char* s) {
  lval* v = malloc(sizeof(lval));
  v->type = LVAL_STR;
  v->str = malloc(strlen(s) + 1);
  strcpy(v->str, s);
  return v;
}
```

Также нужно дополнить соответствующие функции, работающие с `lval`.

Для **удаления**…

```
case LVAL_STR: free(v->str); break;
```

Для **копирования**…

```
case LVAL_STR: x->str = malloc(strlen(v->str) + 1);
  strcpy(x->str, v->str); break;
```

Для **проверки равенства**…

```
case LVAL_STR: return (strcmp(x->str, y->str) == 0);
```

Для **имени типа**…

```
case LVAL_STR: return "String";
```

Для **вывода** понадобится сделать немного больше. Строка, которую мы храним внутри программы, отличается от строки, которую нужно напечатать. Мы хотим выводить её в том виде, в каком пользователь может её ввести: с управляющими последовательностями, например `\n` для новой строки.

Поэтому перед выводом нужно экранировать строку. К счастью, для этого можно использовать функцию `mpc`.

В функцию печати добавим следующее…

```
case LVAL_STR:   lval_print_str(v); break;
```

Где…

```
void lval_print_str(lval* v) {
  /* Make a Copy of the string */
  char* escaped = malloc(strlen(v->str)+1);
  strcpy(escaped, v->str);
  /* Pass it through the escape function */
  escaped = mpcf_escape(escaped);
  /* Print it between " characters */
  printf("\"%s\"", escaped);
  /* free the copied string */
  free(escaped);
}
```

Чтение строк

Теперь нужно добавить разбор строк. Как обычно, для этого сначала введём новое грамматическое правило `string` и добавим его в синтаксический анализатор.

Мы будем использовать правило для строк в стиле C: строка представляет собой последовательность управляющих последовательностей или обычных символов между двойными кавычками `""`. Задать её регулярным выражением в строке грамматики можно так.

```
string  : /\"(\\\\.|[^\"])*\"/ ;
```

Выглядит сложно, но если разобрать выражение по частям, всё становится понятнее. Строка начинается с символа `"`, за которым следуют ноль или больше повторений: либо обратная косая черта `\\` и любой следующий символ `.`, либо любой символ, *кроме* кавычки `[^\\"]`. В конце стоит ещё один символ `"`.

Также нужно добавить соответствующий случай в функцию `lval_read`.

```
if (strstr(t->tag, "string")) { return lval_read_str(t); }
```

Поскольку входная строка записана с экранированием, нам нужна функция `lval_read_str`, которая это обработает. Она немного сложнее, поскольку выполняет несколько задач. Сначала нужно убрать кавычки `"` с обоих концов. Затем снять экранирование, преобразовав последовательности вроде `\n` в соответствующие символы. Наконец, создать новый `lval` и освободить промежуточные данные.

```
lval* lval_read_str(mpc_ast_t* t) {
  /* Cut off the final quote character */
  t->contents[strlen(t->contents)-1] = '\0';
  /* Copy the string missing out the first quote character */
  char* unescaped = malloc(strlen(t->contents+1)+1);
  strcpy(unescaped, t->contents+1);
  /* Pass through the unescape function */
  unescaped = mpcf_unescape(unescaped);
  /* Construct a new lval using the string */
  lval* str = lval_str(unescaped);
  /* Free the string and return */
  free(unescaped);
  return str;
}
```

Если всё работает, мы можем попробовать строки в интерактивной оболочке. Далее добавим функции, которые действительно смогут их использовать.

```
lispy> "hello"
"hello"
lispy> "hello\n"
"hello\n"
lispy> "hello\""
"hello\""
lispy> head {"hello" "world"}
{"hello"}
lispy> eval (head {"hello" "world"})
"hello"
lispy>
```

Комментарии

Раз уж мы добавляем новый синтаксис, рассмотрим и комментарии.

Как и в C, комментарии позволяют рассказать другим людям или самим себе, что делает код и почему он написан именно так. В C комментарии заключаются между `/*` и `*/`. В Лиспе

комментарий, напротив, начинается с `;` и продолжается до конца строки.

Я пытался выяснить, почему в Лиспе для комментариев используется `;`, но, похоже, происхождение этого правила затерялось в глубине веков. Мне хочется представить это маленьким бунтом против императивных языков вроде C и Java, которые так часто и бесстыдно используют точки с запятой для разделения и завершения инструкций. По сравнению с Лиспом все эти языки — просто комментарии.

Итак, комментарий в Лиспе — это точка с запятой `;`, за которой следует любое число символов, кроме символов перевода строки `\r` и `\n`. Его можно описать ещё одним регулярным выражением.

```
comment : /;[^\\r\\n]*/ ;
```

Как и для строк, нужно создать новый анализатор и обновить описание языка в `mpca_lang`. Не забудьте добавить анализатор в `mpc_cleanup` и изменить первый целочисленный аргумент, указав новое число передаваемых анализаторов.

Окончательная грамматика теперь выглядит так.

```
mpca_lang(MPCA_LANG_DEFAULT,
  "                                          \
    number  : /-?[0-9]+/ ;                   \
    symbol  : /[a-zA-Z0-9_+\\-*\\/\\=<>!&]+/ ; \
    string  : /\"(\\.|[^\"])*\"/ ;           \
    comment : /;[^\\r\\n]*/ ;                \
    sexpr   : '(' <expr>* ')' ;              \
    qexpr   : '{' <expr>* '}' ;              \
    expr    : <number>  | <symbol> | <string>  \
            | <comment> | <sexpr>  | <qexpr>;  \
    lispy   : /^/ <expr>* /$/ ;              \
  ",
  Number, Symbol, String, Comment, Sexpr, Qexpr, Expr, Lispy);
```

А функция очистки выглядит так.

```
mpc_cleanup(8,
  Number, Symbol, String, Comment,
  Sexpr,  Qexpr,  Expr,   Lispy);
```

Поскольку комментарии предназначены только для программистов, читающих код, наша внутренняя функция чтения должна просто их игнорировать. В `lval_read` можно добавить условие, подобное обработке фигурных и круглых скобок.

```
if (strstr(t->children[i]->tag, "comment")) { continue; }
```

В интерактивной оболочке комментарии мало пригодятся, но будут очень полезны для пояснений в файлах с кодом.

Функция load

Нам нужна функция, которая принимает строку с именем файла, загружает его и вычисляет содержимое. Для реализации понадобится наша грамматика: с её помощью мы прочитаем содержимое файла, разберём и вычислим его. Функция загрузки будет использовать наш `mpc_parser*` с именем `Lispy`.

Поэтому, как и в случае функций, нужно предварительно объявить указатели на анализаторы в начале файла.

```
mpc_parser_t* Number;
mpc_parser_t* Symbol;
mpc_parser_t* String;
mpc_parser_t* Comment;
mpc_parser_t* Sexpr;
mpc_parser_t* Qexpr;
mpc_parser_t* Expr;
mpc_parser_t* Lispy;
```

Функция `load` будет такой же, как остальные встроенные функции. Сначала проверим, что единственный аргумент — строка. Затем с помощью `mpc_parse_contents` прочитаем и разберём содержимое файла по грамматике. Подобно `mpc_parse`, эта функция помещает результат в объект `mpc_result`: в нашем случае снова получается *абстрактное синтаксическое дерево* либо *ошибка*.

В отличие от интерактивной оболочки, при успешном разборе файла не следует считать его одним выражением. В файле пользователь может записать несколько выражений, и каждое нужно вычислить отдельно. Для этого переберём выражения файла и вычислим их по одному. Ошибки следует печатать и продолжать выполнение.

Если произошла ошибка разбора, извлечём сообщение, поместим его в ошибочный `lval` и вернём. Если ошибок нет, встроенная функция может вернуть пустое выражение. Полный код выглядит так.

```
lval* builtin_load(lenv* e, lval* a) {
  LASSERT_NUM("load", a, 1);
  LASSERT_TYPE("load", a, 0, LVAL_STR);

  /* Parse File given by string name */
  mpc_result_t r;
  if (mpc_parse_contents(a->cell[0]->str, Lispy, &r)) {

    /* Read contents */
    lval* expr = lval_read(r.output);
    mpc_ast_delete(r.output);

    /* Evaluate each Expression */
    while (expr->count) {
      lval* x = lval_eval(e, lval_pop(expr, 0));
      /* If Evaluation leads to error print it */
      if (x->type == LVAL_ERR) { lval_println(x); }
      lval_del(x);
    }

    /* Delete expressions and arguments */
    lval_del(expr);
    lval_del(a);
```

```
    /* Return empty list */
    return lval_sexpr();

  } else {
    /* Get Parse Error as String */
    char* err_msg = mpc_err_string(r.error);
    mpc_err_delete(r.error);

    /* Create new error message using it */
    lval* err = lval_err("Could not load Library %s", err_msg);
    free(err_msg);
    lval_del(a);

    /* Cleanup and return error */
    return err;
  }
}
```

Аргументы командной строки

Теперь, когда мы умеем загружать файлы, можно добавить возможность, характерную для других языков: выполнять файлы, имена которых переданы в командной строке. Например, файл Python запускают командой `python filename.py`.

Аргументы командной строки доступны через переменные `argc` и `argv`, передаваемые в `main`. Первая содержит количество аргументов, вторая — строки аргументов. Значение `argc` всегда не меньше единицы, поскольку первый аргумент — полная вызванная команда.

Поэтому при `argc`, равном `1`, можно запустить интерактивный интерпретатор; иначе каждый аргумент передаётся функции `builtin_load`.

```
/* Supplied with list of files */
if (argc >= 2) {

  /* loop over each supplied filename (starting from 1) */
  for (int i = 1; i < argc; i++) {

    /* Argument list with a single argument, the filename */
    lval* args = lval_add(lval_sexpr(), lval_str(argv[i]));

    /* Pass to builtin load and get the result */
    lval* x = builtin_load(e, args);

    /* If the result is an error be sure to print it */
    if (x->type == LVAL_ERR) { lval_println(x); }
    lval_del(x);
  }
}
```

Теперь можно написать простую программу и попробовать запустить её таким способом.

```
lispy example.lspy
```

Функция print

При запуске программ из командной строки может понадобиться выводить данные, а не только определять функции и значения. Добавим в Лисп функцию `print`, использующую уже существующую `lval_print`.

Она печатает аргументы через пробел и завершает вывод символом новой строки. Возвращает она пустое выражение.

```
lval* builtin_print(lenv* e, lval* a) {

  /* Print each argument followed by a space */
  for (int i = 0; i < a->count; i++) {
    lval_print(a->cell[i]); putchar(' ');
  }

  /* Print a newline and delete arguments */
  putchar('\n');
  lval_del(a);

  return lval_sexpr();
}
```

Функция error

Строки также позволяют добавить функцию сообщения об ошибках. Она может принимать введённую пользователем строку и передавать её в `lval_err` как сообщение об ошибке.

```
lval* builtin_error(lenv* e, lval* a) {
  LASSERT_NUM("error", a, 1);
  LASSERT_TYPE("error", a, 0, LVAL_STR);

  /* Construct Error from first argument */
  lval* err = lval_err(a->cell[0]->str);

  /* Delete arguments and return */
  lval_del(a);
  return err;
}
```

Последний шаг — зарегистрировать эти функции как встроенные. Теперь наконец можно создавать библиотеки и записывать их в файлы.

```
/* String Functions */
lenv_add_builtin(e, "load",  builtin_load);
lenv_add_builtin(e, "error", builtin_error);
lenv_add_builtin(e, "print", builtin_print);
```

```
lispy> print "Hello World!"
"Hello World!"
()
lispy> error "This is an error"
Error: This is an error
lispy> load "hello.lspy"
"Hello World!"
()
lispy>
```

Завершение

Это последняя глава, в которой мы непосредственно работаем над реализацией Лиспа на C. По её окончании реализация нашего языка будет завершена.

В итоге у вас должно получиться около тысячи строк кода. Написать столько кода — непростое дело. Если вы дошли до этого места, значит, написали настоящую программу и начали серьёзный проект. Полученные навыки пригодятся и в других задачах, придав вам уверенности в достижении собственных целей. Теперь у вас есть сложная и красивая программа, с которой можно взаимодействовать и экспериментировать. Этим стоит гордиться. Покажите её друзьям и семье!

В следующей главе мы начнём создавать на нашем Лиспе стандартную библиотеку распространённых функций. После этого я опишу возможные улучшения и направления развития языка. Хотя моя работа с вами подходит к концу, на самом деле это лишь начало. Спасибо, что прошли этот путь вместе со мной, и удачи во всех будущих программах на C!

Справочник

Исходный код: strings.c.

Дополнительные задания

- › Приспособьте встроенную функцию `join` для работы со строками.
- › Приспособьте встроенную функцию `head` для работы со строками.
- › Приспособьте встроенную функцию `tail` для работы со строками.
- › Создайте встроенную функцию `read`, которая читает строку и преобразует её в Q-выражение.
- › Создайте встроенную функцию `show`, которая печатает содержимое строк как есть, без экранирования.
- › Создайте специальное значение `ok`, возвращаемое вместо пустого выражения `()`.
- › Добавьте функции-обёртки для всех функций C по работе с файлами, таких как `fopen` и `fgets`.

Стандартная библиотека • Глава 15

Минимализм

Мы намеренно создали минималистичный Lisp: добавили лишь самое необходимое число основных структур и встроенных функций. Если выбрать их внимательно, как мы и поступили, они позволят дополнить язык всем остальным необходимым.



Библиотека • Построена всего лишь из кожи, бумаги, дерева и чернил.

У минимализма две причины. Во-первых, небольшое ядро проще отлаживать и изучать, что полезно и разработчикам, и пользователям. Подобно бритве Оккама, почти всегда лучше отсечь лишнее, если выразительность языка при этом не страдает. Во-вторых, маленький язык эстетически привлекательнее. Умно, интересно и весело выяснять, насколько маленьким может быть ядро, чтобы результат оставался полезным. Мы, как хакеры, которыми к этому моменту уже должны стать, получаем от этого удовольствие.

Атомы

При работе с условиями мы не добавляли в язык новый логический тип, а значит, и значения `true` и `false`. Вместо них использовались числа. Но читаемость тоже важна, поэтому определим константы для этих значений.

Аналогично многие диалекты Lisp используют слово `nil` для обозначения пустого списка `{}`. Добавим и его. Такие константы иногда называют *атомами*, поскольку они фундаментальны и неизменны.

Пользователь не обязан применять именованные константы: при желании он может использовать числа и пустые списки. Возможность выбора расширяет свободу пользователя, а это важно для нас.

```
; Atoms
(def {nil} {})
(def {true} 1)
(def {false} 0)
```

Строительные блоки

Мы уже придумали несколько полезных функций, которые я использовал в примерах. Одна из них — `fun`, позволяющая объявлять функции более аккуратно. Её определённо стоит включить в стандартную библиотеку.

```
; Function Definitions
(def {fun} (\ {f b} {
  def (head f) (\ (tail f) b)
}))
```

У нас также были функции `unpack` и `pack`. Они тоже понадобятся пользователям. Включим их вместе с псевдонимами `curry` и `uncurry`.

```
; Unpack List for Function
(fun {unpack f l} {
  eval (join (list f) l)
})

; Pack List for Function
(fun {pack f & xs} {f xs})

; Curried and Uncurried calling
(def {curry} unpack)
(def {uncurry} pack)
```

Предположим, нужно выполнить несколько действий по порядку. Один из способов — передать каждое действие как аргумент функции. Мы знаем, что аргументы вычисляются слева направо, то есть события выполняются последовательно. Для функций вроде `print` и `load` результат вычисления не так важен, как порядок действий.

Поэтому создадим функцию `do`, которая вычисляет несколько выражений по порядку и возвращает последнее. Она использует функцию `last`, возвращающую последний элемент списка. Мы определим её позже.

```
; Perform Several things in Sequence
(fun {do & l} {
  if (== l nil)
    {nil}
    {last l}
})
```

Иногда результаты нужно сохранить в локальных переменных с помощью оператора `=`. Внутри функции они и так сохраняются локально, но порой требуется ещё более узкая область видимости. Для этого создадим `let`: она создаёт пустую функцию, в которой выполняется код, и вычисляет её.

```
; Open new scope
(fun {let b} {
  ((\ {_} b) ())
})
```

Её можно использовать вместе с `do`, чтобы переменные не выходили за пределы своей области видимости.

```
lispy> let {do (= {x} 100) (x)}
100
lispy> x
Error: Unbound Symbol 'x'
lispy>
```

Логические операторы

В языке мы не определяли логические операторы вроде `and` и `or`. Возможно, стоит добавить их позже. Пока же их можно имитировать арифметическими операторами. Подумайте, как эти функции работают с различными сочетаниями входных значений `0` и `1`.

```
; Logical Functions
(fun {not x}   {- 1 x})
(fun {or x y}  {+ x y})
(fun {and x y} {* x y})
```

Разные функции

Вот несколько функций, которые трудно отнести к другим разделам. Попробуйте догадаться, для чего они предназначены.

```
(fun {flip f a b} {f b a})
(fun {ghost & xs} {eval xs})
(fun {comp f g x} {f (g x)})
```

Функция `flip` принимает функцию `f` и два аргумента `a` и `b`, а затем применяет `f` к ним в обратном порядке. Это полезно при *частичном применении* функции: если нужно передать только второй аргумент, `flip` даст новую функцию, которая принимает первые два аргумента в обратном порядке.

Иными словами, чтобы применить второй аргумент исходной функции, можно применить первый аргумент к результату `flip`.

```
lispy> (flip def) 1 {x}
()
lispy> x
1
lispy> def {define-one} ((flip def) 1)
()
lispy> define-one {y}
()
lispy> y
1
lispy>
```

Я не придумал применения для функции `ghost`, но она показалась мне интересной. Она принимает любое число аргументов и вычисляет их так, словно они сами являются выражением. Она стоит в начале выражения как призрак и никак не влияет на поведение программы. Если придумаете ей применение, буду рад узнать.

```
lispy> ghost + 2 2
4
```

Функция `comp` выполняет композицию двух функций. Она принимает `f`, `g` и аргумент для `g`, применяет к нему `g`, а затем применяет `f` к результату. Так можно объединить две функции в одну, которая выполняет их последовательно. Как и раньше, частичное применение позволяет строить сложные функции из простых.

Например, объединим две функции: одна меняет знак числа, а другая распаковывает список чисел для перемножения с помощью `*`.

```
lispy> (unpack *) {2 2}
4
lispy> - ((unpack *) {2 2})
-4
lispy> comp - (unpack *)
(\ {x} {f (g x)})
lispy> def {mul-neg} (comp - (unpack *))
()
lispy> mul-neg {2 8}
-16
lispy>
```

Функции для списков

Функция `head` возвращает первый элемент списка, но он по-прежнему заключён в список. Чтобы получить сам элемент, его нужно извлечь.

Список из одного элемента вычисляется в этот элемент, поэтому для извлечения можно использовать `eval`. Определим также несколько вспомогательных функций для получения первого, второго и третьего элементов списка. Позже они нам пригодятся.

```
; First, Second, or Third Item in List
(fun {fst l} { eval (head l) })
(fun {snd l} { eval (head (tail l)) })
(fun {trd l} { eval (head (tail (tail l))) })
```

Несколько глав назад мы кратко рассмотрели рекурсивные функции для списков. Разумеется, этим способом можно определить множество других.

Чтобы найти длину списка, рекурсивно обойдём его и прибавим `1` к длине хвоста. Чтобы найти элемент `nth`, будем применять `tail` и уменьшать индекс, пока не дойдём до `0`. Последний элемент находится по индексу, равному длине минус один.

```
; List Length
(fun {len l} {
```

```
  if (== l nil)
    {0}
    {+ 1 (len (tail l))}
})
```

```
; Nth item in List
(fun {nth n l} {
  if (== n 0)
    {fst l}
    {nth (- n 1) (tail l)}
})
```

```
; Last item in List
(fun {last l} {nth (- (len l) 1) l})
```

Многие другие полезные функции следуют той же схеме. Можно определить функции, которые берут или отбрасывают заданное число первых элементов списка, а также проверяют, содержится ли значение в списке.

```
; Take N items
(fun {take n l} {
  if (== n 0)
    {nil}
    {join (head l) (take (- n 1) (tail l))}
})
```

```
; Drop N items
(fun {drop n l} {
  if (== n 0)
    {l}
    {drop (- n 1) (tail l)}
})
```

```
; Split at N
(fun {split n l} {list (take n l) (drop n l)})
```

```
; Element of List
(fun {elem x l} {
  if (== l nil)
    {false}
    {if (== x (fst l)) {true} {elem x (tail l)}}
})
```

Все эти функции устроены похоже. Было бы здорово выделить общий шаблон, чтобы не писать его каждый раз. Например, может понадобиться применить функцию к каждому элементу списка. Для этого определим `map`. Она принимает функцию и список, применяет `f` к первому элементу, добавляет результат в начало нового списка и вызывает `map` для хвоста.

```
; Apply Function to List
(fun {map f l} {
  if (== l nil)
    {nil}
    {join (list (f (fst l))) (map f (tail l))}
})
```

Теперь можно делать вещи, похожие на циклы. В некотором смысле этот подход мощнее: вместо того чтобы думать о последовательном применении функции к каждому элементу, можно мыслить действием над всеми элементами сразу. Мы *отображаем список*, а не *меняем каждый элемент*.

```
lispy> map - {5 6 7 8 2 22 44}
{-5 -6 -7 -8 -2 -22 -44}
lispy> map (\ {x} {+ x 10}) {5 2 11}
{15 12 21}
lispy> print {"hello" "world"}
{"hello" "world"}
()
lispy> map print {"hello" "world"}
"hello"
"world"
{() ()}
lispy>
```

Развитием этой идеи является функция `filter`: она принимает функцию-условие и оставляет в списке только элементы, которые ему соответствуют.

```
; Apply Filter to List
(fun {filter f l} {
  if (== l nil)
    {nil}
    {join (if (f (fst l)) {head l} {nil}) (filter f (tail l))}
})
```

Вот как это выглядит на практике.

```
lispy> filter (\ {x} {> x 2}) {5 2 11 -7 8 1}
{5 11 8}
```

Некоторые циклы не преобразуют список, а накапливают итог или сводят список к одному значению. Так работают, например, вычисления суммы и произведения. Их можно выразить способом, похожим на уже определённую функцию `len`.

Это называется *свёрткой*. Получив функцию `f`, *начальное значение* `z` и список `l`, свёртка объединяет каждый элемент списка с накопленным результатом, начиная с начального значения.

```
; Fold Left
(fun {foldl f z l} {
  if (== l nil)
    {z}
    {foldl f (f z (fst l)) (tail l)}
})
```

С помощью свёрток можно очень элегантно определить функции `sum` и `product`.

```
(fun {sum l} {foldl + 0 l})
(fun {product l} {foldl * 1 l})
```

Условные функции

Определив `fun`, мы уже показали, насколько мощен наш язык: в нём можно создавать функции, похожие на новые синтаксические конструкции. Другой пример — имитация операторов C `switch` и `case`. В C они встроены в язык, а у нас могут быть частью библиотеки.

Определим функцию `select`, которая принимает ноль или более списков из двух элементов. Для каждого такого списка она сначала вычисляет первый элемент пары. Если он истинен, функция вычисляет и возвращает второй; иначе переходит к оставшимся парам.

```
(fun {select & cs} {
  if (== cs nil)
    {error "No Selection Found"}
    {if (fst (fst cs)) {snd (fst cs)} {unpack select (tail cs)}}
})
```

Можно также определить функцию `otherwise`, которая всегда возвращает `true`. Она напоминает ключевое слово `default` в C.

```
; Default Case
(def {otherwise} true)

; Print Day of Month suffix
(fun {month-day-suffix i} {
  select
    {(== i 0)  "st"}
    {(== i 1)  "nd"}
    {(== i 3)  "rd"}
    {otherwise "th"}
})
```

На самом деле это мощнее оператора C `switch`: в C входное значение сравнивается только на равенство с набором констант, а условия не передаются. Такую функцию сравнения тоже можно определить в нашем Lisp. Она принимает значение `x`, за которым следуют списки из двух элементов. Если первый элемент пары равен `x`, вычисляется второй; иначе поиск продолжается.

```
(fun {case x & cs} {
  if (== cs nil)
    {error "No Case Found"}
    {if (== x (fst (fst cs))) {snd (fst cs)} {
      unpack case (join (list x) (tail cs))}}
})
```

Синтаксис этой функции получается простым и удобным. Подумайте, какие ещё управляющие конструкции или полезные функции можно реализовать подобным способом.

```
(fun {day-name x} {
  case x
    {0 "Monday"}
    {1 "Tuesday"}
    {2 "Wednesday"}
    {3 "Thursday"}
    {4 "Friday"}
    {5 "Saturday"}
```

```
    {6 "Sunday"}
})
```

Фибоначчи

Ни одна стандартная библиотека не обходится без обязательного определения функции Фибоначчи. Используя всё, что мы уже создали, напишем небольшую функцию `fib`: она получается достаточно читаемой и понятной по смыслу.

```
; Fibonacci
(fun {fib n} {
  select
    { (== n 0) 0 }
    { (== n 1) 1 }
    { otherwise (+ (fib (- n 1)) (fib (- n 2))) }
})
```

На этом моя стандартная библиотека заканчивается. Создание стандартной библиотеки — увлекательная часть разработки языка: можно проявить творчество и решить, что включить, а что оставить за её пределами. Попробуйте создать то, что понравится именно вам. Исследовать возможности языка бывает очень интересно.

Справка

Исходный код: prelude.lspy.

Дополнительные задания

- › Перепишите функцию `len` с использованием `foldl`.
- › Перепишите функцию `elem` с использованием `foldl`.
- › Включите стандартную библиотеку непосредственно в язык и загружайте её при запуске.
- › Напишите документацию к стандартной библиотеке с объяснением каждой функции.
- › Напишите примеры программ с использованием библиотеки, чтобы пользователи могли по ним учиться.
- › Напишите тесты для каждой функции стандартной библиотеки.

Дополнительные проекты • Глава 16

Это только начало

Хотя мы многое сделали для нашего Лиспа, до полноценного языка программирования, готового к промышленному применению, ему ещё далеко. В достаточно крупном проекте вы рано или поздно столкнётесь с рядом проблем и необходимостью улучшений. Решение этих проблем приблизит его к настоящему полнофункциональному языку.

Ниже перечислены некоторые трудности, с которыми вы можете встретиться, возможные решения и другие интересные идеи для улучшения. Некоторые потребуют нескольких сотен строк кода, другие — нескольких тысяч. Что именно реализовать, решать вам. Если вы привязались к своему языку, работа над этими проектами может доставить вам удовольствие.

Встроенные типы

Сейчас наш язык лишь оборачивает встроенные типы C `long` и `char*`. Это довольно сильное ограничение для полезных вычислений. Операции с этими типами данных тоже весьма ограниченны. В идеале язык должен оборачивать все встроенные типы C и предоставлять способы работы с ними. Одно из важнейших дополнений — работа с дробными числами. Для этого можно обернуть тип `double` и соответствующие операции. Если числовых типов несколько, нужно убедиться, что арифметические операторы вроде `+` и `-` работают с каждым из них и с их сочетаниями.

Добавление встроенных типов будет интересно тем, кто хочет выполнять в своём языке вычисления с дробными числами и числами с плавающей точкой.

Пользовательские типы

Помимо поддержки встроенных типов, полезно дать пользователям возможность добавлять собственные новые типы, как мы используем структуры в C. Синтаксис и способ реализации выбирайте сами. Это важнейшая часть превращения нашего языка в инструмент, пригодный для достаточно крупных проектов.

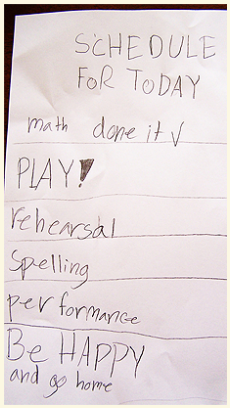
Эта задача может заинтересовать тех, кто уже представляет, как хочет развивать язык и каким видит его окончательный облик.

Литералы списков

В некоторых диалектах Лиспа квадратные скобки `[]` обозначают литерал списка вычисленных значений. Это синтаксический сахар: вместо `list 100 (+ 10 20) 300` можно

написать `[100 (+ 10 20) 300]`. Иногда так явно удобнее, хотя символы `[]` при этом уже нельзя будет использовать для чего-то более интересного.

Это несложное дополнение подойдёт тем, кто хочет попробовать расширить синтаксис.



Важный список • Играй! РАДУЙСЯ и иди домой.

Взаимодействие с операционной системой

Важная часть создания самостоятельного языка — полноценная работа с файлами: открытие, чтение и запись. Для этого нужно обернуть функции C, такие как `fread`, `fwrite`, `fgetc` и другие, их аналогами на Лиспе. Задача довольно проста, но потребует большого числа функций-обёрток. Поэтому до сих пор мы её не решали.

Также было бы здорово предоставить языку доступ к подходящим системным вызовам: возможность менять каталог, получать список файлов и тому подобное. Это несложно, но опять же требует множества обёрток над функциями C. Без этого практически невозможно использовать язык для написания сценариев.

Проект может заинтересовать тех, кто хочет применять свой язык для простых сценариев и обработки строк.

Макросы

Во многих других диалектах Лиспа можно написать `(def x 100)`, чтобы присвоить `x` значение `100`. В нашем Лиспе это не сработает: он попытается вычислить `x` и получить значение, хранящееся под этим именем в окружении. В других диалектах такие функции называются *макросами*: они предотвращают вычисление аргументов и обрабатывают их в исходном виде. Благодаря им можно писать конструкции, похожие на обычные вызовы функций, которые на самом деле выполняют сложные и интересные действия.

Макросы делают язык увлекательнее: они позволяют добавить в его работу немного *магии*. Во многих случаях благодаря им синтаксис становится удобнее, а пользователю не приходится повторяться.

Мне нравится, как наш язык обрабатывает `def` и `if` без макросов. Но если вам не нравится нынешнее поведение и хочется приблизить язык к обычным диалектам Лиспа, возможно, вы захотите реализовать макросы.

Хеш-таблица переменных

Сейчас для поиска переменной мы просто последовательно перебираем все переменные окружения. Чем больше их определено, тем менее эффективным становится поиск.

Более эффективный способ — реализовать *хеш-таблицу*. При этом имя переменной преобразуется в целое число, которое используется как индекс массива известного размера для поиска связанного с символом значения. Это очень важная структура данных: благодаря превосходной производительности при высокой нагрузке она встречается повсюду.

Всем, кто хочет больше узнать о структурах данных и алгоритмах, стоит попробовать реализовать эту структуру или один из её вариантов.

Выделение памяти из пула

Наш Лисп очень прост, но не быстр. Его производительность сравнима с некоторыми языками сценариев, такими как Python и Ruby. Большая часть накладных расходов связана с тем, что почти любое действие требует создания и уничтожения `lval`. Поэтому мы очень часто вызываем `malloc`. Эта функция работает медленно, поскольку требует управления памятью со стороны операционной системы. Во время вычислений происходит множество копирований, выделений и освобождений объектов `lval`.

Чтобы уменьшить накладные расходы, нужно сократить число вызовов `malloc`. Один из способов — вызвать `malloc` один раз в начале программы и выделить большой пул памяти. Затем заменить все вызовы `malloc` обращениями к функции, которая будет распределять части этого пула для нужд программы. Так мы воспроизводим часть работы операционной системы, но быстрее и локально. Этот подход называют *выделением памяти из пула*; он распространён в разработке игр и других приложений, чувствительных к производительности.

Реализовать его правильно бывает непросто, хотя сама идея не обязательно сложна. Если вам нужен относительно быстрый способ значительно повысить производительность, обратите на него внимание.

Сборка мусора

Почти во всех других реализациях Лиспа переменные присваиваются иначе. В окружении хранится не копия значения, а указатель или ссылка на него. Благодаря указателям, как и в C, работа с большими структурами данных требует гораздо меньше накладных расходов.

Если хранить указатели на значения, а не копии, нужно гарантировать, что данные не будут удалены прежде, чем ими воспользуется кто-то ещё. Удалять их следует, когда ссылок на них больше не осталось. Один из способов, называемый *пометкой и очисткой*, состоит в отслеживании значений в окружении и всех выделенных объектов. Когда переменная помещается в окружение, она и всё, на что она ссылается, *помечается*. Затем, когда понадобится освободить память, можно перебрать все выделенные объекты и удалить непомеченные.



Сборка мусора • Поднимите эту банку.

Это называется *сборкой мусора* и является неотъемлемой частью многих языков программирования. Как и выделение из пула, реализация *сборщика мусора* не обязательно сложна, но требует внимательности, особенно если важна эффективность. Такое дополнение необходимо, чтобы язык был пригоден для работы с большими объёмами данных. Очень хороший учебный материал о реализации сборщика мусора на C можно найти здесь.

Эта задача заинтересует тех, кто заботится о производительности языка и хочет изменить правила хранения и изменения переменных.

Оптимизация хвостовых вызовов

Наш язык использует для циклов *рекурсию*. Это очень изящная идея, но на практике она довольно неэффективна. Рекурсивные функции вызывают сами себя, собирая промежуточные результаты вычислений, и лишь затем объединяют их. Это расточительно, если промежуточный результат можно накапливать в цикле. Особенно проблематичны циклы с большим или бесконечным числом повторений.

Некоторые рекурсивные функции можно автоматически преобразовать в соответствующие циклы `while`, которые накапливают результат постепенно. Это преобразование называется *оптимизацией хвостовых вызовов* и необходимо программам, которые часто используют рекурсию для организации циклов.

Проект может заинтересовать тех, кто изучает оптимизацию компиляторов и соответствия между различными формами вычислений.

Лексическая область видимости

Когда наш язык пытается найти неопределённую переменную, он выдаёт ошибку. Лучше было бы сообщать о таких переменных ещё до вычисления программы. Это помогло бы избежать опечаток и других досадных ошибок. Автор называет такой предварительный анализ *лексической областью видимости*: он использует правила определения переменных, чтобы установить, какие имена определены в каждой точке программы, не выполняя её.

Реализовать всё безупречно может быть трудно, но задача заинтересует тех, кто хочет сделать свой язык безопаснее и уменьшить число ошибок.

Статическая типизация



Статическое электричество • Альтернатива, от которой волосы встают дыбом.

Каждое значение в программе имеет тип, известный ещё до вычислений. Наши встроенные функции тоже принимают на вход только определённые типы. Мы должны суметь использовать эту информацию для вывода типов новых пользовательских функций и значений, а также для проверки типов аргументов ещё до запуска программы. Это поможет заранее обнаруживать ошибки, связанные с вызовом функций с неподходящими типами. Такая проверка называется *статической типизацией*.

Системы типов — увлекательная и фундаментальная часть информатики. Сегодня это лучший известный нам способ выявлять ошибки до запуска программы. Проект должен заинтересовать всех, кто изучает безопасность языков программирования и системы типов.

Заключение

Большое спасибо за чтение этой книги. Надеюсь, вы нашли на её страницах что-то интересное. Если книга вам понравилась, расскажите о ней друзьям. Если вы собираетесь продолжить разработку своего языка, желаю удачи и надеюсь, что вы узнаете ещё много нового о C, языках программирования и информатике.

А больше всего я надеюсь, что вам было интересно создавать собственный Лисп. До новых встреч!

Парсер своими руками • Приложение A

Делаем сами



Дорожный каток • Теперь мы сами всё раскатаем

В этой книге весь разбор выполнялся с помощью `mpc`, но в приложении мы заменим её собственным парсером. Именно выбор `mpc` вызвал больше всего жалоб читателей, и прежде чем переходить к подробностям самостоятельной реализации, я хочу рассказать, почему вообще решил использовать библиотеку.

Главная причина: когда я изучал языки программирования, теория формальных языков меня увлекла. Мне очень понравилось учиться мыслить о языках абстрактно. Я считаю, что этому интересно учить, поскольку открывается множество новых направлений для исследований.

Другая причина — возможность для начинающих научиться пользоваться библиотекой и с самого начала привыкнуть к непривычным интерфейсам и чужому коду.

Наконец, возможно, самое важное: библиотека позволила отложить темы управления памятью и указателей как можно дольше. К тому времени читатели должны были освоиться с уже встреченными конструкциями C.

Но всё это не так уж важно: написать парсер самостоятельно — отличная затея. Полнофункциональный парсер может быть сложным, но наш Lisp настолько прост, что нам будет не слишком трудно.

Можно ли выполнить приложение, не закончив книгу?

Если вы не прошли все главы, браться за приложение, вероятно, не стоит. Возможно, вы справитесь после главы 9 о S-выражениях, но до неё многое может оказаться непонятным. Извините!

Заменяем mpc

Результатом работы `mpc` было абстрактное синтаксическое дерево. Но наш язык настолько прост, что мы перейдём прямо от входной строки к S-выражению, которое и само во многом похоже на такое дерево. Объявим функцию `lval_read_expr`: она принимает строку `s`, указатель на позицию в ней `i` и завершающий символ `end`, а на выходе формирует `lval`.

```
lval* lval_read_expr(char* s, int* i, char end);
```

Реализацию определим позже. Пока заменим вызовы `mpc` новой функцией. Для обработки ошибок `lval_read_expr` будет возвращать значение `lval` с ошибкой. При чтении строки С завершающий символ — нулевой символ `\0`.

```
char* input = readline("lispy> ");
/* Read from input to create an S-Expr */
int pos = 0;
lval* expr = lval_read_expr(input, &pos, '\0');

/* Evaluate and print input */
lval* x = lval_eval(e, expr);
```

Нужно заменить `mpc` и в `builtin_load`. Поскольку наша функция читает только строки, сначала загрузим содержимое файла. С помощью `fseek` и `ftell` определим количество символов, затем выделим строку нужного размера плюс один байт и прочитаем файл.

```
lval* builtin_load(lenv* e, lval* a) {
  LASSERT_NUM("load", a, 1);
  LASSERT_TYPE("load", a, 0, LVAL_STR);

  /* Open file and check it exists */
  FILE* f = fopen(a->cell[0]->str, "rb");
  if (f == NULL) {
    lval* err = lval_err("Could not load Library %s", a->cell[0]->str);
    lval_del(a);
    return err;
  }

  /* Read File Contents */
  fseek(f, 0, SEEK_END);
  long length = ftell(f);
  fseek(f, 0, SEEK_SET);
  char* input = calloc(length+1, 1);
  fread(input, 1, length, f);
  fclose(f);
```

После этого можно прочитать строку, как в `main`:

```
  /* Read from input to create an S-Expr */
  int pos = 0;
  lval* expr = lval_read_expr(input, &pos, '\0');
  free(input);

  /* Evaluate all expressions contained in S-Expr */
  if (expr->type != LVAL_ERR) {
    while (expr->count) {
      lval* x = lval_eval(e, lval_pop(expr, 0));
      if (x->type == LVAL_ERR) { lval_println(x); }
      lval_del(x);
    }
  } else {
    lval_println(expr);
  }

  lval_del(expr);
  lval_del(a);

  return lval_sexpr();
}
```

Заменив вызовы, приступим к определению `lval_read_expr`.

По одному символу



Чтение • Весьма приятное занятие

Реализация парсера требует иного взгляда, чем высокоуровневое абстрактное описание с `mpc`: вместо языка мы думаем о процессе.

Обычно процесс устроен очень просто: парсер почти всегда представляет собой цикл, который читает ввод по одному символу и каждый раз решает, что с ним делать. Трудность — сделать процесс изящным. Пробелы, комментарии и прочие детали быстро его усложняют.

Например, встретив во вводе символ `d`, мы можем сохранить его в строке. Мы знаем, что читаем символ языка, поэтому переходим в состояние поиска следующих букв, добавляя их в строку. Когда буквы закончатся, возвращаем результат как символ, например `def`, и начинаем заново.

Примерно так будет работать `lval_read_expr`: получив строку и позицию, функция решает, что делать дальше. Пока следующий символ не совпадает с `end`, она читает очередной элемент, создаёт из него `lval` и добавляет к первому аргументу `v`.

Достигнув символа `end`, она возвращает следующую позицию вызывающей функции. По ней вызывающий код узнает, сколько строки уже обработано и сколько осталось.

Пусть следующий символ пока не `end`. Сначала проверим, не закончился ли ввод. Если конец достигнут раньше завершающего символа, выдадим синтаксическую ошибку и перейдём к концу ввода, чтобы не читать дальше.

```
int lval_read_expr(lval* v, char* s, int i, char end) {

  while (s[i] != end) {

    /* If we reach end of input then there is some syntax error */
    if (s[i] == '\0') {
      lval_add(v, lval_err("Missing %c at end of input", end));
      return strlen(s)+1;
    }
```

Затем проверим пробельные символы. Наш язык не чувствителен к пробелам, поэтому их можно пропускать.

```
    /* Skip all whitespace */
    if (strchr(" \t\v\r\n", s[i])) {
      i++;
      continue;
    }
```

Другой простой случай — точка с запятой `;`. Она начинает комментарий, поэтому можно пропустить всё до следующего перевода строки.

```
/* If next char is ; then read comment */
if (s[i] == ';') {
  while (s[i] != '\n' && s[i] != '\0') { i++; }
  i++;
  continue;
}
```

Открывающая круглая `(` или фигурная `{` скобка начинает S- или Q-выражение. Снова вызовем `lval_read_expr`, передав другой завершающий символ и другое выражение для записи результата.

```
/* If next character is ( then read S-Expr */
if (s[i] == '(') {
  lval* x = lval_sexpr();
  lval_add(v, x);
  i = lval_read_expr(x, s, i+1, ')');
  continue;
}

/* If next character is { then read Q-Expr */
if (s[i] == '{') {
  lval* x = lval_qexpr();
  lval_add(v, x);
  i = lval_read_expr(x, s, i+1, '}');
  continue;
}
```

С простыми случаями закончено. Для буквы или цифры нужно прочитать последовательность таких символов до первого неподходящего. Для простоты будем считать числа частным случаем символов и вызывать `lval_read_sym`, которую определим позже.

```
/* If next character is part of a symbol then read symbol */
if (strchr(
  "abcdefghijklmnopqrstuvwxyz"
  "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
  "0123456789_+-*\\/=<>!&", s[i])) {
  i = lval_read_sym(v, s, i);
  continue;
}
```

Нужно обработать и строки. Встретив `"`, читаем всё до следующей неэкранированной кавычки `"`. Для этого вызовем функцию `lval_read_str`, определённую позже.

```
/* If next character is " then read string */
if (strchr("\"", s[i])) {
  i = lval_read_str(v, s, i+1);
  continue;
}
```

Если встретилось что-то ещё, выдадим ошибку и перейдём в конец ввода. А если встретился символ `end` и цикл завершился, вернём обновлённую позицию.

```
/* Encountered some unknown character */
lval_add(v, lval_err("Unknown Character %c", s[i]));
```

```
    return strlen(s)+1;
  }

  return i+1;
}
```

Тело `lval_read_expr` готово. Осталось заполнить пробелы.

Чтение символов

Чтение символов довольно просто. Сначала выделяем пустую строку, затем добавляем в неё каждый входной символ, допустимый в составе символа языка.

```
int lval_read_sym(lval* v, char* s, int i) {

  /* Allocate Empty String */
  char* part = calloc(1,1);

  /* While valid identifier characters */
  while (strchr(
      "abcdefghijklmnopqrstuvwxyz"
      "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
      "0123456789_+-*\\/=<>!&", s[i]) && s[i] != '\0') {

    /* Append character to end of string */
    part = realloc(part, strlen(part)+2);
    part[strlen(part)+1] = '\0';
    part[strlen(part)+0] = s[i];
    i++;
  }
```

Этот код читает и числа. Поэтому далее проверим, не является ли прочитанный символ числом, — для этого убедимся, что все его знаки являются цифрами.

```
  /* Check if Identifier looks like number */
  int is_num = strchr("-0123456789", part[0]);
  for (int i = 1; i < strlen(part); i++) {
    if (!strchr("0123456789", part[i])) { is_num = 0; break; }
  }
```

Если это число, преобразуем его кодом, похожим на тот, что использовался при чтении дерева `mpc`; иначе используем обычный символ. Затем освободим временную строку через `free` и вернём новую позицию.

```
  /* Add Symbol or Number as lval */
  if (is_num) {
    errno = 0;
    long x = strtol(part, NULL, 10);
    lval_add(v, errno != ERANGE ? lval_num(x) : lval_err("Invalid Number %s", part));
  } else {
    lval_add(v, lval_sym(part));
  }

  /* Free temp string */
  free(part);

  /* Return updated position in input */
```

```
  return i;
}
```

С символами покончено. Переходим к строкам!

Чтение строк

Со строками несколько сложнее из-за *экранирования*. Мы немного касались его ранее, а теперь разберёмся вплотную.

Экранирование позволяет вводить специальные символы сочетаниями, начинающимися с обратной косой черты `\`. Самый известный пример — перевод строки `\n`. Преобразование такой записи в настоящий символ называется снятием экранирования, а обратное преобразование — экранированием.

При чтении пользовательской строки нужно преобразовать эти пары в соответствующие специальные символы. Игнорируя начальную обратную косую черту, можно написать функцию C, задающую это соответствие:

```
/* Function to unescape characters */
char lval_str_unescape(char x) {
  switch (x) {
    case 'a':  return '\a';
    case 'b':  return '\b';
    case 'f':  return '\f';
    case 'n':  return '\n';
    case 'r':  return '\r';
    case 't':  return '\t';
    case 'v':  return '\v';
    case '\\': return '\\';
    case '\'': return '\'';
    case '\"': return '\"';
  }
  return '\0';
}
```

Также полезно перечислить все символы, для которых поддерживается снятие экранирования, чтобы проверять их:

```
/* Possible unescapable characters */
char* lval_str_unescapable = "abfnrtv\\\'\"";
```

Аналогично напишем функции для обратного преобразования:

```
/* List of possible escapable characters */
char* lval_str_escapable = "\a\b\f\n\r\t\v\\\'\"";
```

```
/* Function to escape characters */
char* lval_str_escape(char x) {
  switch (x) {
    case '\a': return "\\a";
    case '\b': return "\\b";
    case '\f': return "\\f";
    case '\n': return "\\n";
    case '\r': return "\\r";
```

```
    case '\t': return "\\t";
    case '\v': return "\\v";
    case '\\': return "\\\\";
    case '\'': return "\\\'";
    case '\"': return "\\\"";
  }
  return "";
}
```

Теперь начнём писать функцию чтения строк. Сначала выделим временную строку и будем обрабатывать входные символы до завершающей кавычки ".

```
int lval_read_str(lval* v, char* s, int i) {

  /* Allocate empty string */
  char* part = calloc(1,1);

  while (s[i] != '"') {

    char c = s[i];
```

Сначала проверим конец ввода: если он наступил, строка не была закрыта. Освободим временную строку через `free` и вернём ошибку.

```
    /* If end of input then there is an unterminated string literal */
    if (c == '\0') {
      lval_add(v, lval_err("Unexpected end of input at string literal"));
      free(part);
      return strlen(s);
    }
```

Затем проверим, не является ли следующий символ обратной косой чертой. Если является, нужно обработать следующий за ней символ. С ранее определёнными функциями это легко: если снятие экранирования поддерживается, выполняем его, иначе выдаём ошибку.

```
    /* If backslash then unescape character after it */
    if (c == '\\') {
      i++;
      /* Check next character is escapable */
      if (strchr(lval_str_unescapable, s[i])) {
        c = lval_str_unescape(s[i]);
      } else {
        lval_add(v, lval_err("Invalid escape character %c", c));
        free(part);
        return strlen(s);
      }
    }
```

Полученный специальный или обычный символ добавляем во временную строку. Закончив чтение, преобразуем её в `lval`, добавляем к аргументу `v`, освобождаем временную строку и возвращаемся.

```


    /* Append character to string */
    part = realloc(part, strlen(part)+2);
    part[strlen(part)+1] = '\0';
    part[strlen(part)+0] = c;
    i++;
  }

  /* Add lval and free temp string */
```

```
  lval_add(v, lval_str(part));

  free(part);

  return i+1;
}
```

Вывод строк

Помимо снятия экранирования при чтении, мы использовали mpc для экранирования строк при выводе. Если мы заменяем все применения mpc, нужно сделать это и здесь. Наши функции преобразования помогут: переберём символы строки, экранируя подходящие и выводя остальные как есть.

```
void lval_print_str(lval* v) {
  putchar('"');
  /* Loop over the characters in the string */
  for (int i = 0; i < strlen(v->str); i++) {
    if (strchr(lval_str_escapable, v->str[i])) {
      /* If the character is escapable then escape it */
      printf("%s", lval_str_escape(v->str[i]));
    } else {
      /* Otherwise print character as it is */
      putchar(v->str[i]);
    }
  }
  putchar('"');
}
```

Уборка

Теперь наконец можно удалить подключение mpc. Вместо него понадобится подключить некоторые библиотеки, которые mpc подключала за нас. Начало файла будет выглядеть так:

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <errno.h>
```

Также нужно удалить предварительные объявления парсеров:

```
/* Parser Declariations */

mpc_parser_t* Number;
mpc_parser_t* Symbol;
mpc_parser_t* String;
mpc_parser_t* Comment;
mpc_parser_t* Sexpr;
mpc_parser_t* Qexpr;
mpc_parser_t* Expr;
mpc_parser_t* Lispy;
```

Вот теперь всё готово! Пользуйтесь своим собственноручно написанным парсером.

Справочные материалы

Исходный код: hand_rolled_parser.c.

Дополнительные задания

- › Добавьте в сообщения о синтаксических ошибках номер строки и столбца.